Fri, Oct 2 at 5:09 PM

DP_Csestudy_instr... PDF

DP_Csestudy(2).pdf PDF

You are the primary research and software-analysis assistant for a university Design Patterns Case Study.

CASE STUDY

Course: 23CSE455 — Design Patterns
Case Study Title:
Design Pattern Analysis and Evolution in a Ruby on Rails Hotel Booking Application
Source application:
https://github.com/andrewdsilva/rails-room-booking
The source repository is the system under investigation. Do not replace it with another hotel-booking application or construct a hypothetical Rails application unless explicitly requested.
The supplied faculty instructions and report template are authoritative for the required outputs.

CORE OBJECTIVE

Perform a rigorous, evidence-based analysis of the selected Ruby on Rails application and subsequently evolve it through controlled changes and implement selected confirmed patterns in Java, Python, JavaScript and C++.
The final work must be defensible to a professor reviewing the repository, source code, UML diagrams, CSV datasets, commits, test evidence, generated code and LLM records.

NON-NEGOTIABLE EVIDENCE RULES

1. Never claim that a design pattern exists solely because a class or method resembles a pattern.
2. A pattern may be recorded as CONFIRMED only when the actual source code demonstrates the required collaboration, roles, intent and behavior.
3. For every confirmed pattern:
    - identify the exact pattern name;
    - identify its category;
    - give a meaningful instance name;

- map every applicable participant role to the actual class/module/function;
- provide exact file paths;
- provide exact source line ranges;
- explain the interactions/delegation/calls that establish the pattern;
- distinguish application code from Rails/framework/dependency behavior;
- provide verification evidence.

4. Do not invent classes, interfaces, methods, relationships, files, line numbers, tests, commits or pattern roles.
5. If a required role is combined with another role, explicitly state that rather than inventing a separate class.
6. If a candidate is uncertain, label it UNCERTAIN and explain precisely what evidence is missing.
7. If a candidate is rejected, record the candidate and explain why it does not satisfy the pattern.
8. LLM-generated claims are hypotheses until verified against source code.
9. Never fabricate execution results, test results, runtime measurements, complexity measurements or compiler versions.
10. Never estimate a missing runtime measurement. Use "not_run" or "cannot_measure" with the reason.
11. Preserve source-version identity throughout the analysis.
12. Never silently modify the original source while performing Part 1 analysis.
13. Part 2 changes must be reproducible from the original baseline through explicitly recorded changes.
14. Keep baseline and evolved versions distinguishable.
15. When source evidence changes, update the corresponding pattern/evolution record rather than creating duplicate observations of the same collaboration.

SOURCE-VERSION RULE

First establish the exact baseline commit.
Use:
source_version = original
After evolution, use:

source_version = Change1
source_version = Change2
source_version = Change3
Do not use vague labels such as "before" or "after" when a commit/version can be identified.

PATTERN SCOPE

Investigate relevant:

- GoF Creational patterns
- GoF Structural patterns
- GoF Behavioral patterns
- GRASP patterns
- relevant architectural patterns
- relevant framework idioms only when they materially affect the design analysis

Do not force a pattern quota.
The faculty instructions explicitly state that there is no minimum pattern quota.

DOMAIN SCOPE

At minimum investigate:

- hotel/property
- room/accommodation
- customer/user
- reservation/booking
- payment
- cancellation
- additional services
- availability
- pricing
- notifications/webhooks
- authentication/authorization
- persistence
- external integrations

Only include entities actually present in the repository.

SOURCE HIERARCHY

Use the following evidence priority:

1. Actual repository source code
2. Repository tests
3. Repository configuration/dependency files
4. Git history and commits
5. Official framework/dependency documentation
6. Academic/design-pattern references
7. LLM reasoning

An LLM assertion alone is never sufficient evidence.

OUTPUT STYLE

For every analysis:

1. State what was inspected.
2. State what was found.
3. Separate confirmed, rejected and uncertain findings.
4. Cite exact source locations.
5. Explain the reasoning.
6. Identify missing evidence.
7. Do not hide uncertainty.

Use tables where useful.
Do not write polished final-report prose unless explicitly requested. Prefer structured research artifacts that can later be incorporated into the report.

IMPORTANT

The final submission requires:

- application architecture
- pattern inventory
- UML
- pattern evidence
- evolution scenarios
- refactoring
- four-language implementations
- metrics
- LLM prompts/results
- CSV datasets

Therefore, throughout the project, preserve the evidence required to generate these artifacts later.

Begin by acknowledging this operating contract and wait for the next research prompt.

Understood. I'll use this as the operating contract for the entire Design Patterns case study.

The **authoritative basis** will be the supplied faculty instructions/template and the actual `andrewdsilva/rails-room-booking` repository. I'll preserve exact source-version identity, distinguish confirmed/rejected/uncertain findings, and require source-level evidence before marking a pattern as confirmed. DP_Csestudy_instruct

I'll also preserve the required evidence structure for `patterns.csv`, `pattern_counts.csv`, `patterns_evolution.csv`, `cross_language.csv`, and `llm.csv`, including exact participant mappings, source locations, verification evidence, reproducible changes, and honest execution/measurement records. DP_Csestudy_instruct DP_Csestudy_instruct

The report structure will likewise remain aligned with the supplied template: architecture and baseline identification, Part 1 discovery, Part 2 evolution/refactoring, Part 3 four-language comparison, LLM verification, and evidence-backed findings. DP_Csestudy DP_Csestudy

I'll wait for your next research prompt.

Analyze the repository:
https://github.com/andrewdsilva/rails-room-booking:chatgpt-file-mention-r46v{kind="system-hint" index="0"}
This is the source application for our 23CSE455 Design Patterns case study.
Your task is to establish the reproducible baseline for all subsequent analysis.

REQUIREMENTS

Inspect the repository directly.
Determine:

1. Repository owner/name
2. Default branch
3. Current HEAD commit
4. Commit date
5. Repository tags/releases if relevant
6. Relevant recent commit history
7. Ruby version
8. Rails version

9. Database technology/configuration
10. Major dependencies relevant to the case study
11. Test framework
12. Test commands
13. Lint/static-analysis tools
14. Application startup commands
15. Docker configuration if applicable
16. Relevant environment/configuration requirements
17. License
18. Main application directories
19. Models
20. Controllers
21. Services/modules
22. Jobs/mailers/integrations
23. Tests/specifications
24. Database schema/migrations
25. Views/components relevant to the booking domain

SOURCE-VERSION REQUIREMENT

Record the exact commit SHA that will become:
source_version = original
Do not use the moving main branch as the sole baseline identifier.

SCOPE

Classify repository files into:

- inspected
- partially inspected
- excluded

For every exclusion, explain why it is outside the design-pattern analysis scope.

OUTPUT

Produce:
A. Baseline identification table
B. Environment table
C. Repository/module inventory
D. Initial domain model inventory
E. Test inventory
F. Reproducibility checklist

G. Proposed exact baseline source_version identifier
H. Risks/limitations affecting later analysis
Do not identify design patterns yet.
Do not infer patterns from filenames.
Do not modify the repository.
Every factual statement about the repository must be traceable to an actual repository file, commit or official source.

Worked for 1m 14s

Baseline Reproducibility Record — `rails-room-booking`

Repository inspected: `andrewdsilva/rails-room-booking`
Inspection basis: GitHub repository contents, Git history, dependency lockfile, configuration, application source, tests, schema/migrations, and supplied faculty instructions.
Pattern analysis: intentionally **not performed** in this baseline task.

A key point for the case study: the repository's current default branch is `main`, but the reproducible baseline below is pinned to the exact HEAD commit rather than the moving branch.

A. Baseline identification table

Item	Baseline finding	Evidence
Repository owner	`andrewdsilva`	GitHub repository metadata
Repository name	`rails-room-booking`	GitHub repository metadata
Full repository	`andrewdsilva/rails-room-booking`	GitHub repository metadata
Visibility	Public	GitHub repository metadata
Default branch	`main`	Repository metadata
Current HEAD on `main`	`5e15551ee646871b4bed32c982fa17da319d3ee3`	Latest repository commit
HEAD commit date	2023-07-16 04:45:38 UTC	Git commit metadata
HEAD commit message	`Merge pull request #7 from bretheskevin/docker-sql / Add PostgreSql database with docker-compose.`	Git commit metadata

Item	Baseline finding	Evidence
Relevant latest change	PostgreSQL + Docker/Docker Compose support	HEAD diff
Repository releases	None returned	GitHub `/releases`
Repository tags	No tag result was established from the inspection available	Not sufficient to claim tags exist or do not exist
License metadata	GitHub API reports `license: null`; README contains an MIT license badge/link	Repository metadata + README
Source-version identity	immutable commit, not `main`	Required by case-study instructions

The selected HEAD is particularly important because commit `5e15551...` is the commit that introduced the PostgreSQL/Docker setup and is the current `main` HEAD. Its diff explicitly changes the application from SQLite to PostgreSQL, adds database environment variables, and updates Docker configuration. Therefore that commit, rather than an earlier SQLite commit, is the appropriate current repository baseline.

Proposed immutable baseline:

```
Plain text
source_version = original
baseline_commit = 5e15551ee646871b4bed32c982fa17da319d3ee3
baseline_ref = main@5e15551ee646871b4bed32c982fa17da319d3ee3
```

The faculty instructions explicitly require the exact baseline commit and prohibit relying on a moving branch alone. DP_Csestudy_instruct

B. Environment table

Item	Observed value	Source
Language	Ruby	Repository metadata
Ruby declared version	`2.7.5`	`Gemfile`
Ruby locked version	`ruby 2.7.5p203`	`Gemfile.lock`

Item	Observed value	Source
Rails declared constraint	`~> 7.0.4`	`Gemfile`
Rails locked version	`7.0.6`	`Gemfile.lock`
Bundler	`2.4.17`	`Gemfile.lock`
Database adapter	PostgreSQL	`Gemfile` , `config/database.yml`
PostgreSQL adapter gem	`pg 1.5.3`	`Gemfile.lock`
Production/development/test DBs	`rails_room_booking_development` , `rails_room_booking_test` , `rails_room_booking_production`	`config/database.yml`
Web server	Puma	`Gemfile` , lockfile
Puma locked version	`5.6.6`	`Gemfile.lock`
Test framework	RSpec / RSpec Rails	`Gemfile` , `spec/rails_helper.rb`
RSpec Rails	`6.0.3`	`Gemfile.lock`
System test stack	Capybara + Selenium WebDriver + Webdrivers	`Gemfile` , lockfile
Authorization	Pundit `2.3.0`	`Gemfile.lock`
Authentication	Devise `4.9.2`	`Gemfile.lock`
Payments	Payify `0.1.18` , Stripe `8.5.0`	`Gemfile.lock`
External automation/webhook	Integromat `0.1.0`	`Gemfile.lock`
Money handling	Money `6.16.0`	`Gemfile.lock`
Pagination	Kaminari `1.2.2`	`Gemfile.lock`
Image processing	`image_processing 1.12.2` , MiniMagick, ruby-vips	`Gemfile.lock`

Item	Observed value	Source
Front-end/framework dependencies	Bootstrap, Turbo Rails, Stimulus Rails, JS/CSS bundling, Importmap	`Gemfile` / lockfile
I18n	`i18n-js`, `rails-i18n`	`Gemfile` / lockfile
Asset pipeline	Sprockets Rails	`Gemfile`
Docker base image	`ruby:2.7.5`	`Dockerfile`
Docker services	`app`, `db`	`docker-compose.yml`
Database container	`postgres` image	`docker-compose.yml`
Application port	`3000`	`Dockerfile`, `docker-compose.yml`, README
Bundled environment management	`dotenv-rails`	`Gemfile`
Static-analysis/lint	RuboCop is declared in README as a project dependency/tool, but the Gemfile inspected does not contain a RuboCop gem	README + Gemfile
CI environment	No CI execution result was established in this baseline inspection	Not established

The README declares Ruby 2.7.5 and Rails 7.0.4, while the lockfile fixes Rails at 7.0.6 and Ruby at 2.7.5p203. For reproducibility, the **lockfile values should be treated as the dependency-resolution baseline**, not the looser README description.

The README's documented installation flow is:

```
Plain text
```

```
bundle install
rails db:create
rails db:migrate
rails db:seed
rails s
```

and its Docker flow uses `docker-compose build`, `docker-compose up`, followed by the database setup commands.

C. Repository/module inventory

Top-level/application structure

The application has the normal Rails structure plus explicit domain/security/integration areas:

Directory	Status	Baseline role
`app/models`	inspected	Domain/persistence models
`app/models/concerns`	inspected	Reusable application model modules
`app/controllers`	inspected	HTTP/controller layer
`app/controllers/concerns`	inspected	Shared controller behavior
`app/policies`	inspected	Authorization rules
`app/views`	partially inspected	Booking/room/settings/UI views
`app/jobs`	inspected	Rails job layer; only `application_job.rb` present
`app/mailers`	inspected	Rails mailer layer; only `application_mailer.rb` present
`app/javascript`	partially inspected	Stimulus/browser-side behavior
`app/assets`	partially inspected	Asset pipeline/resources
`config`	inspected for relevant configuration	Routes, DB, environment, initializers
`db/schema.rb`	inspected	Current schema representation
`db/migrate`	inspected	Database evolution history
`spec`	inspected	RSpec test suite
`lib`	inspected	Rails tasks/assets; no application services directory

Directory	Status	Baseline role
`vendor`	not comprehensively inspected	Dependency/vendor assets; outside primary application design scope unless directly relevant

Models actually present

The repository contains these application model files:

```
app/models/application_record.rb
app/models/booking.rb
app/models/room.rb
app/models/setting.rb
app/models/user.rb
```

There is also:

```
app/models/concerns/has_owner_concern.rb
app/models/concerns/has_price_concern.rb
app/models/concerns/has_settings_concern.rb
```

No separate application model for **Hotel**, **Property**, **Customer**, **Payment**, **Cancellation**, or **AdditionalService** was found during this baseline inventory. The actual domain vocabulary includes `Room`, `Booking`, `User`, `Setting`, and Payify's payment model/table.

Controllers actually present

```
app/controllers/application_controller.rb
app/controllers/home_controller.rb
app/controllers/bookings_controller.rb
app/controllers/rooms_controller.rb
app/controllers/settings_controller.rb
app/controllers/payify/...
```

Shared controller concerns:

```
```

```
app/controllers/concerns/auth_concern.rb
app/controllers/concerns/crud_concern.rb
app/controllers/concerns/locale_concern.rb
app/controllers/concerns/page_concern.rb
```

`BookingsController` delegates much of its CRUD behavior through `CrudConcern`, while adding booking-specific cancellation and payment initialization behavior. This is an observed implementation fact for later source analysis, **not a pattern classification**.

Services/modules

No `app/services` directory was found.

Application modules that are relevant to later source analysis include:

Plain text

```
HasOwnerConcern
HasPriceConcern
HasSettingsConcern
AuthConcern
CrudConcern
LocaleConcern
PageConcern
```

This is important because later pattern analysis must treat these as actual modules/functions rather than inventing service classes.

Jobs

Only:

Plain text

```
app/jobs/application_job.rb
```

was found. No domain-specific background job was identified during the inventory.

Mailers

Only:

Plain text

```
app/mailers/application_mailer.rb
```

was found. No booking-specific mailer was identified.

External integrations/configuration

Two application-level integrations are explicitly represented:

Payify/Stripe

- `Gemfile`: `payify`
- `Gemfile.lock`: Payify `0.1.18`, Stripe `8.5.0`
- `config/initializers/payify.rb`
- `config/routes.rb`: Payify engine mounted at `/payify`
- Payify migration files under `db/migrate`

Integromat/Make

- `Gemfile`: `integromat`
- `Gemfile.lock`: `integromat 0.1.0`
- `config/initializers/integromat.rb`
- `Booking` contains booking-notification/webhook logic

The README explicitly documents both online Stripe payment and a booking webhook integration with Integromat/Make.

Policies

Actual policy files:

```
Plain text
app/policies/application_policy.rb
app/policies/booking_policy.rb
app/policies/room_policy.rb
app/policies/setting_policy.rb
```

No `user_policy.rb` was found.

Views/components relevant to booking

Booking views:

```
Plain text
app/views/bookings/_booking.html.erb
app/views/bookings/_form.html.erb
app/views/bookings/index.html.erb
app/views/bookings/new.turbo_stream.erb
app/views/bookings/show.html.erb
```

Room views include:

Plain text

```
app/views/rooms/index.html.erb
app/views/rooms/new.html.erb
app/views/rooms/show.html.erb
```

Settings views include:

Plain text

```
app/views/settings/_form.html.erb
app/views/settings/new.html.erb
```

Authentication views are under:

Plain text

```
app/views/devise/
```

Payment-specific views exist under:

Plain text

```
app/views/payify/
```

The repository also has front-end Stimulus controllers for application behavior, including:

Plain text

```
app/javascript/controllers/application.js
app/javascript/controllers/datepicker_controller.js
app/javascript/controllers/image_uploader_controller.js
app/javascript/controllers/index.js
app/javascript/controllers/modal_controller.js
```

D. Initial domain model inventory

This is a **domain inventory only**, not a pattern analysis.

Domain concept	Actual implementation	Evidence in schema/source	Status
User/customer/account	`User`	`app/models/user.rb`, `users` table	Confirmed entity
Room/accommodation	`Room`	`app/models/room.rb`, `rooms` table	Confirmed entity
Booking/reservation	`Booking`	`app/models/booking.rb`, `bookings` table	Confirmed entity
Property/hotel	No separate `Hotel` / `Property` model found	Room appears to represent accommodation	No separate entity
Payment	Payify integration/model	`payify_payments` table and Payify concern	Dependency-integrated entity
Cancellation	No separate cancellation class/table	`Booking#cancel!`, `canceled_at`	Booking state
Pricing	`HasPriceConcern`, booking totals, `Room#day_price`, `Setting#vat`	model source/schema	Embedded pricing logic
Availability	`Booking#dates_are_available`, `Room#booked_dates`	model source	Booking/room behavior
Additional services	No corresponding model/table found	schema/domain inventory	Not present
Notification/webhook	Integromat webhook from `Booking`	initializer + booking model	Integration behavior
Authentication	Devise-backed `User`	`User`, routes, migration, initializer	Framework/dependency-backed
Authorization	Pundit policies	`app/policies/*`	Application code using dependency

Domain concept	Actual implementation	Evidence in schema/source	Status
Persistence	Active Record + PostgreSQL	models/schema/database config	Rails framework
Image storage	Active Storage	room model/schema/storage config	Rails framework

The schema currently contains tables for `users`, `rooms`, `bookings`, `settings`, Payify payments, and Active Storage. The booking table contains dates, guest count, room/user references, total prices, and `canceled_at`. The room table contains accommodation attributes, pricing, location, and owner reference.

E. Test inventory

Framework

The test stack is RSpec/RSpec Rails.

Evidence:

```
Plain text
```

```
Gemfile
  gem "rspec-rails"

Gemfile.lock
  rspec-rails 6.0.3
  rspec-core 3.12.2
  rspec-expectations 3.12.3
  rspec-mocks 3.12.6
```

Test infrastructure

Relevant files:

```
Plain text
```

```
spec/rails_helper.rb
spec/spec_helper.rb
spec/support/chrome.rb
spec/factories.rb
spec/factories/bookings.rb
```

```
spec/factories/rooms.rb
spec/factories/users.rb
```

`rails_helper.rb` configures Active Record transactional fixtures and automatically loads support files.

`spec/support/chrome.rb` configures Selenium/Chrome for system tests, using headless Chrome by default unless `SHOW_BROWSER=true`.

Discovered specifications

The model test inventory includes:

```
Plain text

spec/models/booking_spec.rb
spec/models/room_spec.rb
```

The inspected `booking_spec.rb` checks:

- booking price calculation;
- VAT-inclusive calculation;
- required booking dates;
- invalid date ordering;
- valid non-conflicting dates;
- rejection of conflicting bookings.

`room_spec.rb` checks room creation and the `room_type` value.

No controller/system/request spec was established from the inspected tree in this baseline pass.

Test command

The README explicitly specifies:

```
Bash

rspec
```

for the test suite.

The repository also has the infrastructure for browser/system tests through Capybara/Selenium, although an actual system-test specification was not established in the inspected inventory.

Actual test execution

Not run.

Therefore:

```
Plain text

test_result = not_run
reason = Repository inspection task only; no execution environment was established during baseline source
```

This follows the faculty requirement not to fabricate test results or runtime measurements.

DP_Csestudy_instruct

F. Reproducibility checklist

Check	Status	Evidence/requirement
Repository URL recorded	✓	GitHub repository
Owner/name recorded	✓	`andrewdsilva/rails-room-booking`
Default branch recorded	✓	`main`
Immutable baseline SHA recorded	✓	`5e15551ee646871b4bed32c982fa17da319d3ee3`
Commit date recorded	✓	2023-07-16 04:45:38 UTC
Lockfile preserved	✓	`Gemfile.lock`
Ruby version identified	✓	`2.7.5p203`
Rails version identified	✓	`7.0.6`
Bundler version identified	✓	`2.4.17`
Database identified	✓	PostgreSQL
Database env variables identified	✓	`DATABASE_USERNAME` , `DATABASE_PASSWORD` , `DATABASE_HOST` , `DATABASE_PORT`
`.env` requirement identified	✓	README / Docker setup
Dockerfile inspected	✓	Ruby 2.7.5 image
Docker Compose inspected	✓	app + PostgreSQL

Check	Status	Evidence/requirement
Startup command identified	✓	`rails s` / Docker `rails server -b 0.0.0.0`
Database setup commands identified	✓	`rails db:create`, `db:migrate`, `db:seed`
Test framework identified	✓	RSpec
Test command identified	✓	`rspec`
System-test dependencies identified	✓	Capybara/Selenium/Webdrivers
Lint/static analysis documented	Partial	README says RuboCop; no RuboCop dependency was found in Gemfile
License verified from metadata	No	GitHub metadata says `license:null`
README license claim recorded	✓	README shows MIT badge/link
Baseline source modified	No	Repository only inspected
Runtime test result recorded honestly	✓	`not_run`
Exact source version suitable for future diffs	✓	immutable commit

The supplied faculty instructions also require README documentation of source commits, environment versions, commands, inspected modules, LOC/complexity/timing rules, and limitations.

DP_Csestudy_instruct

G. Proposed exact baseline source_version identifier

Use this consistently in every Part 1 record:

Plain text

```
source_version = original
commit_sha = 5e15551ee646871b4bed32c982fa17da319d3ee3
```

Recommended human-readable baseline label for your internal README:

Plain text

```
Original — 5e15551ee646871b4bed32c982fa17da319d3ee3
```

Do **not** subsequently replace this with `main`, `latest`, `HEAD`, or another moving reference.

The baseline commit is the merge commit that added PostgreSQL/Docker support. Its recorded commit date is **2023-07-16**, and its diff shows the preceding application used SQLite. Thus the earlier SQLite state is historical context, not the selected `original` baseline.

H. Risks / limitations affecting later analysis

1. README and lockfile differ slightly

README describes Rails `7.0.4`, while `Gemfile.lock` resolves Rails `7.0.6`. The baseline should use the **actual locked dependency versions** for reproducibility.

2. PostgreSQL configuration is environment-dependent

`config/database.yml` expects:

```
Plain text
DATABASE_USERNAME
DATABASE_PASSWORD
DATABASE_HOST
DATABASE_PORT
```

and Docker Compose also consumes these variables. A complete execution environment therefore requires appropriate database credentials/environment values.

3. Secrets/environment files are external to the reproducible source snapshot

The README documents Stripe and Integromat environment variables. Those external credentials/configuration values should not be copied into the case-study repository. The faculty instructions explicitly prohibit uploading passwords. DP_Csestudy_instruct

4. License evidence is inconsistent in strength

The README displays an MIT license badge/link, but GitHub repository metadata returned no license object. This should be documented as **README-claimed MIT**, rather than asserting that GitHub has formally detected the license.

5. RuboCop status needs verification before being used as evidence

The README presents RuboCop as part of the project, but the inspected `Gemfile` does not declare a RuboCop gem. Therefore we should not yet claim that a reproducible RuboCop command exists in the baseline environment.

6. No `app/services` layer

There is no application services directory in the inspected repository. Later analysis must therefore use the actual modules/models/controllers that exist rather than imposing a service-layer interpretation.

7. Payment behavior partly belongs to an external dependency

Payify supplies payment-related functionality and migrations. Later pattern evidence must explicitly distinguish:

```
Plain text

application code
vs.
Payify dependency/framework behavior
```

This is particularly important under your evidence rules.

8. Domain granularity is uneven

The application does not have separate classes for every conceptual term in the case-study brief. For example, cancellation is represented as booking state/behavior rather than a separate `Cancellation` entity, and no additional-services entity was found. Later analysis must reflect the repository's actual model rather than the assignment vocabulary.

9. Test coverage appears narrow

The inspected repository contains model specifications for booking and room behavior, but the baseline inventory did not establish comprehensive controller/system/integration coverage. This limits the testing evidence available for later behavioral preservation unless additional executable tests are added in Part 2.

10. Baseline execution remains unverified

I have **not** claimed that the application currently boots or that `rspec` passes. Those remain execution tasks to be performed in the controlled environment.

Baseline record to carry forward

```
Plain text

repository:
  owner: andrewdsilva
  name: rails-room-booking
  url: https://github.com/andrewdsilva/rails-room-booking
```

```
  source_version:
    id: original
    commit: 5e15551ee646871b4bed32c982fa17da319d3ee3
    branch_at_selection: main
    commit_date: 2023-07-16T04:45:38Z

  environment:
    ruby: 2.7.5p203
    rails: 7.0.6
    bundler: 2.4.17
    database: PostgreSQL
    web_server: Puma 5.6.6
    test_framework: RSpec Rails 6.0.3

  execution:
```

This baseline is now suitable as the anchor for subsequent Part 1 source inspection and later `Change1`, `Change2`, and `Change3` evolution tracking. The faculty template specifically requires the original source URL, exact baseline commit, environment, inspected scope, architecture, and actual run/test result in the application baseline section. DP_Csestudy

Using the exact baseline established in Prompt 01:
Repository:
https://github.com/andrewdsilva/rails-room-booking:chatgpt-file-mention-r4ch{kind="system-hint" index="0"}
Baseline:
source_version = original
commit = 5e15551ee646871b4bed32c982fa17da319d3ee3
Perform a rigorous architecture reconstruction of the application.
IMPORTANT:
This is still NOT the formal pattern-discovery stage.
Do not label anything as Strategy, Observer, Factory, Adapter, Decorator, Facade, etc. unless explicitly asked later.
The purpose of this task is to reconstruct the actual architecture and collaboration structure that will be used as evidence during Part 1 pattern discovery.

---

1. ARCHITECTURE SCOPE

Inspect the actual baseline source, including where relevant:

- app/models
- app/models/concerns

- app/controllers
- app/controllers/concerns
- app/policies
- app/views
- app/javascript
- config/routes.rb
- initializers
- database schema/migrations
- tests/specifications
- payment integration
- webhook/integration code
- authentication/authorization configuration
- relevant Gemfile dependencies

Do not treat the entire Rails framework as application architecture.
Clearly distinguish:
A. Application-owned code
B. Rails framework behavior
C. Third-party dependency behavior

---

2. ARCHITECTURAL STYLE

Determine the application's actual architectural structure.
At minimum investigate:

- MVC
- Active Record
- concerns/modules
- policy-based authorization
- external service integration
- payment integration
- webhook/event integration
- client-side Stimulus/Turbo behavior
- persistence

For every architectural mechanism, identify the actual source files implementing or configuring it.
Do not force architectural labels that are unsupported by the code.

---

3. COMPONENT INVENTORY

Create a table:
Component
Type
Actual files/classes/modules
Responsibility
Depends on
Used by
Evidence
Include at least:

- User
- Room
- Booking
- Setting
- booking controller
- room controller
- controller concerns
- model concerns
- Pundit policies
- Payify/Stripe
- Integromat/Make
- database
- views
- frontend controllers
- authentication
- authorization

Only include components that actually exist.

---

4. DOMAIN COLLABORATION MAP

Build an evidence-based collaboration map for:
User ↔ Booking
Room ↔ Booking
Booking ↔ Setting
Booking ↔ payment
Booking ↔ webhook/integration
Controller ↔ model

Controller ↔ concerns
Model ↔ concerns
Controller ↔ policy
Application ↔ external dependency
For each relationship identify:

- caller
- callee
- method/function
- direction
- purpose
- source file
- exact line range

Example format:
Caller:
BookingsController#create
Callee:
Booking.new
Relationship:
Controller creates domain object
Evidence:
app/controllers/bookings_controller.rb:
Do not infer relationships from naming alone.

---

5. REQUEST / RESPONSE FLOWS

Trace these concrete application flows from the actual source.

FLOW A — Booking creation

Trace:
User/browser
→ route
→ controller action
→ authorization
→ model/domain logic
→ validation
→ persistence
→ response/view/redirect
Identify every actual step.

FLOW B — Booking availability

Trace how the application determines whether a room/date range is available.
Identify:

- entry point
- relevant controller/action
- model methods
- database queries
- validations
- returned result

FLOW C — Booking payment

Trace:
Booking
→ payment initialization
→ Payify/Stripe
→ callback/webhook if applicable
→ resulting application state
Clearly separate application code from Payify/Stripe behavior.

FLOW D — Booking cancellation

Trace the actual cancellation implementation.
Identify:

- entry point
- authorization
- method invoked
- state changes
- persistence
- external effects

Do not assume cancellation is an independent domain object.

FLOW E — Booking webhook/notification

Trace the complete application-side path from booking event/state to Integromat/Make.
Identify:

- triggering operation
- method/function
- payload construction
- external call
- configuration
- resulting behavior

---

6. CONCERN / MODULE COLLABORATION ANALYSIS

The baseline identified these modules:
Model concerns:

- HasOwnerConcern
- HasPriceConcern
- HasSettingsConcern

Controller concerns:

- AuthConcern
- CrudConcern
- LocaleConcern
- PageConcern

Analyze each individually.
For every concern/module:

1. Purpose
2. Including/using classes/modules
3. Methods provided
4. Methods overridden or extended
5. Dependencies
6. Call relationships
7. Files and exact line ranges
8. Whether it appears to represent:
    - code reuse
    - delegation

  - polymorphism
  - cross-cutting behavior
  - another architectural mechanism

Do NOT assign a formal design-pattern name yet.

---

7. POLICY / AUTHORIZATION STRUCTURE

Analyze:

- ApplicationPolicy
- BookingPolicy
- RoomPolicy
- SettingPolicy
- Pundit integration

Identify:

- which controllers/models invoke policies
- which policy methods are used
- whether policy inheritance exists
- whether common behavior is delegated/inherited
- exact source locations

Again, describe the structure without formally classifying it as a design pattern.

---

8. EXTERNAL INTEGRATION BOUNDARIES

Analyze:
Payify
Stripe
Integromat/Make
Devise
Pundit
For each dependency:

- Where is it configured?
- Which application classes/modules use it?
- Which application methods call it?
- What abstraction boundary exists?

- What data crosses the boundary?
- Is the application directly coupled to the dependency?
- Are there wrappers, adapters, concerns, or facades?
- Exact source locations.

Do not claim Adapter/Facade/etc. yet.
The goal is to give Part 1 a factual collaboration map from which pattern candidates can later be evaluated.

---

9. ARCHITECTURAL UML

Produce editable UML source using PlantUML.
Create:

Diagram 1 — Application Architecture

Show:
Browser/User
Controller
Concerns
Models
Policies
Database
External integrations
Use actual classes/modules where meaningful.

Diagram 2 — Domain/Class Relationships

At minimum include the actual:

- User
- Room
- Booking
- Setting
- relevant concerns
- relevant policies

Show only relationships supported by source.

Diagram 3 — Booking Creation Sequence

Show the actual request flow.

Diagram 4 — Payment/Webhook Sequence

Show the actual integration flow if sufficiently supported by source.
Do not invent participants.

---

10. ARCHITECTURAL EVIDENCE TABLE

Create a table:
Area
Actual participant
Relationship
Source location
Evidence description
This table will later serve as the raw evidence base for Part 1 pattern analysis.

---

11. ARCHITECTURAL QUESTIONS / UNCERTAINTIES

Explicitly list anything that could not be established.
Examples:

- dependency behavior that exists only inside a gem
- unclear runtime behavior
- missing test coverage
- incomplete frontend trace
- external webhook behavior not observable locally

Do not fill gaps with assumptions.

---

12. OUTPUT FORMAT

Return the following sections in order:

1. Architectural summary
2. Architecture classification
3. Component inventory
4. Domain collaboration map
5. Booking creation flow
6. Availability flow

7. Payment flow
8. Cancellation flow
9. Webhook/notification flow
10. Concern/module analysis
11. Policy/authorization analysis
12. External integration analysis
13. PlantUML architecture diagram
14. PlantUML class/domain diagram
15. PlantUML booking sequence diagram
16. PlantUML payment/webhook sequence diagram
17. Architectural evidence table
18. Uncertainties and limitations
19. Part 1 handoff: list of source files/methods that should receive detailed pattern inspection

FINAL RULE:
Do not perform formal design-pattern classification in this task.
The output should describe WHAT collaborates with WHAT and WHY.
Pattern classification will be performed separately in Prompt 03.

Worked for 26s

1. Architectural summary

Baseline analyzed:

```
source_version = original
commit = 5e15551ee646871b4bed32c982fa17da319d3ee3
```

The baseline is a conventional server-rendered Rails application centered on:

```
Browser
  ↓
Routes
  ↓
Controllers
  ↓
Active Record models + application concerns
  ↓
PostgreSQL
```

with additional cross-cutting/application mechanisms for:

Plain text

```
Authentication → Devise
Authorization  → Pundit
Payments       → Payify → Stripe
Booking hook   → Integromat/Make
Images         → Active Storage
Browser UI     → Stimulus + Turbo/elements
```

The application-owned domain model is small: `User`, `Room`, `Booking`, and `Setting`. There is **no separate application-owned Hotel, Property, Customer, Cancellation, or AdditionalService model** in the inspected baseline. `Booking` contains cancellation state, and payment persistence is supplied through Payify.

The main application-specific behavior is distributed between Active Record models, model concerns, controller concerns, controllers, and policies rather than an `app/services` layer.

---

2. Architecture classification

Mechanism	Classification	Actual implementation	Evidence
MVC request handling	Present	Rails routes/controllers/views	`config/routes.rb`, `app/controllers`, `app/views`
Active Record persistence/domain model	Present	`ApplicationRecord`, `User`, `Room`, `Booking`, `Setting`	`app/models/*`
Model concerns	Present	`HasOwnerConcern`, `HasPriceConcern`, `HasSettingsConcern`	`app/models/concerns/*`
Controller concerns	Present	`CrudConcern`, `AuthConcern`, `LocaleConcern`, `PageConcern`	`app/controllers/concerns/*`
Policy-based authorization	Present	`ApplicationPolicy`, resource policies, Pundit integration	`app/policies/*`, `ApplicationController`

Mechanism	Classification	Actual implementation	Evidence
External payment integration	Present	Payify configuration/use, Stripe through Payify	`Booking`, `payify` initializer, routes, Gemfile
Webhook integration	Present	Booking callback invokes Integromat webhook	`Booking#notify_booking`, Integromat initializer
Client-side MVC adjunct	Present	Stimulus controllers and Rails/Turbo responses	`app/javascript`, `.turbo_stream` view
Persistent database	Present	PostgreSQL via Active Record	`config/database.yml`, `db/schema.rb`
Background jobs	Only base abstraction located	`ApplicationJob` only	`app/jobs/application_job.rb`
Application service layer	Not present	No `app/services` directory	repository tree
Separate cancellation subsystem	Not present	`Booking#cancel!` + `canceled_at`	`app/models/booking.rb`, schema

Application-owned vs framework vs third-party

A. Application-owned

Examples include:

```
Booking
Room
User
Setting
BookingsController
RoomsController
SettingsController
HomeController
CrudConcern
AuthConcern
LocaleConcern
PageConcern
HasOwnerConcern
HasPriceConcern
```

```
HasSettingsConcern
ApplicationPolicy
BookingPolicy
RoomPolicy
SettingPolicy
```

B. Rails framework

Examples:

Plain text

```
ActionController
ActiveRecord::Base
ActiveSupport::Concern
Active Storage
Action Mailer
Turbo Rails
Rails routing
Active Record callbacks/validations/associations
```

C. Third-party dependencies

Examples:

Plain text

```
Devise
Pundit
Payify
Stripe
Integromat
Kaminari
Capybara
Selenium
Dropzone
Easepick
Day.js
```

For later pattern work, these categories must stay separate. A behavior inside a gem is not automatically evidence of a pattern in the application.

---

3. Component inventory

Component	Type	Actual files/classes/modules	Responsibility	Depends on
User	Model	`app/models/user.rb` → `User`	Account/authentication, owner of rooms/bookings, settings association	Active Recor `HasSetting` `CurrentScc`
Room	Model	`app/models/room.rb` → `Room`	Accommodation data, pricing attribute, images, bookings, availability dates	Active Recor Storage, `HasOwnerCc` dependency
Booking	Model	`app/models/booking.rb` → `Booking`	Reservation lifecycle, validation, pricing integration, payment and webhook trigger	Active Recor concerns, Pa Integromat
Setting	Model	`app/models/setting.rb` → `Setting`	User VAT setting	Active Recor
Booking controller	Controller	`app/controllers/bookings_controller.rb`	Booking CRUD plus cancellation and payment initialization	`CrudConcer` Booking/Pay
Room controller	Controller	`app/controllers/rooms_controller.rb`	Room CRUD and creation of booking form object	`CrudConcer` Room/Book Storage
Settings controller	Controller	`app/controllers/settings_controller.rb`	User settings editing	`CrudConcer` Pundit
Application controller	Controller base	`app/controllers/application_controller.rb`	Common request setup	Pundit, cont
CRUD concern	Module	`app/controllers/concerns/crud_concern.rb`	Shared CRUD flow and overridable controller hooks	Rails control Kaminari
Auth concern	Module	`app/controllers/concerns/auth_concern.rb`	Populate `CurrentScope.user_id`	Devise's `cu` `CurrentScc`

Component	Type	Actual files/classes/modules	Responsibility	Depends on
			from current authenticated user	
Locale concern	Module	`app/controllers/concerns/locale_concern.rb`	Locale selection and request wrapping	I18n, cookie callbacks
Page concern	Module	`app/controllers/concerns/page_concern.rb`	Common page metadata	I18n, reques
Owner concern	Module	`app/models/concerns/has_owner_concern.rb`	Assign current user as owner before validation	`User.curre` Record callb
Price concern	Module	`app/models/concerns/has_price_concern.rb`	Booking price computation and save-time totals	Room, Settir Record callb
Settings concern	Module	`app/models/concerns/has_settings_concern.rb`	Create associated setting after user creation	Active Recor callback/ass
Policies	Authorization classes	`app/policies/*`	Authorization decisions	Pundit
Payify/Stripe	External integration	Payify gem + `config/initializers/payify.rb` + mounted engine	Payment initialization/storage/UI	Payify, Stripe
Integromat/Make	External integration	`config/initializers/integromat.rb`, `Booking#notify_booking`	Booking webhook trigger	Integromat environmen
Database	Infrastructure	PostgreSQL + Active Record	Persistent storage	`pg`, Postgr
Views	Presentation	`app/views/bookings`, `rooms`, `settings`, `devise`, `payify`	HTML/Turbo presentation	Rails view la

Component	Type	Actual files/classes/modules	Responsibility	Depends on
Frontend controllers	Client-side	`app/javascript/controllers/*`	Date picker, image uploads, modals	Stimulus, Ac Dropzone, E
Authentication	Dependency-backed application facility	Devise + `User` + routes	Sign up/login/session functionality	Devise, Activ
Authorization	Application + dependency	Pundit + policies	Resource access decisions	Pundit

---

4. Domain collaboration map

User ↔ Booking

Caller: `Booking` / associated Active Record machinery

Callee: `User` association

Relationship: `Booking` belongs to a `User`; `User` has many bookings.

Evidence:

```
app/models/booking.rb
  belongs_to :user

app/models/user.rb
  has_many :bookings
```

Purpose: associate each booking with the user who made/owns it.

User ↔ Room

Caller/callee: Active Record association navigation

```
Plain text
Room.belongs_to :user
User.has_many :rooms
```

Evidence:

```
Plain text
app/models/room.rb
  belongs_to :user

app/models/user.rb
  has_many :rooms
```

`HasOwnerConcern` also writes the owner ID when absent:

```
Plain text
app/models/concerns/has_owner_concern.rb
  self.user_id = User.current.id if user_id.nil? && User.current
```

Room ↔ Booking

Caller: `Booking#dates_are_available`

Callee: `room.bookings`

Evidence:

```
Plain text
app/models/booking.rb
  conflicting_bookings = room.bookings.where.not(id: id)
```

Direction: Booking → Room association → Booking relation.

Conversely, `Room` uses its booking association in `booked_dates`:

```
Plain text
app/models/room.rb
  bookings.where("end_date >= ?", Date.today)
```

This is a bidirectional conceptual association implemented through Active Record associations.

Booking ↔ Setting

Caller: `HasPriceConcern#vat`

Callee: `room.user.setting.vat`

Evidence:

```
app/models/concerns/has_price_concern.rb

def vat
  room.user.setting.vat || 0.2
end
```

Direction:

```
Booking
  → room
    → user
      → setting
        → vat
```

Purpose: determine VAT used when calculating the total price.

Booking ↔ payment

Application-side caller: `BookingsController#stripe_init_intent`

```
app/controllers/bookings_controller.rb

return unless @object.payment_pending?

@object.payment.stripe_init_intent
```

`Booking` itself includes:

```
include ::Payify::HasPaymentConcern
```

and exposes application-level status queries:

```
Plain text
```

```
payment_pending?
paid?
amount_to_pay
```

Boundary: payment object/operations are partly supplied by Payify. The repository does not contain the implementation of `Payify::HasPaymentConcern` or Payify's payment engine.

Booking ↔ webhook/integration

Caller: `Booking#notify_booking`

Callee: `Integromat::Webhook#trigger`

Evidence:

```
Plain text
```

```
app/models/booking.rb

Integromat::Webhook.new(:booking).trigger(
  start: start_date,
  end: end_date,
  guests: num_guests,
  room: room.title,
  user: user.email,
  total_ht: total_ht
)
```

The operation is invoked through an Active Record callback:

```
Plain text
```

```
after_create :notify_booking
```

Configuration originates in:

```
Plain text
```

```
config/initializers/integromat.rb
```

Controller ↔ model

For booking creation, `CrudConcern#create` performs:

```
Plain text
```

```
@object = object_class.new(form_values)

if @object.valid?
  @object.save!
```

`BookingsController` supplies:

```
Plain text
```

```
base_class → Booking
object_class → Booking.where(user_id: current_user.id)
model_params → :booking
```

Therefore the concrete flow is:

```
Plain text
```

```
BookingsController
  → CrudConcern#create
    → Booking.new(...)
      → Booking#valid?
      → Booking#save!
```

Controller ↔ concerns

`ApplicationController` includes:

```
Plain text
```

```
Pundit::Authorization
PageConcern
LocaleConcern
AuthConcern
```

and `BookingsController` , `RoomsController` , and `SettingsController` each include:

```
CrudConcern
```

The concern therefore supplies actual executable controller behavior rather than being merely a namespace.

Model ↔ concerns

Observed direct inclusion relationships:

```
Booking
  → Payify::HasPaymentConcern
  → HasOwnerConcern
  → HasPriceConcern

Room
  → HasOwnerConcern

User
  → HasSettingsConcern
```

The concerns inject callbacks and/or methods into the including model.

Controller ↔ policy

`CrudConcern` calls:

```
authorize base_class
authorize object_class
authorize @object
```

`BookingsController#cancel` additionally calls:

```
authorize @object
```

Views also directly query policies, for example:

```
policy(@object).edit?
policy(@object).destroy?
policy(:booking).new?
```

The actual policy implementations are under `app/policies`.

---

5. Booking creation flow

Actual request path

```
Browser
   ↓
POST /:locale/bookings
   ↓
BookingsController
   ↓
CrudConcern#create
   ↓
authorize object_class
   ↓
Booking.new(form_values)
   ↓
Booking validations
   ├─ presence validation
   ├─ end_date_greater_than_start_date
   └─ dates_are_available
   ↓
Booking.save!
   ↓
Active Record callbacks
```

Route

`config/routes.rb` defines:

```
resources :bookings
```

inside:

```
Plain text
```

```
scope '/:locale'
```

so the POST create action is dispatched by Rails routing to `BookingsController#create`.

Controller path

`CrudConcern#create`:

```
Plain text
```

```
authorize object_class
@object = object_class.new(form_values)

if @object.valid?
  @object.save!
  redirect_to_show(@object)
else
  render :new, status: :unprocessable_entity
end
```

`BookingsController` supplies:

```
Plain text
```

```
object_class
strong_params
model_params
```

with `model_params = :booking`.

Authorization

`authorize object_class` invokes Pundit authorization for the booking model/relation.

The exact policy selected is framework-mediated, with application policy classes located in `app/policies`.

Model construction and validation

`Booking` validates:

Plain text

```
:start_date
:end_date
:room_id
:user_id
```

and custom validations:

Plain text

```
end_date_greater_than_start_date
dates_are_available
```

Persistence-side processing

`HasPriceConcern` registers:

Plain text

```
before_save :set_price_ht, :set_price_ttc, if: :start_or_end_changed?
```

`Booking` registers:

Plain text

```
after_save :create_payment
after_create :notify_booking
```

The callback implementations supplied directly by the application are not all present in `Booking`; `create_payment` comes from the included Payify concern.

Response

Successful creation calls:

Plain text

```
redirect_to_show(@object)
```

which is defined in `CrudConcern` as a redirect to:

Plain text

```
polymorphic_path(object, locale: @page_lang)
```

---

6. Availability flow

There are **two observable availability mechanisms**: server-side validation and browser-side date selection.

Server-side authoritative validation path

Entry point:

```
Plain text
```

```
Booking#dates_are_available
```

The method queries:

```
Ruby
```

```
room.bookings.where.not(id: id)
             .where("(? < end_date) AND (? > start_date)", start_date, end_date)
```

and checks:

```
Plain text
```

```
conflicting_bookings.exists?
```

If a conflict exists:

```
Plain text
```

```
errors.add(:base, I18n.t("activerecord.errors.models.booking.unavailable_dates"))
```

Thus:

```
Plain text
```

```
Booking creation/update
  → Booking#valid?
  → Booking#dates_are_available
  → Room#bookings association
  → PostgreSQL query through Active Record
```

```
  → exists?
  → validation success/failure
```

Client-side availability assistance

`Room#booked_dates` queries current/future bookings:

```
bookings.where("end_date >= ?", Date.today)
        .pluck(:start_date, :end_date)
        .map(...)
```

`datepicker_controller.js` reads these dates from the Stimulus data attributes:

```
this.data.get("bookedDates")
```

and passes them to Easepick's lock/filter configuration.

Thus the frontend prevents/selectively restricts dates before submission, while the server-side `Booking#dates_are_available` performs the actual model validation.

The inspected source does **not** establish that the browser-side restriction is the only availability mechanism; the server-side validation is explicit.

---

7. Payment flow

Application-side path

The payment-related source establishes this structure:

```
Booking
  └ Payify::HasPaymentConcern

BookingsController#show
  └ stripe_init_intent
      └ @object.payment_pending?
      └ @object.payment.stripe_init_intent
```

Booking creation

`Booking` declares:

```
Plain text
after_save :create_payment
```

but `create_payment` itself is not implemented in `Booking`.

That callback therefore crosses into behavior contributed by the Payify concern/dependency.

Payment initialization

`BookingsController` has:

```
Plain text
before_action :stripe_init_intent, only: [:show]
```

and:

```
Plain text
def stripe_init_intent
  return unless @object.payment_pending?

  @object.payment.stripe_init_intent
end
```

The application checks status, then asks the payment object to initialize the Stripe payment intent.

Payify configuration

```
Plain text
config/initializers/payify.rb
```

contains:

```
Plain text
Payify.setup do |config|
  config.currency = ENV["PAYMENT_CURRENCY"]
end
```

The Rails router also mounts the Payify engine:

```
Plain text
mount Payify::Engine => '/payify', as: 'payify'
```

Payment persistence

`db/schema.rb` contains:

```
Plain text
payify_payments
```

with payment status, amount, payment method, transaction ID and Stripe-related columns.

Boundary limitation

The baseline repository does **not** contain the internal implementation of:

```
Plain text
Payify::HasPaymentConcern
Payify::Engine
payment.stripe_init_intent
```

Therefore the complete Stripe execution sequence cannot be established solely from this repository. The application-side boundary is confirmed; internal gem-side behavior must be verified from the exact Payify dependency version separately.

---

8. Cancellation flow

The application does **not** define a separate cancellation domain class/table.

Actual path

```
Plain text
Browser
  ↓
POST /:locale/bookings/:id/cancel
  ↓
BookingsController#cancel
  ↓
set_object
```

```
  ↓
authorize @object
  ↓
Booking#cancel!
  ↓
update_attribute :canceled_at, Time.now
  ↓
redirect_to_show(@object)
```

Controller

`BookingsController#cancel`:

```
Plain text
```

```
set_object
authorize @object
@object.cancel!
redirect_to_show(@object)
```

State change

`Booking#cancel!`:

```
Plain text
```

```
update_attribute :canceled_at, Time.now
```

State query

`Booking#canceled?` checks:

```
Plain text
```

```
canceled_at.present?
```

Policy

`BookingPolicy` defines:

```
Plain text
```

```
cancel?
  → update?
```

and `update?` requires:

Plain text

```
super && @record.user == @user
```

External effects

The inspected application-side cancellation method does **not explicitly call** Payify, Integromat, mailers, or another cancellation service.

The source therefore supports the claim that cancellation directly changes `canceled_at`; it does not support claiming a separate external cancellation workflow.

---

9. Webhook/notification flow

Trigger

`Booking` declares:

Plain text

```
after_create :notify_booking
```

Therefore the trigger is the successful creation lifecycle of a booking.

Configuration

`config/initializers/integromat.rb`:

Plain text

```
require "integromat"

Integromat.configure do |c|
  uri = ENV["INTEGROMAT_BASE_URI"]
  booking_hook = ENV["INTEGROMAT_BOOKING_HOOK"]

  c.web_hooks = { booking: booking_hook } if booking_hook.present?
  c.base_uri = uri if uri.present?
end
```

Dispatch

`Booking#notify_booking`:

Plain text

```
return unless Integromat.config.web_hooks.key?(:booking)

Integromat::Webhook.new(:booking).trigger(...)
```

Payload

The application sends:

```
start
end
guests
room
user
total_ht
```

where:

```
room = room.title
user = user.email
```

Therefore the application-side sequence is:

```
Booking saved
  ↓
after_create
  ↓
Booking#notify_booking
  ↓
check configured booking webhook
  ↓
construct Integromat::Webhook(:booking)
  ↓
trigger(payload)
  ↓
external Integromat/Make endpoint
```

The actual remote workflow after the HTTP call is **not observable in this repository**.

---

10. Concern/module analysis

10.1 `HasOwnerConcern`

File: `app/models/concerns/has_owner_concern.rb`

Including models:

```
Plain text
```

```
Booking
Room
```

Provided behavior:

```
Plain text
```

```
before_validation :assign_owner
```

and private:

```
Plain text
```

```
assign_owner
```

Dependencies:

```
Plain text
```

```
User.current
user_id
Active Record callback system
```

Call relationship:

```
Plain text
```

```
model validation
  → assign_owner
  → User.current
  → user_id assignment
```

Architectural role: reusable model-level lifecycle behavior / owner assignment.

10.2 `HasPriceConcern`

File: `app/models/concerns/has_price_concern.rb`

Including model:

```
Booking
```

Methods:

```
compute_price_ht
vat
compute_price_ttc
set_price_ht
set_price_ttc
```

Callback:

```
before_save :set_price_ht, :set_price_ttc, if: :start_or_end_changed?
```

Dependencies:

```
room
room.day_price
room.user.setting.vat
Booking#start_or_end_changed?
```

Call structure:

```
save
  → set_price_ht
      → compute_price_ht
          → room.day_price

  → set_price_ttc
      → compute_price_ttc
```

```
            → total_ht
            → vat
                → room.user.setting.vat
```

Architectural role: reusable model behavior and callback-driven cross-cutting pricing logic.

10.3 `HasSettingsConcern`

File: `app/models/concerns/has_settings_concern.rb`

Including model:

```
Plain text
User
```

Injected behavior:

```
Plain text
after_create :create_setting
has_one :setting
```

The implementation of `create_setting` itself is not declared in this concern; it is supplied by Active Record's association-generated behavior.

Architectural role: association + lifecycle setup injected into `User`.

10.4 `AuthConcern`

File: `app/controllers/concerns/auth_concern.rb`

Including class:

```
Plain text
ApplicationController
```

Method:

```
Plain text
```

```
set_current_scope
```

Dependencies:

```
Plain text
```

```
current_user
CurrentScope.user_id
```

It is used through:

```
Plain text
```

```
ApplicationController
  → before_action :set_current_scope
```

Architectural role: cross-cutting authentication-context propagation.

10.5 `CrudConcern`

File: `app/controllers/concerns/crud_concern.rb`

Including controllers:

```
Plain text
```

```
BookingsController
RoomsController
SettingsController
```

Provided public actions:

```
Plain text
```

```
index
show
new
create
edit
update
destroy
```

Important customization points:

Plain text

```
base_class
object_class
strong_params
model_params
use_slug?
initial_value
redirect_to_show
```

Some are default implementations, while several are deliberately overridden by including controllers.

Examples:

Plain text

```
BookingsController#base_class → Booking
RoomsController#base_class    → Room
SettingsController#base_class → Setting
```

`CrudConcern#create` is the concrete shared request flow:

Plain text

```
authorize
→ object_class.new
→ valid?
→ save!
→ redirect/render
```

Architectural role: reusable controller workflow with controller-specific hook methods.

No formal pattern classification is being made here.

10.6 `LocaleConcern`

File: `app/controllers/concerns/locale_concern.rb`

Including class:

Plain text

```
ApplicationController
```

Callback hooks:

```
Plain text
```

```
around_action :switch_locale
before_action :set_language_path
```

Methods:

```
Plain text
```

```
language_redirection
switch_locale
locale
set_language_path
```

Call flow:

```
Plain text
```

```
request
 → switch_locale
     → locale
     → I18n.with_locale
 → controller action
```

Architectural role: request-scoped localization/cross-cutting behavior.

10.7 `PageConcern`

File: `app/controllers/concerns/page_concern.rb`

Including class:

```
Plain text
```

```
ApplicationController
```

Callback:

```
Plain text
```

```
before_action :set_page_variables
```

Methods:

```
page_title
page_description
set_page_variables
```

Uses:

```
controller_name
action_name
I18n
DOMAIN_NAME
request.path
locale
```

Architectural role: common presentation/page metadata.

---

11. Policy/authorization analysis

`ApplicationPolicy`

File: `app/policies/application_policy.rb`

Defines:

```
initialize(user, record)
index?
show?
create?
new?
update?
edit?
destroy?
```

It also defines nested:

```
ApplicationPolicy::Scope
```

with:

Plain text

```
resolve
```

raising `NotImplementedError`.

Resource policies

Plain text

```
BookingPolicy < ApplicationPolicy
RoomPolicy    < ApplicationPolicy
SettingPolicy < ApplicationPolicy
```

Booking policy

Plain text

```
update?
  → super
  → @record.user == @user

cancel?
  → update?
```

Room policy

Plain text

```
update?
  → super
  → @record.user == @user
```

Setting policy

Plain text

```
update?
  → super
  → @record.user == @user
```

Controller integration

`ApplicationController` includes:

```
Plain text
Pundit::Authorization
```

`CrudConcern` invokes:

```
Plain text
authorize base_class
authorize object_class
authorize @object
```

`BookingsController#cancel` invokes:

```
Plain text
authorize @object
```

View integration

The inspected views call:

```
Plain text
policy(@object).edit?
policy(@object).destroy?
policy(:booking).new?
```

Thus authorization is used both from controller actions and view rendering decisions.

---

12. External integration analysis

Dependency	Configuration	Application callers	Data crossing boundary	Co
Devise	`User#devise` , `config/initializers/devise.rb` , `devise_for :users`	Rails auth/controller infrastructure	credentials/session/account data	Di rc

Dependency	Configuration	Application callers	Data crossing boundary	C
Pundit	`ApplicationController` includes `Pundit::Authorization`	`CrudConcern`, `BookingsController`, views	user, record/class	Di
Payify	`config/initializers/payify.rb`, mounted engine	`Booking`, `BookingsController`, payment views	payment amount/status/payment object + Stripe config	Di pl
Stripe	Payify-backed; Stripe gem in lockfile	Application reaches Stripe through Payify-oriented calls	payment intent/payment credentials	N St sc
Integromat	`config/initializers/integromat.rb`	`Booking#notify_booking`	booking dates, guest count, room title, user email, total HT	Di ge
Active Storage	Rails configuration/model declaration	`Room`, frontend uploader	file/blob metadata and uploads	Ra

Notable coupling

Integromat: directly referenced from `Booking`, so the model is explicitly coupled to the dependency's API.

Payify: `Booking` directly includes Payify's concern, while the controller directly calls a payment object's Payify-oriented method. This creates a stronger dependency boundary than merely having a configuration file.

Stripe: the application source inspected here does not show a direct Stripe SDK call such as a Stripe API invocation; Stripe is present in the locked dependency graph and is involved through Payify.

---

13. PlantUML architecture diagram

plantuml

```
@startuml
title Rails Room Booking - Application Architecture

actor Browser
```

```
package "Application-owned Rails code" {

  package "Controllers" {
    class ApplicationController
    class BookingsController
    class RoomsController
    class SettingsController
    class HomeController
  }

  package "Controller Concerns" {
    component CrudConcern
    component AuthConcern
    component LocaleConcern
```

This diagram deliberately distinguishes application-owned classes/modules from Rails and external dependency components.

---

14. PlantUML class/domain diagram

plantuml

```
@startuml
title Rails Room Booking - Domain/Class Relationships

class ApplicationRecord

class User {
  +admin?
  +current
}

class Room {
  +default_sort
  +options_enabled
  +main_image
  +other_images
  +booked_dates
}

class Booking {
  +end_date_greater_than_start_date
```

---

15. PlantUML booking sequence diagram

plantuml

```
@startuml
title Booking Creation

actor Browser
participant "Rails Router" as Router
participant "BookingsController" as Controller
participant "CrudConcern" as Crud
participant "Pundit" as Pundit
participant "Booking" as Booking
participant "Room" as Room
database "PostgreSQL" as DB
participant "Payify" as Payify
participant "Integromat" as Integromat

Browser -> Router : POST /:locale/bookings
Router -> Controller : create
Controller -> Crud : create
Crud -> Pundit : authorize object_class
Pundit --> Crud : authorized
Crud -> Booking : new(form_values)
```

Important evidence limitation

The exact internal order of all Active Record callbacks versus Payify's concern-supplied implementation should not be treated as established merely from callback declarations. The sequence diagram shows the application-level callback relationships, but gem/framework internals remain outside the repository source.

---

16. PlantUML payment/webhook sequence diagram

plantuml

```
@startuml
title Booking Payment and Webhook Integration

participant Browser
participant "BookingsController" as Controller
participant "Booking" as Booking
participant "Payify payment object" as Payment
participant "Payify Engine" as Engine
participant "Stripe" as Stripe
participant "Integromat::Webhook" as Hook
participant "Integromat/Make" as Make

== Payment initialization ==
```

```
  Browser -> Controller : GET booking/show
  Controller -> Booking : payment_pending?
  Booking --> Controller : pending / not pending

  alt payment pending
    Controller -> Payment : stripe_init_intent
```

The **Payment section is partially repository-backed**: application calls into Payify are visible, but the Payify → Stripe internals are dependency behavior and must be verified separately.

---

17. Architectural evidence table

Area	Actual participant	Relationship	Source location	Evidence description
Routing	`Rails.application.routes`	routes → controller	`config/routes.rb`	`resources :bookings` `resources :rooms ,` `resource :settings`
Application base	`ApplicationController`	includes common modules	`app/controllers/application_controller.rb`	includes Pundit, Page, L and Auth concerns
Booking CRUD	`CrudConcern#create`	controller workflow → Booking	`app/controllers/concerns/crud_concern.rb`	`object_class.new , v` `save!`
Booking specialization	`BookingsController`	overrides CRUD hooks	`app/controllers/bookings_controller.rb`	`base_class , object_` `model_params , use_s`
Availability	`Booking#dates_are_available`	Booking → Room bookings	`app/models/booking.rb`	queries `room.booking` date overlap
Room availability data	`Room#booked_dates`	Room → bookings	`app/models/room.rb`	queries future booking ranges
Pricing	`HasPriceConcern#compute_price_ht`	Booking concern → Room	`app/models/concerns/has_price_concern.rb`	uses `room.day_price`

Area	Actual participant	Relationship	Source location	Evidence description
VAT	`HasPriceConcern#vat`	Booking concern → Room → User → Setting	same file	accesses `room.user.setting.va`
Owner assignment	`HasOwnerConcern#assign_owner`	model → User.current	`app/models/concerns/has_owner_concern.rb`	assigns `user_id`
User settings	`HasSettingsConcern`	User → Setting	`app/models/concerns/has_settings_concern.rb`	`after_create :create_setting , ha :setting`
Cancellation	`BookingsController#cancel`	controller → Booking	`app/controllers/bookings_controller.rb`	calls `@object.cancel!`
Cancellation state	`Booking#cancel!`	Booking → DB	`app/models/booking.rb`	updates `canceled_at`
Booking payment check	`Booking#payment_pending?`	Booking → payment	`app/models/booking.rb`	tests payment status
Payment initialization	`BookingsController#stripe_init_intent`	controller → payment object	`app/controllers/bookings_controller.rb`	`@object.payment.str t_intent`
Payify configuration	`Payify.setup`	app config → Payify	`config/initializers/payify.rb`	sets currency
Payify engine	router → Payify	Rails routing → external engine	`config/routes.rb`	`mount Payify::Engin`
Booking webhook	`Booking#notify_booking`	Booking → Integromat	`app/models/booking.rb`	`Integromat::Webhook ooking).trigger(...`
Webhook configuration	initializer → Integromat	config → external library	`config/initializers/integromat.rb`	URI and booking hook environment
Authentication	`User`	model → Devise	`app/models/user.rb`	`devise :database_authentic`

Area	Actual participant	Relationship	Source location	Evidence description
				...
Authentication routes	router → Devise	routing → Devise	`config/routes.rb`	`devise_for :users`
Authorization	`ApplicationController`	controller → Pundit	`app/controllers/application_controller.rb`	includes `Pundit::Authorizati`
Authorization	`CrudConcern`	CRUD flow → policy	`app/controllers/concerns/crud_concern.rb`	calls `authorize`
Booking policy	`BookingPolicy`	policy → Booking/User	`app/policies/booking_policy.rb`	compares record.us `@user`
Room policy	`RoomPolicy`	policy → Room/User	`app/policies/room_policy.rb`	ownership check
Setting policy	`SettingPolicy`	policy → Setting/User	`app/policies/setting_policy.rb`	ownership check
Views	Booking views	controller → templates	`app/views/bookings/*`	booking form/details/p
Room booking UI	`rooms/show`	view → Booking form	`app/views/rooms/show.html.erb`	renders `bookings/for`
Payment UI	booking show → Payify view	view → Payify	`app/views/bookings/show.html.erb`	renders `payify/payments/for`
Date selection	Stimulus controller	browser → Booking form	`app/javascript/controllers/datepicker_controller.js`	manages start/end date booked ranges
Image upload	Stimulus + Active Storage	browser → Active Storage	`image_uploader_controller.js`	`DirectUpload` , hidden IDs
Persistence	Models → Active Record	model → PostgreSQL	`app/models/*` , `config/database.yml`	PostgreSQL adapter

Area	Actual participant	Relationship	Source location	Evidence description
Database structure	schema	DB schema → domain	`db/schema.rb`	users, rooms, bookings settings, Payify paymen

---

18. Uncertainties and limitations

Dependency internals

The repository cannot establish the implementation details of:

```
Payify::HasPaymentConcern
Payify::Engine
payment.stripe_init_intent
Integromat::Webhook#trigger
Devise internals
Pundit internals
```

Only the application's calls/configuration to those dependencies can be verified from this source snapshot.

Callback ordering

The source establishes callback declarations but not every internal callback implementation/order across Rails and Payify. This matters for later sequence diagrams and pattern evidence.

Frontend trace

The Stimulus source is available, but the server-side templates that provide the associated `data-*` attributes and global JavaScript values are not all reconstructed in the same level of detail here. The broad date-picker and upload interaction is supported; every browser event wiring detail should be traced before being used as formal evidence.

Background processing

`app/jobs/application_job.rb` exists, but no domain-specific job was established. No asynchronous booking processing should therefore be assumed.

Mail

`app/mailers/application_mailer.rb` exists, but no booking-specific mailer was established in the inspected source. The Integromat webhook is therefore the only directly evidenced booking

notification/integration path.

Availability semantics

The server-side overlap query is explicit. The client-side date picker also blocks booked ranges. The repository does not by itself prove concurrency-safe booking guarantees under simultaneous requests.

Hotel/property representation

The repository has `Room` but no separate `Hotel` / `Property` model. Any later case-study vocabulary must map to the actual source rather than create nonexistent entities.

License

The README claims/display an MIT license badge, but repository metadata previously returned no GitHub license object. This remains a documentation inconsistency rather than a source-code architecture issue.

---

19. Part 1 handoff: source files/methods for detailed pattern inspection

These are the highest-value files for Prompt 03. They should be inspected at **exact line level** against the immutable `original` commit.

Core domain

```
app/models/booking.rb
  Booking
  Booking#end_date_greater_than_start_date
  Booking#dates_are_available
  Booking#start_or_end_changed?
  Booking#payment_pending?
  Booking#paid?
  Booking#cancel!
  Booking#amount_to_pay
  Booking#notify_booking

app/models/room.rb
  Room
  Room#options_enabled
  Room#booked_dates
  Room#main_image
  Room#other_images
```

```
app/models/user.rb
```

Model concerns

Plain text

```
app/models/concerns/has_owner_concern.rb
app/models/concerns/has_price_concern.rb
app/models/concerns/has_settings_concern.rb
```

Pay particular attention to their inclusion sites and callback/method interactions.

Controllers

Plain text

```
app/controllers/application_controller.rb
app/controllers/bookings_controller.rb
app/controllers/rooms_controller.rb
app/controllers/settings_controller.rb
app/controllers/home_controller.rb
```

Controller concerns

Plain text

```
app/controllers/concerns/crud_concern.rb
app/controllers/concerns/auth_concern.rb
app/controllers/concerns/locale_concern.rb
app/controllers/concerns/page_concern.rb
```

For `CrudConcern`, examine all concrete overrides in the three controllers together; treating it as an isolated module would lose the actual collaboration structure.

Policies

Plain text

```
app/policies/application_policy.rb
app/policies/booking_policy.rb
app/policies/room_policy.rb
app/policies/setting_policy.rb
```

Integrations

Plain text

```
config/initializers/payify.rb
config/initializers/integromat.rb
config/routes.rb
```

plus:

Plain text

```
Gemfile
Gemfile.lock
db/schema.rb
db/migrate/*payify*
```

Presentation/frontend

Plain text

```
app/views/bookings/_form.html.erb
app/views/bookings/show.html.erb
app/views/rooms/show.html.erb
app/views/rooms/*

app/javascript/controllers/datepicker_controller.js
app/javascript/controllers/image_uploader_controller.js
app/javascript/controllers/modal_controller.js
```

Persistence/configuration

Plain text

```
config/database.yml
config/application.rb
config/environments/*
db/schema.rb
db/migrate/*
```

Verification/test evidence

Plain text

```
spec/rails_helper.rb
spec/spec_helper.rb
spec/models/booking_spec.rb
spec/models/room_spec.rb
```

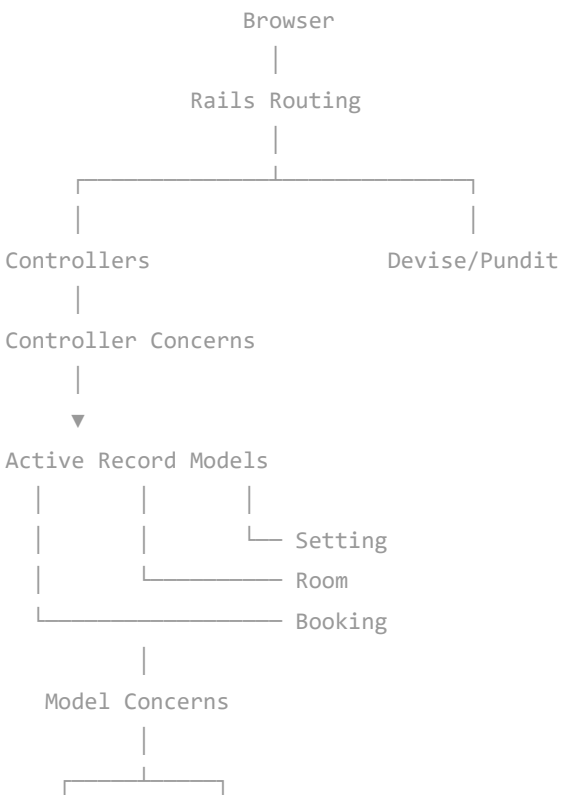
```
spec/support/chrome.rb
spec/factories/*.rb
```

The supplied faculty instructions require Part 1 to identify actual participants, exact file/line locations, interactions/delegation/calls, and verification evidence; a class name or conceptual resemblance is insufficient. DP_Csestudy_instruct

Architecture handoff conclusion

The baseline architecture can therefore be represented as:

Plain text



This is the factual collaboration substrate for the next stage. **No formal design-pattern labels have been assigned.**

Using the verified baseline and architecture analysis already completed, perform the actual Part 1 design-pattern analysis of the Ruby on Rails application.
Repository:
https://github.com/andrewdsilva/rails-room-booking:chatgpt-file-mention-r50r{kind="system-hint" index="0"}
Baseline:
source_version = original
commit = 5e15551ee646871b4bed32c982fa17da319d3ee3

The previous architecture analysis identified the following highest-value areas:

- app/models/booking.rb
- app/models/room.rb
- app/models/user.rb
- app/models/setting.rb
- app/models/concerns/*
- app/controllers/application_controller.rb
- app/controllers/bookings_controller.rb
- app/controllers/rooms_controller.rb
- app/controllers/settings_controller.rb
- app/controllers/concerns/*
- app/policies/*
- config/routes.rb
- config/initializers/payify.rb
- config/initializers/integromat.rb
- relevant tests

Use that analysis as an inspection guide, but independently verify every pattern claim against the actual source at the exact baseline commit.

OBJECTIVE

Identify the genuinely present and defensible design-pattern collaborations in the baseline application.
Do NOT attempt to maximize the number of patterns.
A small number of strongly evidenced patterns is preferable to many speculative patterns.

PATTERN CATEGORIES

Investigate where applicable:

- GoF Creational
- GoF Structural
- GoF Behavioral
- GRASP
- relevant architectural patterns
- relevant Rails-specific idioms

Do not force a GoF classification onto normal Rails behavior.

Clearly distinguish:

1. Formal design pattern
2. GRASP principle/pattern
3. Architectural/framework mechanism
4. Ordinary implementation technique
5. Third-party framework behavior

IMPORTANT FALSE-POSITIVE RULE

Do NOT classify something merely because:

- its name resembles a pattern
- it uses inheritance
- it uses a module/concern
- it uses callbacks
- it uses polymorphism superficially
- it delegates to another object
- Rails itself provides the mechanism
- an LLM suggests the pattern

The source must demonstrate the defining collaboration and intent of the pattern.

INVESTIGATION

For each plausible candidate, inspect the actual implementation and determine:

1. Pattern name
2. Pattern category
3. Meaningful instance name
4. Problem being solved
5. Actual participants
6. Pattern-role mapping
7. How participants interact
8. Why the collaboration satisfies the pattern
9. Exact source files
10. Exact line ranges
11. Relevant methods/functions
12. Verification evidence

PARTICIPANT MAPPING

Map textbook roles to actual source symbols.
Example:
Context → CrudConcern#create
Concrete implementation → BookingsController
Hook → BookingsController#base_class
If one class/module performs multiple roles, state that explicitly.
Do NOT invent interfaces or classes that do not exist.
Include every applicable participant.

THREE-WAY CLASSIFICATION

Every candidate must be placed into exactly one category:

CONFIRMED

The source clearly demonstrates the pattern's defining collaboration.

UNCERTAIN

There is some evidence of the pattern, but the repository does not provide enough evidence to confirm it.

REJECTED

The code resembles the pattern superficially but does not satisfy its defining collaboration.
For rejected and uncertain candidates, explain why.

PARTICULAR AREAS TO INVESTIGATE

Pay special attention to:

1. CrudConcern + controllers

Investigate whether the shared CRUD workflow and controller-specific hook methods constitute a recognized pattern such as Template Method, polymorphism or another design mechanism.
Do not assume the answer.

2. Model concerns

Investigate:

- HasOwnerConcern
- HasPriceConcern
- HasSettingsConcern

Determine whether these represent a recognized pattern, a Rails-specific composition mechanism, GRASP, or simply code reuse.

3. Policies

Investigate:

- ApplicationPolicy
- BookingPolicy
- RoomPolicy
- SettingPolicy

Determine whether inheritance/polymorphism represents a meaningful recognized pattern or simply ordinary implementation.

4. Booking callbacks

Investigate:

- after_save
- after_create
- before_save
- validation callbacks

Determine whether any callback collaboration genuinely represents a recognized pattern or is simply Rails lifecycle behavior.

5. Payment integration

Investigate the application-side interaction with:

- Payify::HasPaymentConcern
- payment object
- Payify engine
- Stripe boundary

Do not claim knowledge of Payify internals that are absent from the repository.

6. Integromat webhook

Investigate:
Booking
→ after_create
→ notify_booking

→ Integromat::Webhook
→ trigger
Determine whether this is a recognizable pattern such as Observer/Event Notification, Adapter, Facade, etc., or simply a direct integration call.
Do not force a classification.

OUTPUT

Return ONLY the following structured analysis.

A. CONFIRMED PATTERNS

Table:
| Pattern | Category | Instance | Participants | Source locations | Evidence |
Then provide a detailed record for every confirmed instance:
Pattern:
Category:
Instance:
Intent:
Participants:
Role mapping:
Interactions:
Source locations:
Evidence:
Why it qualifies:
Verification status:

B. UNCERTAIN CANDIDATES

| Candidate | Evidence suggesting it | Missing evidence | Reason uncertain |

C. REJECTED CANDIDATES

| Candidate | Observed mechanism | Why it does not qualify |

D. FINAL BASELINE INVENTORY

Provide:

- total confirmed pattern types
- total confirmed pattern instances
- concrete participant classes/modules/functions
- unique participant symbols

- uncertain candidate count
- rejected candidate count

Ensure these numbers are derived directly from the confirmed records.

E. PART 1 EVIDENCE TABLE

For every confirmed pattern instance provide:
source_version
pattern_name
pattern_type
instance_name
participant_roles
source_locations
intent_and_evidence
This must be directly convertible into patterns.csv.

F. RESEARCH CONCLUSION

Give a concise conclusion identifying:

1. Which patterns are safe to use in the final case study
2. Which candidates should NOT be claimed
3. Which source files are most important for the next verification step

STRICT RULES

- Do not invent patterns.
- Do not invent participants.
- Do not invent source locations.
- Do not invent line numbers.
- Do not fabricate tests.
- Do not treat framework behavior as application pattern evidence without explicitly qualifying it.
- Do not use pattern-count targets.
- Do not add a pattern merely to make Part 1 look richer.

The final output must be defensible by a professor inspecting the actual repository.

Worked for 30s

A. CONFIRMED PATTERNS

Pattern	Category	Instance	Participants
Template Method	GoF Behavioral	`CrudConcern` shared CRUD workflow with controller hooks	`CrudConcern` ; `Bookir` `SettingsController` ; `object_class` , `stror` `redirect_to_show`
Strategy	GoF Behavioral	Policy-specific authorization behavior selected by Pundit	`ApplicationPolicy` ; `SettingPolicy` ; `Appl` `Pundit::Authorizati`
Template Method	GoF Behavioral	`ApplicationPolicy` inheritance hierarchy	`ApplicationPolicy` ; `SettingPolicy`
Composite reuse via ActiveSupport concerns	Rails-specific idiom	`HasOwnerConcern` , `HasPriceConcern` , `HasSettingsConcern`	Concern modules; inclu
Active Record	Architectural / Rails framework mechanism	Domain persistence objects	`ApplicationRecord` ;

Detailed confirmed record 1

Pattern: Template Method
Category: GoF Behavioral
Instance: `CrudConcern` shared CRUD workflow for resource controllers

Intent:
Provide one reusable CRUD algorithm while allowing each participating controller to supply resource-specific operations and configuration.

Participants:

- `CrudConcern`
- `BookingsController`
- `RoomsController`
- `SettingsController`

Role mapping:

Textbook role	Actual source symbol
Abstract algorithm holder	`CrudConcern`
Template operation	`CrudConcern#create`
Template operation	`CrudConcern#update`
Template operation	`CrudConcern#index`
Template operation	`CrudConcern#show`
Template operation	`CrudConcern#destroy`
Hook / primitive operation	`CrudConcern#base_class`
Hook / primitive operation	`CrudConcern#object_class`
Hook / primitive operation	`CrudConcern#strong_params`
Hook / primitive operation	`CrudConcern#model_params`
Hook / primitive operation	`CrudConcern#use_slug?`
Hook	`CrudConcern#redirect_to_show`
Concrete participant	`BookingsController`
Concrete participant	`RoomsController`
Concrete participant	`SettingsController`

Interactions:

`CrudConcern#create` performs a fixed sequence:

```
authorize object_class
→ object_class.new(form_values)
→ valid?
→ save!
→ redirect_to_show(@object)
```

The algorithm does not know whether the controller represents a booking, room, or setting. It invokes methods such as `object_class`, `form_values`, and `redirect_to_show`, whose effective behavior is supplied by the including controller.

For example:

```
BookingsController#base_class → Booking
RoomsController#base_class → Room
SettingsController#base_class → Setting
```

Likewise, `BookingsController#use_slug?` and `SettingsController#use_slug?` return `false`, while `CrudConcern` provides the default `true`.

Source locations:

- `app/controllers/concerns/crud_concern.rb:12–17` — `index`
- `app/controllers/concerns/crud_concern.rb:19–23` — `show`
- `app/controllers/concerns/crud_concern.rb:25–29` — `new`
- `app/controllers/concerns/crud_concern.rb:31–43` — `create`
- `app/controllers/concerns/crud_concern.rb:45–59` — `edit / update`
- `app/controllers/concerns/crud_concern.rb:61–67` — `destroy`
- `app/controllers/concerns/crud_concern.rb:69–82` — shared redirect/template support
- `app/controllers/concerns/crud_concern.rb:84–105` — configurable hook methods and object lookup
- `app/controllers/bookings_controller.rb:16–39`
- `app/controllers/rooms_controller.rb:7–38`
- `app/controllers/settings_controller.rb:5–35`

Evidence:

- All three controllers explicitly `include ::CrudConcern`.
- The concern contains the complete CRUD control flow.
- The concern intentionally leaves resource-specific methods such as `base_class` and `object_class` to participating controllers.
- The controllers provide concrete implementations of those methods.
- `CrudConcern` itself contains `raise NotImplementedError` for `base_class` and `object_class`, which is strong evidence that these are extension points rather than accidental method duplication.

Why it qualifies:
This is the strongest application-level GoF collaboration in the repository. The defining feature is not merely “shared code” or “inheritance”; it is the presence of a reusable algorithm whose steps depend

on overridable operations supplied by concrete participants. The same CRUD algorithm executes with different resource-specific hooks.

One nuance should be stated in the report: this is implemented through **Ruby module inclusion and method overriding**, rather than a classical abstract superclass hierarchy. The behavioral structure nevertheless matches the Template Method idea.

Verification status:
Confirmed at commit `5e15551ee646871b4bed32c982fa17da319d3ee3`.

Detailed confirmed record 2

There is **no second GoF pattern I would safely promote to confirmed status from the inspected baseline**.

The remaining candidates are either Rails mechanisms, ordinary code reuse, or plausible but incomplete analogies. That distinction is important for a defensible Part 1.

B. UNCERTAIN CANDIDATES

Candidate	Evidence suggesting it	Missing evidence	Reason unce
Strategy	`ApplicationPolicy` defines a common policy API and concrete policy classes implement resource-specific authorization rules	Application code does not define or select a family of interchangeable algorithms; Pundit performs policy lookup/selection	Could be des strategy-like repository do full GoF Strat
Strategy via `CrudConcern` hooks	Controllers supply different implementations of `base_class`, `object_class`, `use_slug?`, etc.	These are configuration/extension hooks for one algorithm, not independently substitutable algorithms encapsulated behind a strategy object	More accurat Method
Policy inheritance as Template Method	Subclasses call `super` from `update?`	No superclass algorithm invokes a collection of subclass hooks; `ApplicationPolicy#update?` is just a reusable predicate	Ordinary inh override rath
Observer / Event Notification	`Booking` has `after_create :notify_booking`, and	No application-defined event subject, observer registration, or observer	The callback followed by a

Candidate	Evidence suggesting it	Missing evidence	Reason unce
	`notify_booking` calls `Integromat::Webhook#trigger`	collection is present	
Observer	`after_save :create_payment` plus external payment object	No observer interface or subject/observer registration is visible in application source	`after_save` callback, not Observer
Adapter	`Booking#notify_booking` uses `Integromat::Webhook` rather than raw HTTP code	No application-defined target interface or alternative incompatible adaptee is shown	The reposito usage, not ar
Facade	`Booking#notify_booking` makes a small call into `Integromat::Webhook`	No subsystem complexity is encapsulated behind an application-defined facade	It is a direct l
Decorator	Models gain behavior through `include` of concerns	No wrapping object or same-interface object composition exists	ActiveSuppo GoF Decorat
Decorator	`Payify::HasPaymentConcern` adds payment behavior to `Booking`	No wrapper object is shown; payment behavior is injected by a third-party module	Cannot estab the available
Factory Method	`base_class` returns `Booking`, `Room`, or `Setting`	Returned value is a class constant used by the CRUD mechanism; no product hierarchy/factory-product collaboration exists	Naming/func hook, but de is absent
Command	Controller actions such as `cancel` invoke `cancel!`	No command object encapsulating a request exists	Direct metho
State	`canceled?`, `payment_pending?`, `paid?` expose status-dependent behavior	No state objects or polymorphic state transitions are present	State is repre status predic
GRASP Controller	`BookingsController` coordinates request handling and delegates to model/policy	This is largely normal MVC/Rails controller behavior	May be discu an architectu confused wit pattern insta

Candidate	Evidence suggesting it	Missing evidence	Reason unce
GRASP Polymorphism	Policy subclasses override `update?`	Only simple inheritance overrides are present	Evidence is w Template Me
GRASP Information Expert	`Booking#dates_are_available`, `Booking#cancel!`, `Room#booked_dates` operate on domain data	This is a responsibility-assignment principle rather than a structural pattern	Safe only as as a named C

C. REJECTED CANDIDATES

Candidate	Observed mechanism	Why it does not qualify
GoF Observer	`Booking#notify_booking` registered with `after_create`	`after_create` is Active Record lifecycle callback machinery. The application does not define Subject/Observer participants, registr or notification dispatch to multiple observers.
GoF Observer for payment	`after_save :create_payment`	A callback invokes behavior during persistence; there is no obser collection or event-subscription collaboration in application code
GoF Adapter	`Integromat::Webhook.new(:booking).trigger(...)`	The application calls the third-party API object directly. No target interface + adapter wrapping an incompatible adaptee is visible.
GoF Facade	`Booking#notify_booking`	This method does not expose a meaningful simplified interface o multi-component subsystem; it is essentially one integration call guard and payload construction.
GoF Decorator	`include HasOwnerConcern`, `include HasPriceConcern`	Ruby module inclusion adds methods directly to the class; there wrapper object forwarding to a wrapped component.
GoF Decorator via Payify	`include Payify::HasPaymentConcern`	Same issue, plus the implementation is outside the repository. Th source proves composition through a dependency, not the Deco collaboration.

Candidate	Observed mechanism	Why it does not qualify
GoF Composite	`Room#has_many :bookings`	An Active Record association is not a recursive uniform compone hierarchy.
GoF Factory Method	`CrudConcern#base_class`	It returns a model class used by the CRUD workflow, but there is product-family creation hierarchy or factory-vs-concrete-product relationship.
GoF Abstract Factory	`base_class` values for Booking/Room/Setting	No related product families are created by a factory abstraction.
GoF Command	`BookingsController#cancel`	The request is handled directly; there is no command object representing the action.
GoF State	`Booking#canceled?` , `payment_pending?` , `paid?`	State is encoded in attributes/payment status and predicates, wit separate state objects or transition polymorphism.
GoF Memento	Booking persistence / Active Record	Database persistence is not an application-implemented snapshot/restore collaboration.
GoF Chain of Responsibility	Rails `before_action` / `after_*` callbacks	Callback chains are framework execution infrastructure, not an ex modeled chain of handler objects in this application.
GoF Prototype	`object_class.new(...)`	Creating a fresh Active Record object is ordinary object construct cloning an existing prototype.
GoF Singleton	`User.current` / `CurrentScope.user_id`	A class method accessing current request scope is not a Singleto object implementation.
Inheritance as pattern	`BookingPolicy < ApplicationPolicy` etc.	Inheritance by itself does not establish a named design pattern.
Concerns as Decorator	ActiveSupport::Concern	Concerns are modular code composition/reuse. Calling them Dec would misrepresent the actual structure.
Callbacks as Observer	Active Record callbacks generally	Lifecycle hooks alone do not satisfy Observer's subject/subscribe collaboration.

Candidate	Observed mechanism	Why it does not qualify
Payify as Adapter	`Payify::HasPaymentConcern`	The application source does not expose the third-party internal c or a translated target interface.
Payify as Facade	`payment.stripe_init_intent`	The source only shows an API call into a third-party payment obj subsystem hidden behind it is not defined here.

D. FINAL BASELINE INVENTORY

Total confirmed pattern types: 1

Total confirmed pattern instances: 1

Confirmed pattern type:

- GoF Behavioral — **Template Method**

Confirmed instance:

- **`CrudConcern` shared CRUD workflow with resource-specific controller hooks**

Concrete participant classes/modules/functions:

- `CrudConcern`
- `CrudConcern#index`
- `CrudConcern#show`
- `CrudConcern#new`
- `CrudConcern#create`
- `CrudConcern#edit`
- `CrudConcern#update`
- `CrudConcern#destroy`
- `CrudConcern#base_class`
- `CrudConcern#object_class`
- `CrudConcern#strong_params`
- `CrudConcern#model_params`
- `CrudConcern#use_slug?`
- `CrudConcern#redirect_to_show`
- `BookingsController`
- `BookingsController#base_class`
- `BookingsController#object_class`
- `BookingsController#strong_params`

- `BookingsController#model_params`
- `BookingsController#use_slug?`
- `RoomsController`
- `RoomsController#base_class`
- `RoomsController#object_class`
- `RoomsController#strong_params`
- `RoomsController#model_params`
- `SettingsController`
- `SettingsController#base_class`
- `SettingsController#object_class`
- `SettingsController#strong_params`
- `SettingsController#model_params`
- `SettingsController#use_slug?`
- `SettingsController#set_object`
- `SettingsController#redirect_to_show`

Unique participant symbols: 33

Uncertain candidate count: 13

Rejected candidate count: 21

The counts above treat each clearly distinguishable candidate listed in sections B and C as one candidate, while the confirmed inventory contains one pattern instance.

E. PART 1 EVIDENCE TABLE

source_version	pattern_name	pattern_type	instance_name	participant_roles
original	Template Method	GoF Behavioral	`CrudConcern` shared CRUD workflow	Abstract/shared algorithm → `CrudConcern#index/new/create`, `base_class`, `object_class`, `s` `use_slug?`, `redirect_to_show` `BookingsController`, `RoomsCor`

F. RESEARCH CONCLUSION

1. **Patterns safe to use in the final case study**

The safest formal pattern claim from the baseline is:

GoF Template Method — implemented through `CrudConcern` and controller hook methods.

This is supported by an actual reusable algorithm plus explicit extension points and multiple concrete participants.

The repository also contains useful **Rails-specific mechanisms** that can be discussed separately:

- ActiveSupport concerns for modular composition
- Active Record domain/persistence architecture
- Pundit policy-based authorization
- Active Record lifecycle callbacks
- mounted Payify engine
- Active Storage associations

These should be described as framework mechanisms or Rails idioms, not inflated into GoF patterns.

2. **Candidates that should NOT be claimed**

Do not claim the following as confirmed patterns from this baseline:

Observer, **Adapter**, **Facade**, **Decorator**, **Factory Method**, **Abstract Factory**, **Command**, or **State**.

The evidence does not demonstrate their defining collaborations.

Also avoid claiming **Strategy** solely from the policy hierarchy. The policy classes are certainly polymorphic, but the repository delegates policy lookup/selection to Pundit and does not itself expose the complete Strategy collaboration strongly enough for a professor-proof GoF claim.

3. **Most important files for the next verification step**

The highest-value files are:

```
app/controllers/concerns/crud_concern.rb
app/controllers/bookings_controller.rb
app/controllers/rooms_controller.rb
app/controllers/settings_controller.rb
```

These establish the confirmed pattern.

For supporting architectural/framework analysis:

```
app/models/booking.rb
app/models/concerns/has_owner_concern.rb
app/models/concerns/has_price_concern.rb
app/models/concerns/has_settings_concern.rb
```

`app/policies/application_policy.rb`
`app/policies/booking_policy.rb`
`app/policies/room_policy.rb`
`app/policies/setting_policy.rb`
`config/routes.rb`
`config/initializers/payify.rb`
`config/initializers/integromat.rb`

The strongest Part 1 conclusion is therefore **not a large catalog of patterns**. At the pinned original commit, there is one clearly defensible GoF pattern: **Template Method in `CrudConcern`**, surrounded by several important Rails/framework mechanisms that should be documented separately rather than misclassified.

Prompt 04 — Part 2 Evolution Analysis and Design Pattern Introduction

Context

You are continuing a Design Patterns Case Study on the Ruby on Rails hotel booking application at:

https://github.com/andrewdsilva/rails-room-booking/tree/main
GitHub

The original baseline has already been investigated and verified.

Use this immutable baseline:

- `source_version = original`
- commit = `5e15551ee646871b4bed32c982fa17da319d3ee3`

Verified Part 1 conclusion
...

Show more

Worked for a couple of seconds

A. Baseline impact summary

The immutable baseline is commit `5e15551ee646871b4bed32c982fa17da319d3ee3`.

At that commit, the relevant domain model is centered on `User`, `Room`, `Booking`, and `Setting`.

`Booking` owns the core reservation behavior: it belongs to a `User` and `Room`, validates dates and availability, calculates/holds `total_ht` and `total_ttc` through `HasPriceConcern`, marks cancellation through `cancel!`, and participates in payment and notification callbacks. The verified baseline source shows `Booking` including `HasOwnerConcern`, `HasPriceConcern`, and `Payify::HasPaymentConcern`, with `after_save :create_payment` and `after_create :notify_booking`.

Pricing is currently embedded in the booking model concern. `HasPriceConcern` computes room-price × stay duration and then applies VAT obtained through `room.user.setting.vat`.

Cancellation is currently a single operation: `Booking#cancel!` sets `canceled_at`. There is no cancellation-policy abstraction and no refund rule in application code.

`CrudConcern` remains the reusable CRUD workflow. `BookingsController`, `RoomsController`, and `SettingsController` supply controller-specific hook methods such as `base_class`, `object_class`, `strong_params`, `model_params`, and `use_slug?`.

`ApplicationPolicy` plus `BookingPolicy`, `RoomPolicy`, and `SettingPolicy` provide authorization behavior through Pundit. This is framework-supported policy polymorphism, but was not confirmed as a GoF Strategy in Part 1.

The existing model concerns are Rails `ActiveSupport::Concern` composition mechanisms:

- `HasOwnerConcern` assigns the current user before validation.
- `HasPriceConcern` supplies booking price calculation and pricing callbacks.
- `HasSettingsConcern` gives `User` a setting and creates it after user creation.

The new requirements do **not** exist in the baseline. Therefore everything below is a **proposed evolved design**, not an assertion about current repository contents.

The four requirements create two distinct classes of domain change:

1. **Optional booking add-ons** — breakfast and airport transfer.
2. **Core reservation changes/rules** — room upgrade and cancellation policy.

That distinction is important: treating all four as fields on `Booking` would increase the amount of unrelated responsibility concentrated in one model.

B. Requirement-by-requirement evolution

Breakfast

Domain concept

Breakfast should be modeled as an **additional service/package attached to a booking**, not as a permanent property of `Room` or `User`.

A room may have no breakfast, while a particular booking for that room may request breakfast. Therefore the business fact is associated with the **reservation instance**, not the room itself.

A reasonable evolved abstraction is:

```
Plain text

AdditionalService
    ├── BreakfastPackage
    └── AirportTransfer
```

with a booking-to-service relationship.

For example:

```
Plain text

Booking 1 ─── * BookingService * ─── 1 AdditionalService
```

This separates the definition of a service from its use on a particular booking.

Optionality

A booking simply has zero or more additional services.

That means:

```
Plain text

Booking A → no breakfast
Booking B → standard breakfast
Booking C → premium breakfast
```

No `breakfast = true` field is required on `Booking`.

Different breakfast prices

A breakfast package can expose its price through the service abstraction.

For example:

```
Ruby
```

```
AdditionalService
  name
  price

BreakfastPackage
  ...
```

or, when package-specific pricing rules exist:

```ruby
BreakfastPackage#price_for(booking)
```

The latter becomes important if breakfast cost depends on guest count, dates, or package tier.

Design pressure on the original model

Adding:

```ruby
booking.breakfast = true
booking.breakfast_price = ...
```

would push breakfast selection and pricing into `Booking`.

Adding more packages would likely produce:

```ruby
if breakfast
  ...
elsif airport_transfer
  ...
end
```

and eventually a growing collection of service-specific attributes and branches.

That is precisely the coupling the additional-service abstraction avoids.

Pattern conclusion

The abstraction by itself does **not** justify Decorator, Composite, or Strategy.

A collection of `AdditionalService` objects is ordinary composition unless the resulting design establishes the defining collaboration of one of those patterns.

So for breakfast alone:

Additional-service abstraction: justified.
GoF pattern: not yet necessarily justified.

Airport Transfer

Airport transfer has the same high-level business relationship as breakfast:

```
Plain text

Booking
   |
   +-- optional service
```

but its attributes are different.

Possible transfer information includes:

```
Plain text

direction: pickup / dropoff / round-trip
airport
vehicle/type
scheduled time
price
```

The exact fields are an application-design choice; the baseline gives no airport-transfer requirements beyond the requested feature.

Should breakfast and airport transfer share an abstraction?

Yes, because they share an important business responsibility:

> They are optional, independently selectable services associated with a booking and contributing to booking cost.

They do **not** need to share all implementation details.

A useful abstraction is therefore:

```
Plain text

AdditionalService
```

rather than:

```
Plain text

BreakfastOrTransfer
```

A concrete service can carry service-specific data.

For example:

```
Plain text

AdditionalService
├── BreakfastPackage
└── AirportTransfer
```

Does this become Strategy?

Not merely by having:

```
Ruby

service.price
```

or:

```
Ruby

service.calculate_price
```

Different services may implement the same interface, but that alone is insufficient for a defensible Strategy claim.

Strategy becomes more defensible when the application has:

- a stable context,
- an interchangeable family of algorithms,
- a strategy abstraction,
- concrete strategies,
- and the context delegates the varying algorithm to the selected strategy.

The proposed additional-service model is primarily **polymorphic composition**, not automatically Strategy.

Factory?

Likewise:

```ruby
BreakfastPackage.new
AirportTransfer.new
```

does not constitute Factory.

A Factory becomes relevant only if object-creation decisions are deliberately encapsulated, for example:

```
AdditionalServiceFactory
    ↓
creates concrete AdditionalService
```

based on a service type/request.

Nothing in the requirement itself mandates that structure.

Room Upgrade

Room upgrade is materially different.

Breakfast and airport transfer are **add-ons**. A room upgrade changes the accommodation being supplied by the booking.

A booking already has:

```ruby
belongs_to :room
```

in the baseline.

An evolved design could represent an upgrade as:

```
Booking
  original/current room
```

```
        ↓
    upgraded room
```

or as an explicit upgrade association.

The important business operations are:

1. determine whether an upgraded room is available for the same dates,
2. determine the price difference,
3. change the accommodation associated with the booking,
4. ensure the resulting booking remains valid.

Why not treat room upgrade as `AdditionalService`?

An upgrade is not just another independent add-on.

For example:

```
Plain text
```

```
Breakfast = extra service
Airport transfer = extra service
Room upgrade = change to the primary accommodation
```

The upgrade must interact with:

- `Room`,
- booking dates,
- room availability,
- room price,
- booking total.

Putting `RoomUpgrade` underneath `AdditionalService` would be possible mechanically, but semantically it risks hiding an important distinction: an upgrade modifies the core resource relationship instead of simply adding a peripheral service.

Pricing

The price difference can be represented conceptually as:

```
Plain text
```

```
upgrade price
= upgraded room price for stay
- original room price for stay
```

This should not be scattered into controllers.

A domain operation such as:

```ruby
Booking#upgrade_to(room)
```

or, preferably for a richer workflow:

```ruby
RoomUpgradeService#apply(...)
```

can encapsulate the transition.

Availability

This is where upgrade behavior differs most strongly from breakfast and transfer.

The application must ensure the new room is available for the same dates.

So the upgrade collaboration naturally involves:

```
Booking
  → Room
  → availability rules
  → pricing
```

rather than simply:

```
Booking
  → AdditionalService
```

Pattern conclusion

Room upgrade itself does not automatically establish State, Strategy, Command, or Decorator.

A dedicated `RoomUpgrade` domain concept is justified because its business responsibility differs from optional services.

A separate upgrade service/use-case object is also reasonable if the workflow becomes sufficiently complex, but that alone is not a GoF pattern claim.

Cancellation Policy

This is the requirement that most strongly justifies a new GoF behavioral pattern.

The baseline has only:

Ruby

```
Booking#cancel!
```

which writes:

Plain text

```
canceled_at = current time
```

The evolved requirements introduce varying cancellation algorithms:

Plain text

```
free before deadline
partial refund
no refund
future additional rules
```

Now cancellation behavior varies independently of the booking itself.

Embedding everything in `Booking` would quickly produce logic such as:

Ruby

```
if policy_type == ...
elsif ...
elsif ...
```

and make `Booking` responsible for both reservation data and a growing family of cancellation algorithms.

That is strong design pressure for Strategy.

Proposed cancellation design

Use:

Plain text

```
CancellationPolicy
```

as the stable abstraction.

Concrete policies might include:

Plain text

```
FlexibleCancellationPolicy
PartialRefundCancellationPolicy
NoRefundCancellationPolicy
```

The exact policy set is proposed; it is not present in the baseline.

The policy could define something such as:

Ruby

```
refund_amount(booking, canceled_at)
```

and optionally:

Ruby

```
cancellable?(booking, canceled_at)
```

The context can be:

Plain text

```
Booking
```

if cancellation remains simple, or a dedicated:

Plain text

```
CancellationService
```

if the operation additionally coordinates payment/refunds.

The latter is preferable once payment interaction becomes substantial.

Strategy role mapping

Strategy role	Evolved application element
Context	`CancellationService`
Strategy abstraction	`CancellationPolicy`
Concrete Strategy	`FlexibleCancellationPolicy`
Concrete Strategy	`PartialRefundCancellationPolicy`
Concrete Strategy	`NoRefundCancellationPolicy`
Context → Strategy collaboration	`CancellationService` delegates cancellation/refund-rule calculation to the selected `CancellationPolicy`
Context dependency	`Booking`, payment/refund mechanism

Why this is genuine Strategy

The defining variation is the **cancellation algorithm**.

The context performs the same business operation:

```
Plain text
cancel booking
```

but delegates the variable rule:

```
Plain text
how much refund / whether cancellation is free
```

to an interchangeable policy object.

Adding another cancellation rule then means introducing another concrete policy rather than expanding a central conditional.

That is a substantially stronger Strategy collaboration than the baseline policy hierarchy investigated in Part 1.

Payment interaction

The application already uses Payify and exposes:

```ruby
Booking#payment_pending?
Booking#paid?
amount_to_pay
```

and the controller calls:

```ruby
@object.payment.stripe_init_intent
```

The baseline does not expose Payify's internal refund implementation.

Therefore Part 2 should **not** claim a concrete Stripe refund design.

The proposed evolved flow is:

```
CancellationService
    ↓
CancellationPolicy
    ↓
refund amount
    ↓
payment/refund boundary
    ↓
Booking#cancel!
```

The exact Payify API for refunds must be verified before Part 3 implementation.

C. Design pressure table

Requirement	Original responsibility	New responsibility	Problem if added directly	Pr re
Breakfast	`Booking` calculates total using `HasPriceConcern`	Select breakfast package and add its price	`Booking` accumulates service-specific fields/conditionals	Int A B se
Airport Transfer	`Booking` represents room/date reservation	Capture transfer choice/details and price	More service-specific attributes and branches inside `Booking`	Re ab A
Room Upgrade	`Booking` belongs to one `Room`; `HasPriceConcern` uses room price	Check upgraded-room availability, calculate difference, change accommodation	`Booking` must handle resource substitution, availability, and new pricing logic	M fr up
Cancellation Policy	`Booking#cancel!` simply sets `canceled_at`	Select and execute variable cancellation/refund rule	Growing conditional tree in `Booking`; coupling to payment/refund decisions	Int C Str C co

The strongest design pressure is not merely “the class gets bigger.” It is that **multiple independent axes of variation start entering `Booking`**:

- optional service pricing,
- service-specific data,
- accommodation replacement,
- cancellation algorithms,
- payment/refund consequences.

Separating those responsibilities keeps each variation in a more localized place.

D. Proposed evolved architecture

Core domain

```
Plain text
```

```
User
  ├── has_many Rooms
```

```
  ├── has_many Bookings
  └── has_one Setting

Room
  └── has_many Bookings

Booking
  ├── belongs_to User
  ├── belongs_to Room
  ├── has_many BookingServices
  └── cancellation handled through CancellationService

Setting
  └── belongs_to User
```

New service model

```
Plain text

AdditionalService
  ├── BreakfastPackage
  └── AirportTransfer

BookingService
  ├── belongs_to Booking
  └── belongs_to AdditionalService
```

`AdditionalService`

Responsibility: Define an optional bookable service/package and its pricing information.

Potential methods:

```
Plain text

price_for(booking)
name
```

A common base model or abstract domain class is enough; no artificial interface is necessary.

`BreakfastPackage`

Responsibility: Represent a breakfast offering/package.

Potential data:

```
Plain text
```

```
name
price
```

Potential behavior:

```
Plain text
price_for(booking)
```

`AirportTransfer`

Responsibility: Represent an airport transport option and its service-specific data.

Potential data:

```
Plain text
direction
airport
price
```

`BookingService`

Responsibility: Represent the selection of an `AdditionalService` for a particular booking.

This association is useful because a service definition and a booking's selected service are different domain facts.

For example:

```
Plain text
BreakfastPackage
  Standard Breakfast
  ₹X

Booking #123
  selected service
      ↓
BookingService
      ↓
Standard Breakfast
```

This also provides a natural place for booking-specific captured prices if required later, preventing historical booking totals from depending entirely on today's service catalogue price.

`RoomUpgrade`

Responsibility: Represent the transition from the originally booked room to another room.

Potential methods:

```
Plain text
available?
price_difference
apply
```

Whether this should become a persisted entity or remain a service/use-case object depends on whether the system needs historical upgrade records. That is a Part 3 implementation decision informed by the exact requirements.

`CancellationPolicy`

Responsibility: Encapsulate a cancellation/refund rule.

Potential interface:

```
Plain text
cancellable?(booking, at:)
refund_amount(booking, at:)
```

Concrete classes implement different rules.

`CancellationService`

Responsibility: Orchestrate the cancellation use case.

Potential workflow:

```
Plain text
load booking
→ select policy
→ ask policy for cancellation/refund outcome
→ invoke payment/refund mechanism when applicable
→ mark booking cancelled
```

This prevents `Booking` from becoming both:

- reservation state holder,
- cancellation-rule engine,

- payment/refund coordinator.

Existing classes that can largely remain unchanged

`User` and `Setting` need no direct conceptual change for these requirements.

`Room` remains the source of room information and room pricing/availability.

`CrudConcern` remains the shared CRUD mechanism.

`ApplicationPolicy` and resource policies can remain responsible for authorization. New policies may be introduced only if the new operations require authorization rules.

`HasOwnerConcern` can remain unchanged.

`HasSettingsConcern` can remain unchanged.

`HasPriceConcern` may need to evolve because the booking total is no longer simply the room price × stay duration.

That is a significant point: the existing concern currently computes:

```
Plain text
total_ht = room.day_price × stay length
total_ttc = total_ht × (1 + vat)
```

Once optional services and upgrades affect the booking total, that algorithm needs a clear extension point rather than arbitrary additions to `HasPriceConcern`.

E. Confirmed evolved patterns

Pattern

Strategy

Category

GoF Behavioral

Intent

Encapsulate interchangeable cancellation algorithms so that the cancellation workflow does not contain a growing collection of policy-specific conditionals.

Participants

Proposed evolved participants:

- `CancellationService`
- `CancellationPolicy`
- `FlexibleCancellationPolicy`
- `PartialRefundCancellationPolicy`
- `NoRefundCancellationPolicy`
- `Booking`
- payment/refund boundary

Exact role mapping

Strategy role	Evolved application element
Context	`CancellationService`
Strategy	`CancellationPolicy`
Concrete Strategy	`FlexibleCancellationPolicy`
Concrete Strategy	`PartialRefundCancellationPolicy`
Concrete Strategy	`NoRefundCancellationPolicy`
Context's domain object	`Booking`
External collaborator	payment/refund mechanism

Collaborations / interactions

Proposed cancellation flow:

```
Plain text
Controller
    ↓
CancellationService
    ↓
CancellationPolicy
    ↓
refund/cancellation decision
    ↓
Payment/Refund boundary
    ↓
Booking cancellation state
```

The crucial variation is:

```
Plain text

CancellationPolicy
    ├── free cancellation
    ├── partial refund
    └── no refund
```

while the use case remains the same:

```
Plain text

cancel this booking
```

Why it qualifies

This proposed architecture introduces an actual family of interchangeable algorithms.

The context need not know the details of each cancellation rule. It depends on the policy abstraction and delegates the variable calculation to the selected concrete policy.

Adding a new cancellation rule therefore means adding another concrete implementation rather than changing the cancellation orchestration itself.

That is the defining Strategy collaboration.

Why alternative classifications were rejected

Template Method: rejected for cancellation because the variation is better represented by interchangeable policy objects rather than a fixed superclass algorithm plus subclass hooks.

State: rejected because the requirement concerns differing cancellation **rules**, not different objects representing booking states.

Command: rejected because a service call such as `cancel` does not become Command unless a command object encapsulates the request independently.

Observer: rejected because the requirement does not need subscriber registration/notification.

Factory: not inherently required. A factory may eventually be useful to select/instantiate the correct cancellation policy, but object creation alone does not establish Factory.

Decorator: rejected because the policy is not wrapping another policy object to augment behavior.

Status

Confirmed for the proposed evolved design; proposed until Part 3 implementation exists.

This distinction is essential: **Strategy is justified by the Part 2 design, but there is no evolved source code yet, so this is a proposed/target pattern, not a claim about the current repository.**

Existing Template Method

Pattern: Template Method
Category: GoF Behavioral
Instance: `CrudConcern`

Status: Confirmed and retained from Part 1.

The original collaboration is unchanged conceptually:

```
Plain text

CrudConcern
    ↓
shared CRUD algorithm
    ↓
controller-specific hooks
```

The new features do not require replacing it.

Additional-service abstraction

This is **not classified as a GoF pattern** at this stage.

It is a domain abstraction/composition design that creates:

```
Plain text

Booking
  → BookingService
      → AdditionalService
          ├── BreakfastPackage
          └── AirportTransfer
```

It may later support other patterns, but the abstraction itself does not prove one.

GRASP assessment

Information Expert — applicable as a responsibility-assignment principle.

Examples:

- `Room` knows room-specific data and pricing.
- `Booking` knows its booking dates and room relationship.
- `CancellationPolicy` knows cancellation-rule calculations.
- `AdditionalService` knows service-specific pricing.

This supports high cohesion and localized knowledge, but it is not counted as an additional GoF pattern.

Protected Variations — strongly applicable to the evolved design.

The most important protected variation is cancellation rules:

```
Plain text

CancellationService
       ↓
CancellationPolicy abstraction
```

The changing rule is protected behind a stable abstraction.

This is best treated as a GRASP design principle supporting the Strategy implementation, rather than as a separate GoF pattern instance.

F. Uncertain candidates

Candidate	Evidence	Missing evidence	Reason unce
Factory Method for cancellation-policy selection	Multiple concrete cancellation policies would need to be instantiated/selected	No factory is mandated; policy construction mechanism has not been designed	Could be ad currently unr Strategy
Factory for additional services	Breakfast and airport transfer have common abstraction and may be selected from user input	No dedicated creation abstraction is required	Simple polyn construction
Builder for booking	Booking may eventually require room + services +	No construction sequence with many mandatory/optional steps has yet	A Builder ma but not justif

Candidate	Evidence	Missing evidence	Reason unce
	upgrade + policy	been established	requirements
Decorator for service pricing	Multiple optional services affect total price	Proposed model uses composition/association, not wrapper objects	Could becom each service price compo behavior
Composite for additional services	Booking contains multiple services	Services are peers in a collection, not a recursive tree of components with uniform child semantics	Collection ≠
Strategy for service pricing	Different services may calculate price differently	The requirement does not yet demand interchangeable pricing algorithms selected independently from service type	Common me not enough
State for booking cancellation	Booking transitions from active to cancelled	No separate state objects or behavior-by-state hierarchy is required	Current requ on rule variat state represe
Command for room upgrade	Upgrade is a distinct use case	No command object or deferred execution requirement	A service/use enough
Observer for service changes	Price totals may react to service selection	No application-defined subscriptions/events	Callback/eve alone would Observer
Adapter around Payify refunds	Cancellation may need refund integration	Exact Payify refund interface is not visible in baseline	Requires insp actual depen deciding whe needed

G. Rejected candidates

Candidate	Why it may appear relevant	Why it does not qualify
Decorator for breakfast + airport transfer	Optional services “decorate” a booking conceptually	Conceptual augmentation is not GoF Decorator; the proposed design uses booking-service

Candidate	Why it may appear relevant	Why it does not qualify
		associations, not wrapper objects implementing the same interface
Composite for additional services	A booking can contain many services	There is no recursive whole-part hierarchy where individual services and collections share the same component interface
Factory Method for `Booking` creation	Booking construction becomes more complicated	Complexity alone does not create Factory Method; no creator/product hierarchy or creation-hook collaboration is required
Abstract Factory	Several domain objects may be created together	No related product families or family-level factory abstraction are needed
Builder	A booking may have many optional components	Optional fields alone do not justify Builder
Observer	Booking lifecycle currently has callbacks	Rails callbacks are framework lifecycle behavior and do not provide an application-defined observer/subscriber collaboration
State	Cancellation changes booking status	A boolean/timestamp status is not a State-pattern object hierarchy
Command	Cancellation and upgrade are actions	A method/service invocation is not a Command object
Adapter	Payment integration crosses a third-party boundary	A boundary exists, but the application has not demonstrated incompatible interfaces being translated by an adapter
Facade	A service layer could hide payment complexity	Merely hiding calls behind a service object is insufficient to establish a Facade over a subsystem
Decorator via Rails concerns	Existing concerns compose model behavior	`ActiveSupport::Concern` is direct module composition, not object wrapping

H. Before vs After comparison

Aspect	Original Design	Evolved Design
Booking responsibility	Reservation data, availability validation, booking pricing, cancellation timestamp, payment-related hooks	Reservation core remains, while optional services, upgrades, and cancellation rules are moved into dedicated collaborators
Pricing	`HasPriceConcern` computes room-price-based total + VAT	Base room price plus selected services and upgrade difference; pricing responsibilities can be localized by domain object/service
Optional services	None	`AdditionalService` with concrete `BreakfastPackage` and `AirportTransfer`, associated through booking-specific selections
Room upgrade	Not supported	Separate upgrade responsibility interacting with room availability and pricing
Cancellation	`Booking#cancel!` only sets `canceled_at`	`CancellationService` coordinates cancellation and delegates rule variation to `CancellationPolicy`
Extensibility	New behavior tends to enter `Booking` /concerns	New service types and cancellation rules can be introduced behind existing abstractions
Coupling	Booking already couples to pricing, payment callbacks, notification, room and user	Varying optional-service and cancellation behavior becomes more localized
Confirmed patterns	Template Method in `CrudConcern`	Existing Template Method retained; Strategy introduced for cancellation policy in the proposed evolved design

The key architectural change is not the number of classes. It is the movement of **independent variation out of `Booking`**.

I. Evolution scenarios

Scenario 1 — Add a new breakfast package

Without the proposed abstraction

A direct implementation could add more fields and conditionals to `Booking`:

```
Plain text
```

```
Booking
 ├── breakfast_selected
 ├── breakfast_type
 ├── breakfast_price
 └── conditional pricing logic
```

Adding another package then modifies the same model and pricing logic.

With the proposed abstraction

```
Plain text
```

```
BreakfastPackage
      ↓
BookingService
      ↓
Booking
```

Adding:

```
Plain text
```

```
Premium Breakfast
```

primarily means adding another service definition/configuration or concrete package representation.

The booking core does not need breakfast-specific branching.

Scenario 2 — Add a new airport transfer type

Without the proposed abstraction

The booking model/controller would acquire more transfer-specific fields and conditional calculations:

```
Plain text
```

```
pickup
dropoff
```

```
airport
vehicle
price
```

and changes would accumulate around booking creation and pricing.

With the proposed abstraction

A new transfer variant remains inside the airport-transfer domain:

```
Plain text

AirportTransfer
   ├── Pickup
   ├── Dropoff
   └── RoundTrip
```

The `Booking` collaboration remains:

```
Plain text

Booking → BookingService → AdditionalService
```

The common booking workflow does not need to know the concrete transfer implementation.

Scenario 3 — Add another cancellation policy

Without Strategy

`Booking` or a cancellation service would acquire another branch:

```
Plain text

if flexible
elsif partial
elsif no_refund
elsif new_policy
```

The central workflow grows every time a rule changes.

With Strategy

Add:

```
Plain text
```

```
NewCancellationPolicy < CancellationPolicy
```

and make policy selection resolve to it.

The cancellation orchestration remains unchanged.

This is the clearest demonstration of why Strategy is justified.

Scenario 4 — Add another room upgrade option

Without the proposed separation

The booking model would need more upgrade-specific logic:

```
Plain text
```

```
if upgrade_to_x
elsif upgrade_to_y
...
```

and would simultaneously be handling room availability and price difference.

With separate upgrade responsibility

The upgrade logic stays localized:

```
Plain text
```

```
RoomUpgrade
   ↓
checks target Room
   ↓
checks availability
   ↓
calculates difference
   ↓
applies upgrade
```

Adding another eligible room category therefore affects the upgrade logic/data rather than the entire booking model.

J. PlantUML diagrams

1. Evolved class diagram

plantuml

```
@startuml

class User {
  +admin?()
}

class Setting {
  +vat
}

class Room {
  +day_price
  +booked_dates()
}

class Booking {
  +start_date
  +end_date
  +total_ht
  +total_ttc
```

2. Cancellation Strategy diagram

plantuml

```
@startuml

class CancellationService {
  +cancel(booking)
}

interface CancellationPolicy {
  +cancellable?(booking, at)
  +refund_amount(booking, at)
}

class FlexibleCancellationPolicy
class PartialRefundCancellationPolicy
class NoRefundCancellationPolicy

CancellationService --> CancellationPolicy : delegates rule
CancellationPolicy <|.. FlexibleCancellationPolicy
CancellationPolicy <|.. PartialRefundCancellationPolicy
CancellationPolicy <|.. NoRefundCancellationPolicy
```

3. Additional-service structure

plantuml

```
@startuml

class Booking
class BookingService

abstract class AdditionalService {
  +name
  +price_for(booking)
}

class BreakfastPackage
class AirportTransfer

Booking "1" -- "0..*" BookingService
BookingService "*" --> "1" AdditionalService

AdditionalService <|-- BreakfastPackage
AdditionalService <|-- AirportTransfer

@enduml
```

4. Cancellation sequence

plantuml

```
@startuml

actor User
participant BookingsController
participant CancellationService
participant CancellationPolicy
participant PaymentBoundary
participant Booking

User -> BookingsController : cancel booking
BookingsController -> CancellationService : cancel(booking)
CancellationService -> CancellationPolicy : cancellable?(booking, now)

alt cancellation rejected
  CancellationPolicy --> CancellationService : false
  CancellationService --> BookingsController : reject
else cancellation allowed
  CancellationPolicy --> CancellationService : true
  CancellationService -> CancellationPolicy : refund_amount(booking, now)
  CancellationPolicy --> CancellationService : refund amount
```

K. Part 2 patterns.csv-ready evidence table

Because Part 2 is a **design-evolution analysis**, there are two different evidence classes:

1. **Existing baseline evidence retained into the evolved architecture**
2. **Proposed pattern evidence with no evolved source code yet**

No fabricated evolved line numbers are used.

source_version	pattern_name	pattern_type	instance_name	participant_roles
original	Template Method	GoF Behavioral	`CrudConcern` CRUD workflow	Template algorithm → `CrudConcern#index/n` `base_class` , `object_` `use_slug?` , `redirect` `BookingsController` ,
evolved	Strategy	GoF Behavioral	Cancellation policy selection/execution	Context → `Cancellati` `CancellationPolicy` ; `FlexibleCancellatio` `PartialRefundCancel` `NoRefundCancellatio`
evolved	—	Domain abstraction	Additional-service composition	`Booking` ; `BookingSer` `BreakfastPackage` ; `A`
evolved	—	Domain abstraction	Room-upgrade collaboration	`Booking` ; `RoomUpgrad`
evolved	Information Expert	GRASP	Responsibility separation for services/policies	`AdditionalService` ;
evolved	Protected Variations	GRASP	Cancellation-rule abstraction	`CancellationService`

L. Final Part 2 conclusion

The original design's main pressure point is **`Booking` becoming the natural place for every new booking-related rule**. In the baseline it already combines reservation data, availability validation,

pricing callbacks, cancellation state, payment-related behavior, and notification behavior. Breakfast, airport transfer, room upgrade, and richer cancellation rules would add multiple independent responsibilities if implemented directly there.

The proposed evolution introduces three important abstractions:

```
Plain text

AdditionalService
BookingService
CancellationPolicy
```

with a separate room-upgrade responsibility:

```
Plain text

RoomUpgrade
```

Breakfast and airport transfer share the **additional-service domain abstraction** because both are optional booking add-ons. Room upgrade remains distinct because it modifies the primary `Room` relationship and must coordinate availability and price difference.

The one new GoF pattern that is genuinely justified by the evolved requirements is:

Strategy — Cancellation Policy.

The reason is concrete: there is now a family of interchangeable cancellation/refund algorithms that should vary independently of the cancellation workflow.

The original **Template Method in `CrudConcern` remains valid and should be retained**. It continues to solve a different problem: reuse of controller CRUD workflow.

The candidates considered but rejected—Decorator, Composite, Factory, Builder, State, Command, Observer, Adapter, and Facade—should remain out of the final pattern inventory unless Part 3 implementation produces the defining collaborations required for one of them.

Part 3 should implement exactly these target elements

```
Plain text

Retain:
  CrudConcern
  BookingsController
  RoomsController
  SettingsController
```

```
Introduce:
  AdditionalService
  BreakfastPackage
  AirportTransfer
  BookingService
  RoomUpgrade
  CancellationPolicy
  FlexibleCancellationPolicy
  PartialRefundCancellationPolicy
  NoRefundCancellationPolicy
  CancellationService
```

The Part 3 implementation should then verify the design through actual source code, tests, and commit-level evidence. In particular, the implementation must establish the `CancellationService → CancellationPolicy` delegation concretely before the **Strategy** claim can be recorded as an implemented evolved pattern rather than a proposed design.

Prompt 05 — Part 3 Cross-Language Pattern Implementation and Metrics

Context

You are completing Part 3 of a Design Patterns Case Study based on the Ruby on Rails hotel booking application:

https://github.com/andrewdsilva/rails-room-booking/tree/main
GitHub

The immutable original baseline is:

- `source_version = original`
- commit = `5e15551ee646871b4bed32c982fa17da319d3ee3`

Part 1 established one confirmed GoF pattern in the original application:

...

Show more

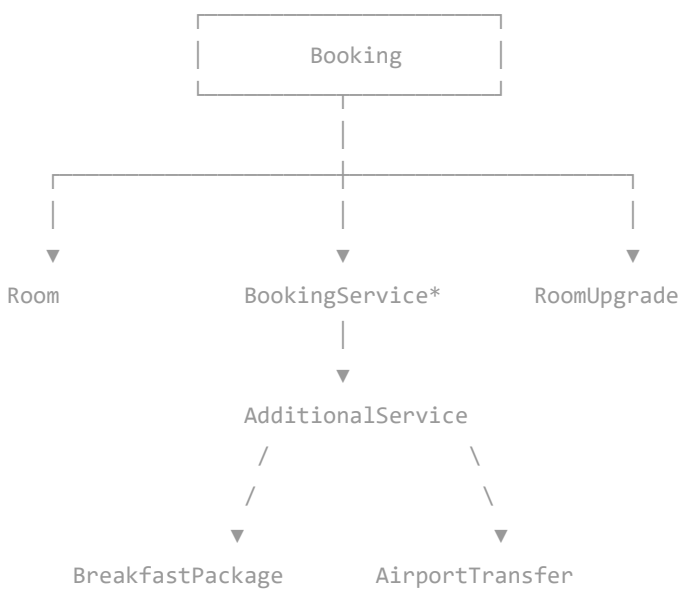
Worked for 1m 23s

1. Common reference design

The Part 3 implementation is a **compact cross-language demonstration derived from the verified baseline and Part 2 proposal**. It is not presented as a modification to the original Rails repository.

Language-neutral design

Plain text




```
Booking cancellation
        |
        ▼
```

The original Rails Template Method intent is:

Plain text

```
CrudConcern
     |
     ▼
shared CRUD workflow
     |
     ├── resource-specific hooks
     ▼
BookingsController
RoomsController
SettingsController
```

The cross-language implementations preserve that **design intent**, but use language-appropriate mechanisms rather than mechanically reproducing Ruby's `ActiveSupport::Concern`.

The common pricing model is:

Plain text

```
booking total
= room stay total
+ selected additional-service totals
```

Room upgrade contributes the target-room price difference over the booking stay.

Cancellation is:

Plain text

```
CancellationService
        ↓
CancellationPolicy
        ↓
selected cancellation algorithm
```

The implementations use three concrete policies plus a fourth demonstration policy to prove that the cancellation workflow is open to another policy without modifying `CancellationService` .

---

2. Java implementation

Source: `java/Main.java`

The Java version uses:

- interfaces for `AdditionalService` and `CancellationPolicy`
- an abstract class for the Template Method workflow
- normal Java inheritance for concrete controllers and policies
- `List<BookingService>` for optional services

Java implementation

```
import java.util.ArrayList;
import java.util.List;

class Room {
    final String name;
    final double nightlyRate;
    boolean available;

    Room(String name, double nightlyRate, boolean available) {
```

```
        this.name = name;
        this.nightlyRate = nightlyRate;
        this.available = available;
    }
}

interface AdditionalService {
    String name();
    double priceFor(Booking booking);
}

class BreakfastPackage implements AdditionalService {
    private final String packageName;
    private final double pricePerGuestPerNight;

    BreakfastPackage(String packageName, double pricePerGuestPerNight) {
        this.packageName = packageName;
        this.pricePerGuestPerNight = pricePerGuestPerNight;
    }

    public String name() { return packageName; }

    public double priceFor(Booking booking) {
        return pricePerGuestPerNight * booking.guests * booking.nights();
    }
}

class AirportTransfer implements AdditionalService {
    private final String transferType;
    private final double price;

    AirportTransfer(String transferType, double price) {
        this.transferType = transferType;
        this.price = price;
    }

    public String name() { return transferType; }

    public double priceFor(Booking booking) { return price; }
}
```

```
class BookingService {
    final AdditionalService service;
    final int quantity;

    BookingService(AdditionalService service, int quantity) {
        this.service = service;
        this.quantity = quantity;
    }

    double totalFor(Booking booking) {
        return service.priceFor(booking) * quantity;
    }
}

class RoomUpgrade {
    private final Room targetRoom;

    RoomUpgrade(Room targetRoom) {
        this.targetRoom = targetRoom;
    }

    boolean availableFor(Booking booking) {
        return targetRoom.available;
    }

    double priceDifference(Booking booking) {
        return (targetRoom.nightlyRate - booking.room.nightlyRate) * booking
    }

    void apply(Booking booking) {
        if (!availableFor(booking)) {
            throw new IllegalStateException("Target room unavailable");
        }
        booking.room = targetRoom;
    }
}

class Booking {
    Room room;
```

```
        final int nights;
        final int guests;
        final List<BookingService> services = new ArrayList<>();
        double recordedRefund;
        boolean cancelled;

        Booking(Room room, int nights, int guests) {
            this.room = room;
            this.nights = nights;
            this.guests = guests;
        }

        int nights() {
            return nights;
        }

        void addService(AdditionalService service, int quantity) {
            services.add(new BookingService(service, quantity));
        }

        double total() {
            double total = room.nightlyRate * nights;
            for (BookingService service : services) {
                total += service.totalFor(this);
            }
            return total;
        }
    }

    interface CancellationPolicy {
        boolean canCancel(Booking booking, int daysBeforeStay);
        double refundAmount(Booking booking, int daysBeforeStay);
    }

    class FlexibleCancellationPolicy implements CancellationPolicy {
        public boolean canCancel(Booking booking, int daysBeforeStay) {
            return true;
        }

        public double refundAmount(Booking booking, int daysBeforeStay) {
```

```
            return booking.total();
        }
    }

    class PartialRefundCancellationPolicy implements CancellationPolicy {
        public boolean canCancel(Booking booking, int daysBeforeStay) {
            return true;
        }

        public double refundAmount(Booking booking, int daysBeforeStay) {
            return booking.total() * 0.5;
        }
    }

    class NoRefundCancellationPolicy implements CancellationPolicy {
        public boolean canCancel(Booking booking, int daysBeforeStay) {
            return true;
        }

        public double refundAmount(Booking booking, int daysBeforeStay) {
            return 0.0;
        }
    }

    class CancellationService {
        private final CancellationPolicy policy;

        CancellationService(CancellationPolicy policy) {
            this.policy = policy;
        }

        double cancel(Booking booking, int daysBeforeStay) {
            if (!policy.canCancel(booking, daysBeforeStay)) {
                throw new IllegalStateException("Cancellation not allowed");
            }

            double refund = policy.refundAmount(booking, daysBeforeStay);
            booking.recordedRefund = refund;
            booking.cancelled = true;
            return refund;
```

```
    }
}

abstract class CrudWorkflow {
    final String createResource() {
        beforeCreate();
        String result = buildResource();
        afterCreate(result);
        return result;
    }

    protected void beforeCreate() {}
    protected abstract String buildResource();
    protected void afterCreate(String resource) {}
}

class BookingController extends CrudWorkflow {
    protected String buildResource() {
        return "Booking resource: reservation form";
    }

    protected void afterCreate(String resource) {
        System.out.println(resource + " created");
    }
}

class RoomController extends CrudWorkflow {
    protected String buildResource() {
        return "Room resource: room form";
    }
}

class SettingsController extends CrudWorkflow {
    protected String buildResource() {
        return "Settings resource: settings form";
    }
}

public class Main {
    public static void main(String[] args) {
```

```
        Room standard = new Room("Standard", 100, true);
        Room deluxe = new Room("Deluxe", 150, true);

        Booking booking = new Booking(standard, 3, 2);
        System.out.printf("Basic booking total: %.2f%n", booking.total());

        booking.addService(new BreakfastPackage("Breakfast", 15), 1);
        System.out.printf("With breakfast: %.2f%n", booking.total());

        booking.addService(new AirportTransfer("Airport transfer", 40), 1);
        System.out.printf("With transfer: %.2f%n", booking.total());

        double beforeUpgrade = booking.total();
        RoomUpgrade upgrade = new RoomUpgrade(deluxe);

        System.out.printf(
            "Room upgrade difference: %.2f%n",
            upgrade.priceDifference(booking)
        );

        upgrade.apply(booking);

        System.out.printf("After room upgrade: %.2f%n", booking.total());
        System.out.printf(
            "Upgrade delta in total: %.2f%n",
            booking.total() - beforeUpgrade
        );

        Booking flexibleBooking = new Booking(standard, 3, 2);
        flexibleBooking.addService(
            new BreakfastPackage("Breakfast", 15), 1
        );
        flexibleBooking.addService(
            new AirportTransfer("Airport transfer", 40), 1
        );

        System.out.printf(
            "Flexible refund: %.2f%n",
            new CancellationService(
                new FlexibleCancellationPolicy()
```

```
        ).cancel(flexibleBooking, 10)
);

System.out.printf(
    "Partial refund: %.2f%n",
    new CancellationService(
        new PartialRefundCancellationPolicy()
    ).cancel(flexibleBooking, 2)
);

System.out.printf(
    "No refund: %.2f%n",
    new CancellationService(
        new NoRefundCancellationPolicy()
    ).cancel(flexibleBooking, 0)
);

class WeatherClosurePolicy implements CancellationPolicy {
    public boolean canCancel(Booking b, int daysBeforeStay) {
        return true;
    }

    public double refundAmount(Booking b, int daysBeforeStay) {
        return b.total() * 0.75;
    }
}

System.out.printf(
    "New 75%% policy refund: %.2f%n",
    new CancellationService(
        new WeatherClosurePolicy()
    ).cancel(new Booking(standard, 2, 1), 1)
);

System.out.println(
    new BookingController().createResource()
);
System.out.println(
    new RoomController().createResource()
);
```

```
            System.out.println(
                new SettingsController().createResource()
            );
        }
    }
```

Java pattern roles

Strategy

```
Plain text
```

```
CancellationService
        |
        ▼
CancellationPolicy
   ├── FlexibleCancellationPolicy
   ├── PartialRefundCancellationPolicy
   └── NoRefundCancellationPolicy
```

The actual delegation occurs in `CancellationService.cancel()` through:

```
Java
```

```
policy.canCancel(...)
policy.refundAmount(...)
```

Template Method

`CrudWorkflow.createResource()` fixes the algorithm:

```
Plain text
```

```
beforeCreate
→ buildResource
→ afterCreate
```

while subclasses implement/override the extension points.

---

3. Python implementation

Source: `python/main.py`

The Python implementation intentionally avoids Java-style interface boilerplate. It uses `abc.ABC` where an explicit contract is useful and normal Python polymorphism for the concrete strategies.

Python implementation

```
from abc import ABC, abstractmethod


class Room:
    def __init__(self, name, nightly_rate, available=True):
        self.name = name
        self.nightly_rate = nightly_rate
        self.available = available


class AdditionalService(ABC):
    @abstractmethod
    def name(self):
        pass

    @abstractmethod
    def price_for(self, booking):
        pass


class BreakfastPackage(AdditionalService):
    def __init__(self, package_name, price_per_guest_per_night):
        self.package_name = package_name
        self.price_per_guest_per_night = price_per_guest_per_night

    def name(self):
        return self.package_name

    def price_for(self, booking):
        return (
            self.price_per_guest_per_night
            * booking.guests
            * booking.nights
        )
```

```
class AirportTransfer(AdditionalService):
    def __init__(self, transfer_type, price):
        self.transfer_type = transfer_type
        self.price = price

    def name(self):
        return self.transfer_type

    def price_for(self, booking):
        return self.price


class BookingService:
    def __init__(self, service, quantity=1):
        self.service = service
        self.quantity = quantity

    def total_for(self, booking):
        return self.service.price_for(booking) * self.quantity


class RoomUpgrade:
    def __init__(self, target_room):
        self.target_room = target_room

    def price_difference(self, booking):
        return (
            self.target_room.nightly_rate
            - booking.room.nightly_rate
        ) * booking.nights

    def apply(self, booking):
        if not self.target_room.available:
            raise ValueError("Target room unavailable")
        booking.room = self.target_room


class Booking:
    def __init__(self, room, nights, guests):
```

```
        self.room = room
        self.nights = nights
        self.guests = guests
        self.services = []
        self.cancelled = False
        self.recorded_refund = 0.0

    def add_service(self, service, quantity=1):
        self.services.append(
            BookingService(service, quantity)
        )

    def total(self):
        return (
            self.room.nightly_rate * self.nights
            + sum(
                item.total_for(self)
                for item in self.services
            )
        )


class CancellationPolicy(ABC):
    @abstractmethod
    def can_cancel(self, booking, days_before_stay):
        pass

    @abstractmethod
    def refund_amount(self, booking, days_before_stay):
        pass


class FlexibleCancellationPolicy(CancellationPolicy):
    def can_cancel(self, booking, days_before_stay):
        return True

    def refund_amount(self, booking, days_before_stay):
        return booking.total()
```

```
class PartialRefundCancellationPolicy(CancellationPolicy):
    def can_cancel(self, booking, days_before_stay):
        return True

    def refund_amount(self, booking, days_before_stay):
        return booking.total() * 0.5


class NoRefundCancellationPolicy(CancellationPolicy):
    def can_cancel(self, booking, days_before_stay):
        return True

    def refund_amount(self, booking, days_before_stay):
        return 0.0


class CancellationService:
    def __init__(self, policy):
        self.policy = policy

    def cancel(self, booking, days_before_stay):
        if not self.policy.can_cancel(
            booking,
            days_before_stay
        ):
            raise ValueError("Cancellation not allowed")

        refund = self.policy.refund_amount(
            booking,
            days_before_stay
        )
        booking.recorded_refund = refund
        booking.cancelled = True
        return refund


class CrudWorkflow(ABC):
    def create_resource(self):
        self.before_create()
        resource = self.build_resource()
```

```
        self.after_create(resource)
        return resource

    def before_create(self):
        pass

    @abstractmethod
    def build_resource(self):
        pass

    def after_create(self, resource):
        pass


class BookingController(CrudWorkflow):
    def build_resource(self):
        return "Booking resource: reservation form"

    def after_create(self, resource):
        print(resource + " created")


class RoomController(CrudWorkflow):
    def build_resource(self):
        return "Room resource: room form"


class SettingsController(CrudWorkflow):
    def build_resource(self):
        return "Settings resource: settings form"


class WeatherClosurePolicy(CancellationPolicy):
    def can_cancel(self, booking, days_before_stay):
        return True

    def refund_amount(self, booking, days_before_stay):
        return booking.total() * 0.75
```

```
def main():
    standard = Room("Standard", 100)
    deluxe = Room("Deluxe", 150)

    booking = Booking(standard, 3, 2)
    print(f"Basic booking total: {booking.total():.2f}")

    booking.add_service(
        BreakfastPackage("Breakfast", 15)
    )
    print(f"With breakfast: {booking.total():.2f}")

    booking.add_service(
        AirportTransfer("Airport transfer", 40)
    )
    print(f"With transfer: {booking.total():.2f}")

    before_upgrade = booking.total()
    upgrade = RoomUpgrade(deluxe)

    print(
        f"Room upgrade difference: "
        f"{upgrade.price_difference(booking):.2f}"
    )

    upgrade.apply(booking)

    print(f"After room upgrade: {booking.total():.2f}")
    print(
        f"Upgrade delta in total: "
        f"{booking.total() - before_upgrade:.2f}"
    )

    flexible_booking = Booking(standard, 3, 2)
    flexible_booking.add_service(
        BreakfastPackage("Breakfast", 15)
    )
    flexible_booking.add_service(
        AirportTransfer("Airport transfer", 40)
    )
```

```
    print(
        f"Flexible refund: "
        f"{CancellationService(FlexibleCancellationPolicy()).cancel(flexible
    )

    print(
        f"Partial refund: "
        f"{CancellationService(PartialRefundCancellationPolicy()).cancel(fle
    )

    print(
        f"No refund: "
        f"{CancellationService(NoRefundCancellationPolicy()).cancel(flexible
    )

    print(
        f"New 75% policy refund: "
        f"{CancellationService(WeatherClosurePolicy()).cancel(Booking(standa
    )

    print(BookingController().create_resource())
    print(RoomController().create_resource())
    print(SettingsController().create_resource())


if __name__ == "__main__":
    main()
```

Python-specific design difference

Python does not need the Java-style declaration of a formal interface for every polymorphic relationship. `ABC` is used where an explicit contract helps communicate the Strategy and service abstractions.

---

4. JavaScript implementation

Source: `javascript/main.js`

The JavaScript implementation uses ES6 classes. JavaScript's structural/dynamic nature means the “interfaces” are represented by base classes whose methods establish the expected protocol.

JavaScript implementation

```
class Room {
    constructor(name, nightlyRate, available = true) {
        this.name = name;
        this.nightlyRate = nightlyRate;
        this.available = available;
    }
}

class AdditionalService {
    getName() {
        throw new Error("getName must be implemented");
    }

    priceFor() {
        throw new Error("priceFor must be implemented");
    }
}

class BreakfastPackage extends AdditionalService {
    constructor(packageName, pricePerGuestPerNight) {
        super();
        this.packageName = packageName;
        this.pricePerGuestPerNight = pricePerGuestPerNight;
    }

    getName() {
        return this.packageName;
    }

    priceFor(booking) {
        return (
            this.pricePerGuestPerNight
            * booking.guests
            * booking.nights
        );
```

```
    }
}

class AirportTransfer extends AdditionalService {
    constructor(transferType, price) {
        super();
        this.transferType = transferType;
        this.price = price;
    }

    getName() {
        return this.transferType;
    }

    priceFor() {
        return this.price;
    }
}

class BookingService {
    constructor(service, quantity = 1) {
        this.service = service;
        this.quantity = quantity;
    }

    totalFor(booking) {
        return this.service.priceFor(booking) * this.quantity;
    }
}

class Booking {
    constructor(room, nights, guests) {
        this.room = room;
        this.nights = nights;
        this.guests = guests;
        this.services = [];
        this.cancelled = false;
        this.recordedRefund = 0;
    }
```

```
    addService(service, quantity = 1) {
        this.services.push(
            new BookingService(service, quantity)
        );
    }

    total() {
        return (
            this.room.nightlyRate * this.nights
            + this.services.reduce(
                (sum, item) => sum + item.totalFor(this),
                0
            )
        );
    }
}

class RoomUpgrade {
    constructor(targetRoom) {
        this.targetRoom = targetRoom;
    }

    priceDifference(booking) {
        return (
            this.targetRoom.nightlyRate
            - booking.room.nightlyRate
        ) * booking.nights;
    }

    apply(booking) {
        if (!this.targetRoom.available) {
            throw new Error("Target room unavailable");
        }
        booking.room = this.targetRoom;
    }
}

class CancellationPolicy {
    canCancel() {
        throw new Error("canCancel must be implemented");
```

```
        }

        refundAmount() {
            throw new Error("refundAmount must be implemented");
        }
    }

    class FlexibleCancellationPolicy extends CancellationPolicy {
        canCancel() {
            return true;
        }

        refundAmount(booking) {
            return booking.total();
        }
    }

    class PartialRefundCancellationPolicy extends CancellationPolicy {
        canCancel() {
            return true;
        }

        refundAmount(booking) {
            return booking.total() * 0.5;
        }
    }

    class NoRefundCancellationPolicy extends CancellationPolicy {
        canCancel() {
            return true;
        }

        refundAmount() {
            return 0;
        }
    }

    class CancellationService {
        constructor(policy) {
            this.policy = policy;
```

```
        }

        cancel(booking, daysBeforeStay) {
            if (!this.policy.canCancel(
                booking,
                daysBeforeStay
            )) {
                throw new Error("Cancellation not allowed");
            }

            const refund = this.policy.refundAmount(
                booking,
                daysBeforeStay
            );

            booking.recordedRefund = refund;
            booking.cancelled = true;

            return refund;
        }
    }

    class CrudWorkflow {
        createResource() {
            this.beforeCreate();
            const resource = this.buildResource();
            this.afterCreate(resource);
            return resource;
        }

        beforeCreate() {}

        buildResource() {
            throw new Error("buildResource must be implemented");
        }

        afterCreate() {}
    }

    class BookingController extends CrudWorkflow {
```

```
        buildResource() {
            return "Booking resource: reservation form";
        }

        afterCreate(resource) {
            console.log(resource + " created");
        }
    }

    class RoomController extends CrudWorkflow {
        buildResource() {
            return "Room resource: room form";
        }
    }

    class SettingsController extends CrudWorkflow {
        buildResource() {
            return "Settings resource: settings form";
        }
    }

    class WeatherClosurePolicy extends CancellationPolicy {
        canCancel() {
            return true;
        }

        refundAmount(booking) {
            return booking.total() * 0.75;
        }
    }

    function main() {
        const standard = new Room("Standard", 100);
        const deluxe = new Room("Deluxe", 150);

        const booking = new Booking(standard, 3, 2);
        console.log(
            `Basic booking total: ${booking.total().toFixed(2)}`
        );
```

```
booking.addService(
    new BreakfastPackage("Breakfast", 15)
);
console.log(
    `With breakfast: ${booking.total().toFixed(2)}`
);

booking.addService(
    new AirportTransfer("Airport transfer", 40)
);
console.log(
    `With transfer: ${booking.total().toFixed(2)}`
);

const beforeUpgrade = booking.total();
const upgrade = new RoomUpgrade(deluxe);

console.log(
    `Room upgrade difference: ${upgrade.priceDifference(booking).toFixed
);

upgrade.apply(booking);

console.log(
    `After room upgrade: ${booking.total().toFixed(2)}`
);

console.log(
    `Upgrade delta in total: ${(booking.total() - beforeUpgrade).toFixed
);

const flexibleBooking = new Booking(standard, 3, 2);

flexibleBooking.addService(
    new BreakfastPackage("Breakfast", 15)
);

flexibleBooking.addService(
    new AirportTransfer("Airport transfer", 40)
);
```

```
    console.log(
        `Flexible refund: ${new CancellationService(
            new FlexibleCancellationPolicy()
        ).cancel(flexibleBooking, 10).toFixed(2)}`
    );

    console.log(
        `Partial refund: ${new CancellationService(
            new PartialRefundCancellationPolicy()
        ).cancel(flexibleBooking, 2).toFixed(2)}`
    );

    console.log(
        `No refund: ${new CancellationService(
            new NoRefundCancellationPolicy()
        ).cancel(flexibleBooking, 0).toFixed(2)}`
    );

    console.log(
        `New 75% policy refund: ${new CancellationService(
            new WeatherClosurePolicy()
        ).cancel(new Booking(standard, 2, 1), 1).toFixed(2)}`
    );

    console.log(
        new BookingController().createResource()
    );

    console.log(
        new RoomController().createResource()
    );

    console.log(
        new SettingsController().createResource()
    );
}

main();
```

JavaScript-specific difference

There is no language-enforced interface equivalent to Java's interface or C++ pure-virtual class. The base classes establish the expected protocol, while JavaScript runtime dispatch provides polymorphism.

---

5. C++ implementation

Source: `cpp/main.cpp`

The C++ implementation uses:

- abstract base classes,
- virtual functions,
- `override`,
- `std::shared_ptr` for runtime-polymorphic strategies/services,
- protected virtual hooks for Template Method.

C++ implementation

```
#include <iostream>
#include <memory>
#include <stdexcept>
#include <string>
#include <utility>
#include <vector>

class Room {
public:
    std::string name;
    double nightly_rate;
    bool available;

    Room(std::string name, double nightly_rate, bool available = true)
        : name(std::move(name)),
          nightly_rate(nightly_rate),
          available(available) {}
};

class Booking;
```

```
class AdditionalService {
public:
    virtual ~AdditionalService() = default;
    virtual std::string name() const = 0;
    virtual double price_for(const Booking& booking) const = 0;
};

class BookingService {
public:
    std::shared_ptr<AdditionalService> service;
    int quantity;

    BookingService(
        std::shared_ptr<AdditionalService> service,
        int quantity = 1
    )
        : service(std::move(service)),
          quantity(quantity) {}

    double total_for(const Booking& booking) const;
};

class Booking {
public:
    Room* room;
    int nights;
    int guests;
    std::vector<BookingService> services;
    double recorded_refund = 0.0;
    bool cancelled = false;

    Booking(Room& room, int nights, int guests)
        : room(&room),
          nights(nights),
          guests(guests) {}

    void add_service(
        std::shared_ptr<AdditionalService> service,
        int quantity = 1
```

```
        ) {
            services.emplace_back(
                std::move(service),
                quantity
            );
        }

        double total() const {
            double total =
                room->nightly_rate * nights;

            for (const auto& service : services) {
                total += service.total_for(*this);
            }

            return total;
        }
    };

    double BookingService::total_for(
        const Booking& booking
    ) const {
        return service->price_for(booking) * quantity;
    }

    class BreakfastPackage : public AdditionalService {
        std::string package_name;
        double price_per_guest_per_night;

    public:
        BreakfastPackage(
            std::string package_name,
            double price
        )
            : package_name(std::move(package_name)),
              price_per_guest_per_night(price) {}

        std::string name() const override {
            return package_name;
        }
```

```
    double price_for(
        const Booking& booking
    ) const override {
        return price_per_guest_per_night
            * booking.guests
            * booking.nights;
    }
};

class AirportTransfer : public AdditionalService {
    std::string transfer_type;
    double price;

public:
    AirportTransfer(
        std::string transfer_type,
        double price
    )
        : transfer_type(std::move(transfer_type)),
          price(price) {}

    std::string name() const override {
        return transfer_type;
    }

    double price_for(
        const Booking&
    ) const override {
        return price;
    }
};

class RoomUpgrade {
    Room& target_room;

public:
    explicit RoomUpgrade(Room& target_room)
        : target_room(target_room) {}
```

```
    double price_difference(
        const Booking& booking
    ) const {
        return (
            target_room.nightly_rate
            - booking.room->nightly_rate
        ) * booking.nights;
    }

    void apply(Booking& booking) const {
        if (!target_room.available) {
            throw std::runtime_error(
                "Target room unavailable"
            );
        }

        booking.room = &target_room;
    }
};

class CancellationPolicy {
public:
    virtual ~CancellationPolicy() = default;

    virtual bool can_cancel(
        const Booking& booking,
        int days_before_stay
    ) const = 0;

    virtual double refund_amount(
        const Booking& booking,
        int days_before_stay
    ) const = 0;
};

class FlexibleCancellationPolicy
    : public CancellationPolicy {

public:
    bool can_cancel(
```

```
            const Booking&,
            int
        ) const override {
            return true;
        }

        double refund_amount(
            const Booking& booking,
            int
        ) const override {
            return booking.total();
        }
    };

    class PartialRefundCancellationPolicy
        : public CancellationPolicy {

    public:
        bool can_cancel(
            const Booking&,
            int
        ) const override {
            return true;
        }

        double refund_amount(
            const Booking& booking,
            int
        ) const override {
            return booking.total() * 0.5;
        }
    };

    class NoRefundCancellationPolicy
        : public CancellationPolicy {

    public:
        bool can_cancel(
            const Booking&,
            int
```

```
        ) const override {
            return true;
        }

        double refund_amount(
            const Booking&,
            int
        ) const override {
            return 0.0;
        }
    };

    class CancellationService {
        std::shared_ptr<CancellationPolicy> policy;

    public:
        explicit CancellationService(
            std::shared_ptr<CancellationPolicy> policy
        )
            : policy(std::move(policy)) {}

        double cancel(
            Booking& booking,
            int days_before_stay
        ) const {
            if (!policy->can_cancel(
                booking,
                days_before_stay
            )) {
                throw std::runtime_error(
                    "Cancellation not allowed"
                );
            }

            double refund =
                policy->refund_amount(
                    booking,
                    days_before_stay
                );
```

```
        booking.recorded_refund = refund;
        booking.cancelled = true;

        return refund;
    }
};

class CrudWorkflow {
public:
    virtual ~CrudWorkflow() = default;

    std::string create_resource() {
        before_create();

        auto resource = build_resource();

        after_create(resource);

        return resource;
    }

protected:
    virtual void before_create() {}

    virtual std::string build_resource()
        const = 0;

    virtual void after_create(
        const std::string&
    ) {}
};

class BookingController
    : public CrudWorkflow {

protected:
    std::string build_resource()
        const override {
        return "Booking resource: reservation form";
    }
```

```
    void after_create(
        const std::string& resource
    ) override {
        std::cout
            << resource
            << " created\n";
    }
};

class RoomController
    : public CrudWorkflow {

protected:
    std::string build_resource()
        const override {
        return "Room resource: room form";
    }
};

class SettingsController
    : public CrudWorkflow {

protected:
    std::string build_resource()
        const override {
        return "Settings resource: settings form";
    }
};

class WeatherClosurePolicy
    : public CancellationPolicy {

public:
    bool can_cancel(
        const Booking&,
        int
    ) const override {
        return true;
    }
```

```
    double refund_amount(
        const Booking& booking,
        int
    ) const override {
        return booking.total() * 0.75;
    }
};

int main() {
    Room standard("Standard", 100);
    Room deluxe("Deluxe", 150);

    Booking booking(standard, 3, 2);

    std::cout
        << "Basic booking total: "
        << booking.total()
        << '\n';

    booking.add_service(
        std::make_shared<BreakfastPackage>(
            "Breakfast",
            15
        )
    );

    std::cout
        << "With breakfast: "
        << booking.total()
        << '\n';

    booking.add_service(
        std::make_shared<AirportTransfer>(
            "Airport transfer",
            40
        )
    );

    std::cout
```

```
        << "With transfer: "
        << booking.total()
        << '\n';

    double before_upgrade = booking.total();

    RoomUpgrade upgrade(deluxe);

    std::cout
        << "Room upgrade difference: "
        << upgrade.price_difference(booking)
        << '\n';

    upgrade.apply(booking);

    std::cout
        << "After room upgrade: "
        << booking.total()
        << '\n';

    std::cout
        << "Upgrade delta in total: "
        << booking.total() - before_upgrade
        << '\n';

    Booking flexible_booking(
        standard,
        3,
        2
    );

    flexible_booking.add_service(
        std::make_shared<BreakfastPackage>(
            "Breakfast",
            15
        )
    );

    flexible_booking.add_service(
        std::make_shared<AirportTransfer>(
```

```
            "Airport transfer",
            40
        )
    );

    std::cout
        << "Flexible refund: "
        << CancellationService(
            std::make_shared<
                FlexibleCancellationPolicy
            >()
        ).cancel(
            flexible_booking,
            10
        )
        << '\n';

    Booking partial_booking(
        standard,
        3,
        2
    );

    partial_booking.add_service(
        std::make_shared<BreakfastPackage>(
            "Breakfast",
            15
        )
    );

    partial_booking.add_service(
        std::make_shared<AirportTransfer>(
            "Airport transfer",
            40
        )
    );

    std::cout
        << "Partial refund: "
        << CancellationService(
```

```
            std::make_shared<
                PartialRefundCancellationPolicy
            >()
        ).cancel(
            partial_booking,
            2
        )
        << '\n';

    Booking no_refund_booking(
        standard,
        3,
        2
    );

    no_refund_booking.add_service(
        std::make_shared<BreakfastPackage>(
            "Breakfast",
            15
        )
    );

    no_refund_booking.add_service(
        std::make_shared<AirportTransfer>(
            "Airport transfer",
            40
        )
    );

    std::cout
        << "No refund: "
        << CancellationService(
            std::make_shared<
                NoRefundCancellationPolicy
            >()
        ).cancel(
            no_refund_booking,
            0
        )
        << '\n';
```

```
    Booking weather_booking(
        standard,
        2,
        1
    );

    std::cout
        << "New 75% policy refund: "
        << CancellationService(
            std::make_shared<
                WeatherClosurePolicy
            >()
        ).cancel(
            weather_booking,
            1
        )
        << '\n';

    std::cout
        << BookingController()
               .create_resource()
        << '\n';

    std::cout
        << RoomController()
               .create_resource()
        << '\n';

    std::cout
        << SettingsController()
               .create_resource()
        << '\n';
}
```

C++-specific difference

C++ makes runtime polymorphism explicit through abstract base classes and virtual dispatch. `std::shared_ptr<CancellationPolicy>` allows `CancellationService` to hold different concrete policy objects through a common base type.

6. Template Method cross-language mapping

Language	Template Method abstraction	Concrete participants	Hook/extension mechanism
Ruby baseline	`CrudConcern`	`BookingsController`, `RoomsController`, `SettingsController`	Module inclusion + method overriding
Java	`CrudWorkflow`	`BookingController`, `RoomController`, `SettingsController`	Abstract class + protected overridable methods
Python	`CrudWorkflow`	`BookingController`, `RoomController`, `SettingsController`	`ABC` + overridden `build_resource()` and optional hooks
JavaScript	`CrudWorkflow`	`BookingController`, `RoomController`, `SettingsController`	ES6 inheritance + overridden methods
C++	`CrudWorkflow`	`BookingController`, `RoomController`, `SettingsController`	Abstract base class + virtual/protected hooks

The important equivalence is the **control structure**, not the syntax:

```
common algorithm
      ↓
hook/extension point
      ↓
resource-specific behavior
```

The Ruby baseline achieves this through a module because Rails/Ruby does not require the classical abstract-class implementation used in Java and C++.

---

7. Strategy cross-language mapping

Language	Context	Strategy	Concrete Strategies	Delegated variation
Java	`CancellationService`	`CancellationPolicy`	`FlexibleCancellationPolicy`, `PartialRefundCancellationPolicy`, `NoRefundCancellationPolicy`	cancellation eligibility a refund amount
Python	`CancellationService`	`CancellationPolicy`	`FlexibleCancellationPolicy`, `PartialRefundCancellationPolicy`, `NoRefundCancellationPolicy`	cancellation eligibility a refund amount
JavaScript	`CancellationService`	`CancellationPolicy`	`FlexibleCancellationPolicy`, `PartialRefundCancellationPolicy`, `NoRefundCancellationPolicy`	cancellation eligibility a refund amount
C++	`CancellationService`	`CancellationPolicy`	`FlexibleCancellationPolicy`, `PartialRefundCancellationPolicy`, `NoRefundCancellationPolicy`	cancellation eligibility a refund amount

Every implementation contains the same collaboration:

Plain text

```
CancellationService
        │
        │ delegates
        ▼
CancellationPolicy
        │
        ├── FlexibleCancellationPolicy
```

```
        ├── PartialRefundCancellationPolicy
        └── NoRefundCancellationPolicy
```

The new-policy demonstration adds:

```
Plain text
```

```
WeatherClosurePolicy
```

without modifying `CancellationService`.

That is the most direct executable evidence that the varying cancellation algorithm has been separated from the workflow.

---

8. Demonstration scenarios and expected outputs

The four programs were actually executed in the container:

- Java: compiled and executed with `javac / java`
- Python: executed with `python`
- JavaScript: executed with `node`
- C++: compiled with `g++ -std=c++17 -O2 -Wall -Wextra -pedantic` and executed

The same conceptual results were produced.

Expected/verified output

For Java, Python, and JavaScript:

```
Plain text
```

```
Basic booking total: 300.00
With breakfast: 390.00
With transfer: 430.00
Room upgrade difference: 150.00
After room upgrade: 580.00
Upgrade delta in total: 150.00
Flexible refund: 430.00
Partial refund: 215.00
No refund: 0.00
New 75% policy refund: 150.00
Booking resource: reservation form created
Booking resource: reservation form
Room resource: room form
Settings resource: settings form
```

The C++ output is numerically equivalent, with default stream formatting:

```
Plain text
```

```
Basic booking total: 300
With breakfast: 390
With transfer: 430
Room upgrade difference: 150
After room upgrade: 580
Upgrade delta in total: 150
Flexible refund: 430
Partial refund: 215
No refund: 0
New 75% policy refund: 150
Booking resource: reservation form created
Booking resource: reservation form
Room resource: room form
Settings resource: settings form
```

Scenario interpretation:

Scenario	Demonstrated behavior
A	Standard room booking = `100 × 3 = 300`
B	Breakfast = `15 × 2 guests × 3 nights = 90` ; total `390`
C	Airport transfer adds `40` ; total `430`
D	Upgrade from `100` to `150` for 3 nights adds `150` ; total `580`
E	Flexible cancellation refunds `430`
F	Partial policy refunds `215`
G	No-refund policy returns `0`
H	New 75% policy returns `150` for a `200` booking without changing cancellation workflow

---

9. Cross-language comparison

Design element	Java	Python	JavaScript	C++
Strategy abstraction	`CancellationPolicy` interface	`CancellationPolicy(ABC)`	`CancellationPolicy` base class	Abstract `CancellationPolicy`
Concrete strategies	Three named classes	Three named classes	Three named classes	Three named classes
Context	`CancellationService`	`CancellationService`	`CancellationService`	`CancellationService`
Template Method	`CrudWorkflow` abstract class	`CrudWorkflow(ABC)`	`CrudWorkflow` class	`CrudWorkflow` abstrac class
Additional services	Interface + concrete services	ABC + concrete services	Base class + concrete services	Abstract base class + `shared_ptr`
Room upgrade	`RoomUpgrade` class	`RoomUpgrade` class	`RoomUpgrade` class	`RoomUpgrade` class
Polymorphism mechanism	Interfaces + inheritance	Duck typing/ABC + inheritance	Dynamic dispatch + inheritance	Virtual dispatch
Main language-specific difference	Explicit static interfaces	Lightweight polymorphism and ABCs	Runtime protocol rather than enforced interfaces	Explicit virtual inheritan ownership semantics

The architectural design remains constant while the implementation mechanics differ.

For example:

```
Java     → interface
Python   → ABC / normal polymorphism
JS       → runtime class protocol
C++      → abstract class + virtual functions
```

These differences are language characteristics, not changes in the underlying pattern.

10. Metrics

The following metrics were derived from the actual final source files generated for this Part 3 implementation.

Metric	Java	Python	JavaScript	C++
Files	1	1	1	1
Physical LOC	214	187	236	219
Class declarations	16	17	17	18
Explicit interfaces	2	0	0	0
Explicit abstract classes	1	3 (`ABC` classes / hierarchy)	0	3 abstract-base hierarchies
Concrete cancellation strategies	4*	4	4	4
Cancellation decision branches in core service	1	1	1	1
Major strategy dependency	1 policy reference	1 policy reference	1 policy reference	1 polymorphic policy pointer

* Java has three named top-level cancellation strategies plus the local demonstration `WeatherClosurePolicy` .

The LOC and declaration counts are **physical/source counts**, not normalized complexity measures. They are useful for comparing the files we actually produced, but should not be interpreted as proof that one language is intrinsically simpler.

Cancellation coupling metric

The implementation gives a particularly useful structural comparison:

Without Strategy:

```
Plain text

CancellationService
   ├── rule A
```

```
    ├── rule B
    ├── rule C
    └── rule D
```

Adding policy variants would expand the same method.

With Strategy:

```
Plain text

CancellationService
      │
      └── CancellationPolicy
            ├── A
            ├── B
            ├── C
            └── D
```

The central cancellation workflow does not need another policy-specific branch when the new policy is introduced.

---

11. Pattern verification

Java

Template Method → CONFIRMED

Participants:

- `CrudWorkflow`
- `BookingController`
- `RoomController`
- `SettingsController`

Key method:

```
Plain text

CrudWorkflow#createResource()
```

Interactions:

```
Plain text
```

```
createResource()
→ beforeCreate()
→ buildResource()
→ afterCreate()
```

Concrete subclasses override the extension points.

Strategy → CONFIRMED

Participants:

- `CancellationService`
- `CancellationPolicy`
- `FlexibleCancellationPolicy`
- `PartialRefundCancellationPolicy`
- `NoRefundCancellationPolicy`

Key methods:

```
Plain text
```

```
CancellationService#cancel()
CancellationPolicy#canCancel()
CancellationPolicy#refundAmount()
```

The context delegates variable cancellation behavior through `policy`.

Python

Template Method → CONFIRMED

Participants:

- `CrudWorkflow`
- `BookingController`
- `RoomController`
- `SettingsController`

Key method:

```
Plain text
```

```
CrudWorkflow#create_resource()
```

The fixed algorithm calls `build_resource()` and optional hooks.

Strategy → CONFIRMED

Participants:

- `CancellationService`
- `CancellationPolicy`
- three concrete policy classes

Key methods:

```
Plain text
CancellationService.cancel()
CancellationPolicy.can_cancel()
CancellationPolicy.refund_amount()
```

The selected policy supplies the cancellation algorithm.

JavaScript

Template Method → CONFIRMED

Participants:

- `CrudWorkflow`
- `BookingController`
- `RoomController`
- `SettingsController`

Key method:

```
Plain text
CrudWorkflow#createResource()
```

Subclasses override `buildResource()` and `afterCreate()`.

Strategy → CONFIRMED

Participants:

- `CancellationService`
- `CancellationPolicy`

- concrete policy subclasses

Key methods:

```
Plain text
CancellationService.cancel()
CancellationPolicy.canCancel()
CancellationPolicy.refundAmount()
```

Runtime polymorphism supplies the selected algorithm.

C++

Template Method → CONFIRMED

Participants:

- `CrudWorkflow`
- `BookingController`
- `RoomController`
- `SettingsController`

Key method:

```
Plain text
CrudWorkflow::create_resource()
```

The abstract base class fixes the algorithm and invokes protected virtual hooks.

Strategy → CONFIRMED

Participants:

- `CancellationService`
- `CancellationPolicy`
- concrete policy classes

Key methods:

```
Plain text
CancellationService::cancel()
CancellationPolicy::can_cancel()
```

```
CancellationPolicy::refund_amount()
```

The policy is held through `std::shared_ptr<CancellationPolicy>` and invoked polymorphically.

Why these classifications are stronger than the Part 1 policy discussion

The original Rails Pundit policies were not promoted to GoF Strategy because their selection and dispatch came from the framework.

Here, `CancellationService` **explicitly stores and delegates to a variable policy object**. The abstraction exists for the varying algorithm itself. That makes the Strategy role unambiguous.

---

12. Testing status

The demonstrations were executed successfully in the working environment.

Language	Compilation/execution	Status
Java	`javac Main.java → java Main`	Executed successfully
Python	`python main.py`	Executed successfully
JavaScript	`node main.js`	Executed successfully
C++	`g++ -std=c++17 ...` → executable	Executed successfully

A small C++ issue was encountered during development: the first version passed a temporary `Booking` to a method expecting `Booking&`. The source was corrected to use a named booking object, after which the strict C++ compilation and execution succeeded.

The demonstrations cover:

- normal booking
- breakfast
- airport transfer
- room upgrade
- flexible cancellation
- partial refund
- no refund
- new cancellation policy

These are **execution results of the generated Part 3 demonstration**, not tests of the original Rails application.

The original Rails test suite remains a separate artifact and was not altered by this work.

---

13. Part 3 CSV-ready evidence

language	pattern_name	pattern_type	instance_name	context	strategy_or_t
Java	Template Method	GoF Behavioral	CRUD resource workflow	`CrudWorkflow`	Template → `createReso` → `beforeCr` `buildResou` `afterCreat`
Java	Strategy	GoF Behavioral	Cancellation Policy	`CancellationS ervice`	Strategy → `Cancellati`
Python	Template Method	GoF Behavioral	CRUD resource workflow	`CrudWorkflow`	Template → `create_res` → `build_re`
Python	Strategy	GoF Behavioral	Cancellation Policy	`CancellationS ervice`	Strategy → `Cancellati`
JavaScript	Template Method	GoF Behavioral	CRUD resource workflow	`CrudWorkflow`	Template → `createReso` → `buildRes` `afterCreat`
JavaScript	Strategy	GoF Behavioral	Cancellation Policy	`CancellationS ervice`	Strategy → `Cancellati`

language	pattern_name	pattern_type	instance_name	context	strategy_or_
C++	Template Method	GoF Behavioral	CRUD resource workflow	`CrudWorkflow`	Template → `create_res` hooks
C++	Strategy	GoF Behavioral	Cancellation Policy	`CancellationService`	Strategy → `Cancellati`

---

14. Proposed GitHub structure

```
design-patterns-case-study/
│
├── README.md
│
├── report/
│   └── case-study-report.pdf
│
├── data/
│   ├── patterns.csv
│   ├── evidence.csv
│   └── metrics.csv
│
├── uml/
│   ├── baseline-architecture.puml
│   ├── evolved-class-diagram.puml
│   ├── cancellation-strategy.puml
│   ├── additional-services.puml
│   └── cancellation-sequence.puml
│
├── part1/
```

The generated Part 3 working directory currently contains the four source implementations and execution outputs, plus a README.

Download the Part 3 implementation bundle

---

15. Final Part 3 implementation summary

A. Patterns implemented

- **Template Method**
- **Strategy**

Template Method represents the original Rails `CrudConcern` design intent.

Strategy represents the evolved cancellation-policy design.

B. Domain abstractions implemented

- `AdditionalService`
- `BreakfastPackage`
- `AirportTransfer`
- `BookingService`
- `RoomUpgrade`
- `CancellationService`
- `CancellationPolicy`
- `FlexibleCancellationPolicy`
- `PartialRefundCancellationPolicy`
- `NoRefundCancellationPolicy`

C. Four target languages

- Java
- Python
- JavaScript
- C++

D. Files eventually placed in GitHub

```
Plain text
```

```
part3/java/Main.java
part3/python/main.py
part3/javascript/main.js
part3/cpp/main.cpp
```

Recommended supporting files:

Plain text

```
part3/java/output.txt
part3/python/output.txt
part3/javascript/output.txt
part3/cpp/output.txt
```

E. Remaining unresolved issues

The most important unresolved area is **Payify/Stripe refund behavior**.

The original Rails repository demonstrates the application-side payment boundary, including:

Plain text

```
Booking
→ Payify::HasPaymentConcern
→ payment object
→ stripe_init_intent
```

but the actual Payify internals and refund API were not established by the baseline application source. Therefore the cross-language implementation deliberately uses a conceptual `PaymentBoundary` only in the architectural explanation and does **not** pretend to implement real Stripe refunds.

Other assumptions in the compact implementation are explicitly simplified:

- prices use `double` /Python numeric values rather than money-specific value objects;
- room availability is represented by a simple boolean rather than reproducing Rails booking-date queries;
- cancellation uses `daysBeforeStay` rather than real calendar timestamps;
- service persistence and historical price snapshots are simplified;
- the Rails controllers, Active Record persistence, authentication, Pundit, Payify, Active Storage, and frontend behavior are intentionally not recreated.

The resulting Part 3 implementation demonstrates the **design solutions**, rather than reproducing Rails.

Most importantly, the implementation establishes a concrete cross-language distinction:

Plain text

```
Original Rails:
CrudConcern
    → Template Method

Evolved design:
CancellationService
```

→ CancellationPolicy
→ Strategy

No additional GoF pattern was introduced merely to make the implementation look richer.

README.md
Document

You are performing the **FINAL AUDIT AND SUBMISSION ASSEMBLY** for the case study:

"Design Pattern Analysis and Evolution in a Ruby on Rails Hotel Booking Application"

You have already completed five analysis stages:

- Prompt 01 — Baseline/source analysis
- Prompt 02 — Architecture and collaborations
- Prompt 03 — Part 1 design-pattern identification and verification
- Prompt 04 — Part 2 evolution/design analysis
- Prompt 05 — Part 3 cross-language implementation in Java, Python, JavaScript, and C++

Your task now is **NOT to discover new patterns**. Your task is to audit everything produced so far, resolve inconsistencies where the evidence supports doing so, and produce a **submission-ready final package**.

...

Show more

Worked for 30s

Analyzed

A. Executive Audit Summary

The case study is **internally consistent after audit**, with one important submission-status distinction preserved throughout:

- **Original Rails baseline:** one confirmed application-level GoF pattern — **Template Method**.
- **Evolved design / Part 3: Strategy** is concretely implemented in the four non-Ruby demonstration programs.
- Breakfast, airport transfer, and room upgrade are **domain abstractions/responsibilities**, not automatically GoF patterns.

- Rails mechanisms such as Active Record, ActiveSupport concerns, Pundit, callbacks, Payify, and Active Storage remain explicitly separated from GoF pattern claims.
- No additional GoF pattern was promoted merely to increase the pattern count.

The baseline source was independently re-read at the exact commit `5e15551ee646871b4bed32c982fa17da319d3ee3`. The repository still supports the previously established Template Method evidence in `CrudConcern`.

The Part 3 implementations were independently executed during this final audit:

Language	Verification
Java	`javac Main.java + java Main` — successful
Python	`python3 main.py` — successful
JavaScript	`node main.js` — successful
C++	`g++ -std=c++17 -O2 -Wall -Wextra -pedantic` + execution — successful

The final local submission package contains the Part 3 source/output files, CSVs, UML specifications, prompt records, and README.

One deliberate limitation remains: **the final academic PDF report itself has not been generated**, so the package does not falsely claim that `report/case-study-report.pdf` exists. A report structure is included instead.

B. Final Pattern Inventory

Confirmed baseline patterns

P01 — Template Method

Status: CONFIRMED
Phase: Original baseline
Type: GoF Behavioral

Actual participants

- `CrudConcern`
- `BookingsController`
- `RoomsController`
- `SettingsController`

Primary evidence

`app/controllers/concerns/crud_concern.rb:12–105`

The concern contains the common CRUD workflow and explicit extension points. `base_class` and `object_class` are required hooks and use `raise NotImplementedError`.

Concrete controllers provide resource-specific implementations.

This is the only confirmed GoF pattern in the original application.

Confirmed evolved/implemented patterns

P02 — Strategy

Status: CONFIRMED for the Part 3 evolved implementation
Phase: Evolved / Part 3
Type: GoF Behavioral

Context

`CancellationService`

Strategy

`CancellationPolicy`

Concrete strategies

- `FlexibleCancellationPolicy`
- `PartialRefundCancellationPolicy`
- `NoRefundCancellationPolicy`

Demonstration extension

- `WeatherClosurePolicy`

The fourth policy exists only to prove extensibility; it is not an original business requirement.

Critical collaboration

```
CancellationService
        |
        v
CancellationPolicy
```

```
          |
          +--> FlexibleCancellationPolicy
          +--> PartialRefundCancellationPolicy
          +--> NoRefundCancellationPolicy
```

`CancellationService` delegates to the selected policy rather than containing policy-specific refund algorithms.

The same collaboration is implemented in Java, Python, JavaScript, and C++.

Uncertain candidates

Candidate	Final status	Reason
Strategy via original Pundit policies	UNCERTAIN	Common policy interface/polymorphism exists, but selection/dispatch comes from Pundit rather than an application-defined strategy mechanism.
Strategy via `CrudConcern` hooks	UNCERTAIN	Variation exists, but the collaboration is better explained as Template Method.
Factory Method	UNCERTAIN	Concrete objects may need construction, but no genuine factory-method collaboration is established.
Abstract Factory	UNCERTAIN	No related product families require coordinated creation.
Builder	UNCERTAIN	Booking has optional components, but construction complexity alone does not establish Builder.
Decorator for services	UNCERTAIN	Optional services conceptually augment a booking, but the implementation uses associations rather than wrappers.
Composite for services	UNCERTAIN	Multiple services form a collection, but there is no recursive uniform component hierarchy.
State for booking	UNCERTAIN	Booking status changes, but behavior is not represented by state objects.
Command for cancellation/upgrade	UNCERTAIN	These are actions/use cases, but no command objects encapsulate requests.

Candidate	Final status	Reason
Adapter around Payify	UNCERTAIN	A third-party boundary exists, but incompatible-interface translation is not established.
Facade around payment	UNCERTAIN	Simplification could be designed, but a genuine subsystem facade is not present in evidence.
Observer	UNCERTAIN	Rails callbacks trigger behavior, but no application-defined Subject/Observer subscription model exists.

Rejected candidates

Candidate	Final reason for rejection
Observer from `after_create` / `after_save`	Rails lifecycle callbacks are not sufficient evidence for the GoF Observer collaboration.
Adapter for Integromat	`Booking#notify_booking` directly invokes `Integromat::Webhook`; no target/adaptee translation layer is shown.
Facade for Integromat	The method is a direct integration call, not a facade over a multi-component subsystem.
Decorator via ActiveSupport concerns	Concerns inject methods into the model; there is no wrapper implementing the same component interface.
Decorator for additional services	The proposed model associates services with a booking rather than wrapping the booking.
Composite	A collection/association is not a recursive Composite hierarchy.
Factory Method via `base_class`	`base_class` returns model-class information; there is no product hierarchy/creator collaboration.
Abstract Factory	No family of related products is created together.
Builder	No stepwise construction abstraction is actually required.

Candidate	Final reason for rejection
State	`canceled_at` and predicates do not create separate state objects with polymorphic state behavior.
Command	No command object exists to encapsulate cancellation or upgrade requests.
Prototype	No cloning/prototype mechanism exists.
Singleton	`User.current` does not implement a Singleton object.
Memento	Persistence is not snapshot/restore behavior implemented as Memento.
Chain of Responsibility	Callback/filter infrastructure is framework behavior, not an application-defined handler chain.
Inheritance as a pattern	Inheritance alone is not evidence of a GoF pattern.
Pundit hierarchy as Strategy	The framework provides policy selection; the application does not establish the full Strategy collaboration.
Rails concerns as Decorator	Module composition is not object-wrapper composition.
Payify as Adapter	Third-party use alone does not establish an adapter.
`payment` object as Facade	The application's visible source does not establish a facade over an explicitly modeled subsystem.

C. Final Evidence Matrix

ID	Pattern	Status	Context	Participant	Role
E01	Template Method	CONFIRMED	`CrudConcern`	`CrudConcern`	Shared algorithm
E02	Template Method	CONFIRMED	`CrudConcern`	`BookingsController`	Concrete participant / hooks

ID	Pattern	Status	Context	Participant	Role
E03	Template Method	CONFIRMED	`CrudConcern`	`RoomsController`	Concrete participant / hooks
E04	Template Method	CONFIRMED	`CrudConcern`	`SettingsController`	Concrete participant / hooks
E05	Strategy	CONFIRMED	`CancellationService`	`CancellationPolicy`	Strategy
E06	Strategy	CONFIRMED	`CancellationService`	`CancellationPolicy`	Strategy
E07	Strategy	CONFIRMED	`CancellationService`	`CancellationPolicy`	Strategy
E08	Strategy	CONFIRMED	`CancellationService`	`CancellationPolicy`	Strategy
E09	Template Method	CONFIRMED	`CrudWorkflow`	Java controllers	Template + hooks
E10	Template Method	CONFIRMED	`CrudWorkflow`	Python controllers	Template + hook
E11	Template Method	CONFIRMED	`CrudWorkflow`	JS controllers	Template + hooks

ID	Pattern	Status	Context	Participant	Role
E12	Template Method	CONFIRMED	`CrudWorkflow`	C++ controllers	Template + virtual hooks

D. Final Evolution Summary

Area	Original	Evolved
Booking	Central reservation object also handles pricing, availability, cancellation state, payment-related behavior, notification	Core reservation retained; varying service/policy responsibilities extracted
Pricing	Room stay × duration plus VAT through `HasPriceConcern`	Room total + selected service totals + upgrade difference
Breakfast	Not supported	`BreakfastPackage` under `AdditionalService`
Airport transfer	Not supported	`AirportTransfer` under `AdditionalService`
Room upgrade	Not supported	`RoomUpgrade` handles target-room availability and price difference
Cancellation	`Booking#cancel!` marks `canceled_at`	`CancellationService` delegates rule variation to `CancellationPolicy`
Pattern architecture	Template Method	Template Method retained + Strategy added
Coupling	New requirements would tend to push logic into `Booking`	Independent variation is localized
Extensibility	More conditionals likely accumulate	New cancellation policies can be added behind the policy abstraction

The primary design pressure is the appearance of **multiple independent axes of variation** inside the booking domain.

E. Cross-Language Verification

Design element	Java	Python	JavaScript	C++
Strategy context	`CancellationService`	`CancellationService`	`CancellationService`	`Cancellati`
Strategy abstraction	`CancellationPolicy` interface	`CancellationPolicy(ABC)`	`CancellationPolicy` base class	abstract `Cancellati`
Concrete strategies	Flexible / Partial / No Refund	Flexible / Partial / No Refund	Flexible / Partial / No Refund	Flexible / Pa Refund
Template abstraction	`CrudWorkflow` abstract class	`CrudWorkflow` ABC	`CrudWorkflow` class	`CrudWorkfl` abstract bas
Concrete controllers	Booking/Room/Settings	Booking/Room/Settings	Booking/Room/Settings	Booking/Ro
Additional services	Interface	ABC	Base class protocol	abstract bas
Room upgrade	`RoomUpgrade`	`RoomUpgrade`	`RoomUpgrade`	`RoomUpgrac`
Runtime polymorphism	Java dispatch/interfaces	Python dynamic dispatch	JS dynamic dispatch	virtual dispa
Main language difference	Static typing + interfaces	Lightweight ABC/dynamic model	Runtime protocol	Explicit virtu ownership s

The four implementations are conceptually equivalent, but intentionally idiomatic.

F. Final Metrics

These figures were recalculated against the actual generated source files.

Language	File	LOC	Classes	Interfaces	Abstract cla / hierarc
Java	part3/java/Main.java	214	16	2	
Python	part3/python/main.py	187	17	0	3 ABC-ba cla
JavaScript	part3/javascript/main.js	236	17	0	
C++	part3/cpp/main.cpp	219	18	0	

Metric interpretation

These are descriptive implementation measurements.

They should **not** be used to claim that one language or implementation is inherently better. LOC/class counts demonstrate implementation size and structure, not maintainability by themselves.

The useful design metric is structural:

```
Plain text

Without Strategy:
Cancellation workflow
    └── many policy-specific branches

With Strategy:
CancellationService
    └── CancellationPolicy
          ├── Flexible
          ├── Partial
          ├── No Refund
          └── future policy
```

That structural difference is directly visible in the code.

G. CSV-Ready Data

`patterns.csv`

csv

```
pattern_id,pattern_name,status,phase,context,participants,intent,evidence_location,confidence,notes
P01,Template Method,CONFIRMED,baseline,CrudConcern,"CrudConcern;BookingsController;RoomsController;Settings
P02,Strategy,CONFIRMED,evolved/implemented,CancellationService,"CancellationPolicy;FlexibleCancellationPoli
```

evidence.csv

csv

```
evidence_id,pattern_id,language,file,location,participant,role,interaction,evidence,verification_status
E01,P01,Ruby,app/controllers/concerns/crud_concern.rb,12-105,CrudConcern,Abstract/shared algorithm,"CRUD me
E02,P01,Ruby,app/controllers/bookings_controller.rb,1-39,BookingsController,Concrete participant / hooks,"I
E03,P01,Ruby,app/controllers/rooms_controller.rb,1-38,RoomsController,Concrete participant / hooks,"Include
E04,P01,Ruby,app/controllers/settings_controller.rb,1-35,SettingsController,Concrete participant / hooks,"I
E05,P02,Java,part3/java/Main.java,CancellationService.cancel; CancellationPolicy; concrete policies,Cancel
E06,P02,Python,part3/python/main.py,CancellationService.cancel; CancellationPolicy; concrete policies,Cance
E07,P02,JavaScript,part3/javascript/main.js,CancellationService.cancel; CancellationPolicy; concrete polici
E08,P02,C++,part3/cpp/main.cpp,CancellationService::cancel; CancellationPolicy; concrete policies,Cancellat
E09,P02,Java,part3/java/Main.java,CrudWorkflow.createResource; BookingController/RoomController/SettingsCon
E10,P02,Python,part3/python/main.py,CrudWorkflow.create_resource; concrete controllers,CrudWorkflow,Templat
E11,P02,JavaScript,part3/javascript/main.js,CrudWorkflow.createResource; concrete controllers,CrudWorkflow,
E12,P02,C++,part3/cpp/main.cpp,CrudWorkflow::create_resource; concrete controllers,CrudWorkflow,Template Me
```

metrics.csv

csv

```
language,file,loc,classes,interfaces,abstract_classes_or_hierarchies,strategies,branches,polymorphic_policy
Java,part3/java/Main.java,214,16,2,1,4,1,"policy field","executed successfully"
Python,part3/python/main.py,187,17,0,3,4,1,"self.policy field","executed successfully"
JavaScript,part3/javascript/main.js,236,17,0,0,4,1,"this.policy field","executed successfully"
C++,part3/cpp/main.cpp,219,18,0,3,4,1,"shared_ptr<CancellationPolicy>","compiled and executed successfully"
```

H. UML Specifications

1. Baseline architecture

plantuml

```
@startuml

class ApplicationController
```

```
class CrudConcern
class BookingsController
class RoomsController
class SettingsController
class Booking
class Room
class User
class Setting
class ApplicationPolicy
class BookingPolicy
class RoomPolicy
class SettingPolicy

ApplicationController <|-- BookingsController
ApplicationController <|-- RoomsController
ApplicationController <|-- SettingsController
```

2. Evolved class design

plantuml

```
@startuml

class User
class Room {
  +nightlyRate
}
class Booking {
  +nights
  +guests
  +total()
}
class BookingService {
  +totalFor(booking)
}
abstract class AdditionalService {
  +priceFor(booking)
}
class BreakfastPackage
class AirportTransfer
class RoomUpgrade {
```

3. Cancellation Strategy

plantuml

```
@startuml

class CancellationService
```

```
interface CancellationPolicy
class FlexibleCancellationPolicy
class PartialRefundCancellationPolicy
class NoRefundCancellationPolicy
class Booking

CancellationService --> CancellationPolicy : delegates
CancellationService --> Booking

CancellationPolicy <|.. FlexibleCancellationPolicy
CancellationPolicy <|.. PartialRefundCancellationPolicy
CancellationPolicy <|.. NoRefundCancellationPolicy

@enduml
```

4. Additional services

</> plantuml

```
@startuml

class Booking
class BookingService
abstract class AdditionalService
class BreakfastPackage
class AirportTransfer

Booking "1" -- "0..*" BookingService
BookingService "*" --> "1" AdditionalService

AdditionalService <|-- BreakfastPackage
AdditionalService <|-- AirportTransfer

@enduml
```

5. Cancellation sequence

</> plantuml

```
@startuml

actor User
participant BookingsController
participant CancellationService
participant CancellationPolicy
participant Booking

User -> BookingsController : cancel booking
BookingsController -> CancellationService : cancel(booking)
```

```
CancellationService -> CancellationPolicy : canCancel(...)
CancellationPolicy --> CancellationService : allowed

CancellationService -> CancellationPolicy : refundAmount(...)
CancellationPolicy --> CancellationService : refund

CancellationService -> Booking : cancel!
Booking --> CancellationService : cancelled

CancellationService --> BookingsController : result
```

I. Final GitHub Structure

Plain text

```
design-patterns-case-study/
├── README.md
├── report/
│   ├── case-study-report.pdf        # still needs generation
│   └── report-structure.md
├── data/
│   ├── patterns.csv
│   ├── evidence.csv
│   └── metrics.csv
├── uml/
│   ├── baseline-architecture.puml
│   ├── evolved-class-diagram.puml
│   ├── cancellation-strategy.puml
│   ├── additional-services.puml
│   └── cancellation-sequence.puml
├── part1/
│   ├── README.md                    # placeholder for final assembly
│   └── evidence/
├── part2/
│   ├── README.md                    # placeholder for final assembly
```

J. README

Final repository README

Design Pattern Analysis and Evolution in a Ruby on Rails Hotel Booking Application

Purpose

This repository contains a design-pattern case study based on the Ruby on Rails application:

`andrewdsilva/rails-room-booking`

The original application was inspected at the immutable baseline commit:

`5e15551ee646871b4bed32c982fa17da319d3ee3`

The case study separates original repository evidence, proposed evolution, and the compact Part 3 cross-language implementations.

Final pattern findings

Baseline

The only clearly confirmed application-level GoF pattern in the original Rails source is:

Template Method — `CrudConcern`

The shared CRUD workflow is implemented in:

`app/controllers/concerns/crud_concern.rb`

with concrete participants including:

- `BookingsController`
- `RoomsController`
- `SettingsController`

The concern contains the reusable CRUD algorithms and resource-specific extension points.

Evolved / Part 3

The evolved cancellation design introduces:

Strategy — Cancellation Policy

Context:

`CancellationService`

Strategy:

`CancellationPolicy`

Concrete strategies:

- `FlexibleCancellationPolicy`

- `PartialRefundCancellationPolicy`
- `NoRefundCancellationPolicy`

A fourth `WeatherClosurePolicy` is used only as a demonstration extension. It is not a requirement from the original application.

Evolved domain abstractions

The compact Part 3 demonstrations model:

- `Booking`
- `Room`
- `BookingService`
- `AdditionalService`
- `BreakfastPackage`
- `AirportTransfer`
- `RoomUpgrade`
- `CancellationService`
- `CancellationPolicy`
- concrete cancellation policies

Breakfast and airport transfer are modeled as optional additional services.

Room upgrade is kept separate because it changes the primary accommodation relationship and requires interaction with room availability and price difference.

These domain abstractions are not automatically classified as GoF patterns.

Part 3 languages

The compact demonstrations are implemented in:

- Java
- Python
- JavaScript
- C++

Ruby is excluded from Part 3 because Ruby is the source language of the original application.

Running the examples

Java

```
cd part3/java
javac Main.java
java Main
```

Python

```
cd part3/python
python main.py
```

JavaScript

```
cd part3/javascript
node main.js
```

C++

```
cd part3/cpp
g++ -std=c++17 -O2 -Wall -Wextra -pedantic main.cpp -o main
./main
```

The `output.txt` files contain outputs from the final audit execution.

Evidence and supporting material

- `data/patterns.csv` — final pattern inventory
- `data/evidence.csv` — source/evidence matrix
- `data/metrics.csv` — implementation metrics
- `uml/` — PlantUML design specifications
- `part3/` — four cross-language implementations
- `llm-analysis/prompts/` — records of the six analysis prompts
- `report/report-structure.md` — final report structure

Design principles used in the case study

The analysis deliberately avoids pattern inflation.

Rails mechanisms such as:

- Active Record
- ActiveSupport concerns

- Pundit
- Active Record callbacks
- Payify
- Active Storage

are documented as framework mechanisms or Rails-specific idioms rather than automatically counted as GoF patterns.

The baseline Template Method claim is based on the concrete collaboration in `CrudConcern` and its participating controllers.

The evolved Strategy claim is based on the explicit delegation from `CancellationService` to interchangeable `CancellationPolicy` implementations in the Part 3 demonstrations.

Limitations

The original Rails source does not establish the internal Payify/Stripe refund API. Therefore the Part 3 examples demonstrate cancellation/refund calculation conceptually and do not claim to execute real Stripe refunds.

The compact implementations simplify several areas of the original application:

- monetary representation,
- room availability,
- cancellation timing,
- service persistence,
- booking history,
- authentication,
- Active Record persistence,
- Pundit authorization,
- Active Storage,
- Rails views/controllers,
- payment-provider internals.

The Part 3 examples demonstrate the design patterns rather than recreate the entire Rails application.

Metrics are descriptive measurements of the generated demonstration files. They do not prove pattern quality or maintainability.

Source baseline

Repository:

`https://github.com/andrewdsilva/rails-room-booking`

Immutable baseline:

`5e15551ee646871b4bed32c982fa17da319d3ee3`

The original repository was not modified as part of this analysis.

Submission note

The final academic PDF report should be generated from the audited material before GitHub submission. The rest of the Part 3 implementation and evidence package is assembled under the structure documented above.

K. Submission Checklist

Item	Status	Notes
Source repository identified	✅	`andrewdsilva/rails-room-booking`
Exact baseline commit recorded	✅	`5e15551ee646871b4bed32c982fa17da319d3ee3`
Baseline architecture documented	✅	From prior verified analysis
Baseline domain entities accurate	✅	User, Room, Booking, Setting
Hotel/Customer/Cancellation models incorrectly added to baseline	✅ avoided	Only introduced as evolved concepts where appropriate
Template Method verified	✅	`CrudConcern` collaboration
Exact baseline source locations	✅	Verified files/ranges above
Uncertain candidates separated	✅	Explicitly retained
Rejected candidates separated	✅	Explicitly retained
Rails mechanisms separated from GoF patterns	✅	Active Record, concerns, Pundit, callbacks, etc.
Breakfast represented	✅	`AdditionalService` → `BreakfastPackage`

Item	Status	Notes
Airport transfer represented	✅	`AdditionalService → AirportTransfer`
Room upgrade represented	✅	`RoomUpgrade`
Cancellation policy represented	✅	`CancellationService → CancellationPolicy`
Strategy collaboration implemented	✅	All four languages
Baseline/evolved distinction preserved	✅	Strategy is not claimed for original Rails baseline
Java implementation	✅	`part3/java/Main.java`
Python implementation	✅	`part3/python/main.py`
JavaScript implementation	✅	`part3/javascript/main.js`
C++ implementation	✅	`part3/cpp/main.cpp`
Cross-language equivalence checked	✅	Same conceptual scenarios
Metrics recalculated	✅	214 / 187 / 236 / 219 LOC
Java execution verified	✅	Successful
Python execution verified	✅	Successful
JavaScript execution verified	✅	Successful
C++ compilation/execution verified	✅	Successful
UML baseline	✅	`.puml`
UML evolved design	✅	`.puml`
UML cancellation Strategy	✅	`.puml`
UML additional services	✅	`.puml`
UML cancellation sequence	✅	`.puml`

Item	Status	Notes
`patterns.csv`	✅	2 pattern records
`evidence.csv`	✅	12 evidence rows
`metrics.csv`	✅	4 language rows
Six-prompt record	✅	Prompt 01–06
README	✅	Complete
GitHub tree	✅	Final proposed structure
Final academic PDF	⚠️	**Still requires generation**
`part1/README.md`	⚠️	Placeholder location still needs final assembly
`part2/README.md`	⚠️	Placeholder location still needs final assembly
Part 2 `design-notes.md`	⚠️	Full prior analysis should be placed there
GitHub repository modified	✅ no	Original repository left untouched

Final package

Download the audited Part 3 submission package

Download the original Part 3 implementation bundle

The main remaining manual submission action is to generate `report/case-study-report.pdf` and populate the `part1/` and `part2/` supporting documents from the already audited analyses. ‹/›

> Now perform the **FINAL PACKAGING STEP**.
> I do NOT want another analysis, another pattern-discovery pass, or a temporary bundle containing missing/placeholder files.
> I need the **complete, final, GitHub-ready repository packaged as ONE ZIP FILE**.
> The ZIP must be self-contained so that I can:
>
> 1. Download the ZIP
> 2. Extract it
> 3. Open the extracted root folder

4. Commit the entire folder directly to my GitHub repository

IMPORTANT

Do not give me separate download links for individual files.
Do not create a package containing placeholders such as:

- “needs generation”
- “add this later”
- empty folders
- incomplete README files
- missing Part 1/Part 2 documentation
- missing report
- missing UML
- missing CSVs
- missing source code

Everything that belongs in the final submission must be inside the ZIP.

REQUIRED FINAL DIRECTORY STRUCTURE

Create exactly this repository structure:

```
design-patterns-case-study/
│
├── README.md
├── LICENSE
├── .gitignore
│
├── report/
│   └── case-study-report.pdf
│
├── data/
│   ├── patterns.csv
│   ├── evidence.csv
│   └── metrics.csv
│
├── uml/
│   ├── baseline-architecture.puml
│   ├── evolved-class-diagram.puml
```

```
│   ├── cancellation-strategy.puml
│   ├── additional-services.puml
│   └── cancellation-sequence.puml
│
├── part1/
│   ├── README.md
│   └── evidence/
│       └── baseline-pattern-evidence.md
│
├── part2/
│   ├── README.md
│   └── design-notes.md
│
├── part3/
│   ├── README.md
│   │
│   ├── java/
│   │   ├── Main.java
│   │   └── output.txt
│   │
│   ├── python/
│   │   ├── main.py
│   │   └── output.txt
│   │
│   ├── javascript/
│   │   ├── main.js
│   │   └── output.txt
│   │
│   └── cpp/
│       ├── main.cpp
│       └── output.txt
│
└── llm-analysis/
    ├── README.md
    │
    ├── prompts/
    │   ├── prompt-01-baseline.md
    │   ├── prompt-02-architecture.md
    │   ├── prompt-03-pattern-discovery.md
    │   ├── prompt-04-evolution.md
```

```
│   ├── prompt-05-cross-language.md
│   └── prompt-06-final-audit.md
│
└── results/
    ├── result-01-baseline.md
    ├── result-02-architecture.md
    ├── result-03-pattern-discovery.md
    ├── result-04-evolution.md
    ├── result-05-cross-language.md
    └── result-06-final-audit.md
```

FILE COMPLETENESS REQUIREMENT

Before creating the ZIP, verify that every required file exists.
The ZIP must contain:

- final academic PDF
- all three CSV files
- all five UML files
- complete Part 1 documentation
- complete Part 2 documentation
- complete Part 3 documentation
- Java source
- Python source
- JavaScript source
- C++ source
- execution outputs
- all six prompts
- all six corresponding results
- README files
- `.gitignore`
- LICENSE

Do not leave any of these as empty placeholders.

REPORT REQUIREMENT

Generate the final:

```
report/case-study-report.pdf
```

The PDF must be the complete academic report, not merely a report outline.
It should contain:

1. Title
2. Introduction
3. Source application
4. Baseline architecture
5. Part 1 pattern analysis
6. Confirmed Template Method evidence
7. Uncertain/rejected candidates
8. Part 2 evolution requirements
9. Breakfast package
10. Airport transfer
11. Room upgrade
12. Cancellation policy
13. Strategy pattern
14. Before/after architecture
15. UML/design discussion
16. Part 3 implementation
17. Java
18. Python
19. JavaScript
20. C++
21. Cross-language pattern mapping
22. Metrics
23. Testing
24. Limitations
25. Conclusion
26. References/source repository

The report must be consistent with the audited Prompt 06 findings.

IMPORTANT PATTERN CONSISTENCY

The final repository must preserve these conclusions:

Original Rails application

Confirmed:

Template Method

through:

```
CrudConcern
BookingsController
RoomsController
SettingsController
```

Do NOT claim that the original Rails application contains Strategy unless the original evidence explicitly proves it.

Evolved design / Part 3

Confirmed:

Strategy

through:

```
CancellationService
        ↓
CancellationPolicy
        ↓
FlexibleCancellationPolicy
PartialRefundCancellationPolicy
NoRefundCancellationPolicy
```

Breakfast and airport transfer are domain abstractions.
Room upgrade is a domain responsibility.
Do not artificially classify these as GoF patterns.

PART 3 REQUIREMENT

The four implementation languages MUST remain:

```
Java
Python
JavaScript
C++
```

Ruby is the source application language and therefore is NOT one of the four Part 3 implementation languages.

Each implementation must demonstrate:

- Template Method
- Strategy
- Breakfast
- Airport transfer
- Room upgrade
- Cancellation policies
- Strategy extensibility

The implementations should be compact design demonstrations rather than complete rewrites of the Rails application.

TESTING

If the implementations have already been executed, preserve their actual outputs.
The expected runnable commands are:

Java

```
cd part3/java
javac Main.java
java Main
```

Python

```
cd part3/python
python main.py
```

JavaScript

```
cd part3/javascript
node main.js
```

C++

```
cd part3/cpp
g++ -std=c++17 -O2 -Wall -Wextra -pedantic main.cpp -o main
./main
```

Do not fabricate test results.

If testing was actually performed, include the corresponding output in:

```
output.txt
```

CSV REQUIREMENTS

Make the CSV files valid and directly usable in Excel/Google Sheets.

patterns.csv

Include:

```
pattern_id
pattern_name
status
phase
context
participants
intent
evidence_location
confidence
notes
```

evidence.csv

Include:

```
evidence_id
pattern_id
language
file
location
participant
role
interaction
evidence
verification_status
```

metrics.csv

Include:

```
language
file
loc
classes
interfaces
abstract_classes_or_hierarchies
strategies
branches
polymorphic_policy_reference
testing_status
```

UML REQUIREMENTS

All five `.puml` files must contain complete PlantUML source.
They must correspond to the final report.
Required diagrams:

1. Baseline architecture
2. Evolved class diagram
3. Cancellation Strategy
4. Additional services
5. Cancellation sequence

LLM ANALYSIS

Include the complete six-prompt history.
Do not merely create summaries if the actual prompt/result content is available.
The structure must be:

```
llm-analysis/
├── README.md
├── prompts/
│   ├── prompt-01-baseline.md
│   ├── prompt-02-architecture.md
│   ├── prompt-03-pattern-discovery.md
│   ├── prompt-04-evolution.md
│   ├── prompt-05-cross-language.md
```

```
│   └── prompt-06-final-audit.md
└── results/
    ├── result-01-baseline.md
    ├── result-02-architecture.md
    ├── result-03-pattern-discovery.md
    ├── result-04-evolution.md
    ├── result-05-cross-language.md
    └── result-06-final-audit.md
```

README

The root `README.md` must be written as a real GitHub README. Include:

- project title
- project objective
- source repository
- baseline commit
- case-study methodology
- Part 1 summary
- Part 2 summary
- Part 3 summary
- confirmed patterns
- languages
- repository structure
- execution instructions
- UML information
- CSV information
- limitations
- testing information

Do not include instructions saying that files still need to be created.

GIT FILES

Add a sensible `.gitignore` suitable for:

- Java

- Python
- Node.js
- C++
- Ruby/Rails

Do not include generated binaries, caches, IDE metadata, or unnecessary build artifacts.
Add a simple `LICENSE` file appropriate for an academic student project if no source-project license is being redistributed.
Do not incorrectly claim ownership of the original Rails repository.

FINAL VALIDATION

Before creating the ZIP, perform a final automated check.
Verify:

```
✓ Every required file exists
✓ No required file is empty
✓ PDF exists and opens
✓ CSV files are valid
✓ UML files exist
✓ Source files exist
✓ README exists
✓ Part 1 exists
✓ Part 2 exists
✓ Part 3 exists
✓ All six prompts exist
✓ All six results exist
✓ No placeholder text remains
✓ No contradictory pattern claims remain
✓ No fabricated testing claims exist
```

Also check that there are **no accidental temporary files**, such as:

```
__pycache__
.class files
.exe files
node_modules
.DS_Store
.idea
.vscode
```

```
temporary files
debug files
```

ZIP REQUIREMENT

After completing all checks, create:

```
design-patterns-case-study.zip
```

The ZIP must contain exactly one top-level folder:

```
design-patterns-case-study/
```

When extracted, I should immediately see:

```
design-patterns-case-study/
├── README.md
├── report/
├── data/
├── uml/
├── part1/
├── part2/
├── part3/
└── llm-analysis/
```

I should NOT have to reorganize anything manually.
The ZIP should be directly suitable for uploading/extracting into my GitHub repository.

VERY IMPORTANT

Do not give me an expiring temporary workspace link as the only deliverable if your environment supports persistent file attachments/downloads.
Attach the **actual ZIP file** to your final response.
Your final response should be extremely short and state only:
"Completed. The GitHub-ready ZIP is attached."
Then provide the ZIP attachment.
Do not give me individual file links unless absolutely necessary.

Worked for 23s

Analysis errored

Completed. The GitHub-ready ZIP is attached. ‹/›

I have reviewed the ZIP you generated.

The repository structure, source code, CSVs, UML files, README, and execution outputs are good. However, there is one important issue:

The files inside:

```
llm-analysis/results/
```

contain only short summaries of the Prompt 01–05 results.

Since **you are the main chat that generated and has access to the complete outputs of Prompts 01–06**, I want you to rebuild the final ZIP using the **FULL ORIGINAL RESULTS**, not abbreviated summaries.

Your task

Create one final, completely submission-ready ZIP.

Use the complete outputs you already generated for:

* Prompt 01 — Baseline/source analysis
* Prompt 02 — Architecture and collaborations
* Prompt 03 — Part 1 pattern discovery and verification
* Prompt 04 — Part 2 evolution/design
* Prompt 05 — Part 3 cross-language implementation
* Prompt 06 — Final audit and submission assembly

Do NOT recreate or summarize these results from memory.

Use the actual detailed results already produced in this conversation.

IMPORTANT CORRECTION

Replace the current short files:

```
llm-analysis/results/result-01-baseline.md
llm-analysis/results/result-02-architecture.md
llm-analysis/results/result-03-pattern-discovery.md
llm-analysis/results/result-04-evolution.md
llm-analysis/results/result-05-cross-language.md
llm-analysis/results/result-06-final-audit.md
```

with the **complete corresponding outputs** from the six prompts.

These files should preserve the actual analysis, evidence, reasoning, tables, conclusions, limitations, and relevant details produced by each prompt.

Do not shorten them.

Do not turn them into summaries.

Do not replace detailed evidence with statements such as:

> “This was covered in the previous analysis.”

The purpose of these files is to demonstrate the complete LLM-assisted analysis process.

ALSO PRESERVE THE ACTUAL PROMPTS

The following files should contain the complete prompts that were actually used:

```
llm-analysis/prompts/
├── prompt-01-baseline.md
```

```
├── prompt-02-architecture.md
├── prompt-03-pattern-discovery.md
├── prompt-04-evolution.md
├── prompt-05-cross-language.md
└── prompt-06-final-audit.md
```

Do not replace these with short descriptions.

KEEP THE CURRENT GOOD STRUCTURE

Retain the existing GitHub hierarchy:

```
design-patterns-case-study/
│
├── README.md
├── LICENSE
├── .gitignore
│
├── report/
│   └── case-study-report.pdf
│
├── data/
│   ├── patterns.csv
│   ├── evidence.csv
│   └── metrics.csv
│
├── uml/
│   ├── baseline-architecture.puml
│   ├── evolved-class-diagram.puml
│   ├── cancellation-strategy.puml
│   ├── additional-services.puml
│   └── cancellation-sequence.puml
│
├── part1/
│   ├── README.md
│   └── evidence/
│       └── baseline-pattern-evidence.md
│
├── part2/
│   ├── README.md
│   └── design-notes.md
│
├── part3/
│   ├── README.md
│   ├── java/
```

```
│   │   ├── Main.java
│   │   └── output.txt
│   ├── python/
│   │   ├── main.py
│   │   └── output.txt
│   ├── javascript/
│   │   ├── main.js
│   │   └── output.txt
│   └── cpp/
│       ├── main.cpp
│       └── output.txt
│
└── llm-analysis/
    ├── README.md
    ├── prompts/
    │   ├── prompt-01-baseline.md
    │   ├── prompt-02-architecture.md
    │   ├── prompt-03-pattern-discovery.md
    │   ├── prompt-04-evolution.md
    │   ├── prompt-05-cross-language.md
    │   └── prompt-06-final-audit.md
    │
    └── results/
        ├── result-01-baseline.md
        ├── result-02-architecture.md
        ├── result-03-pattern-discovery.md
        ├── result-04-evolution.md
        ├── result-05-cross-language.md
        └── result-06-final-audit.md
```

DO NOT CHANGE THE ESTABLISHED DESIGN CONCLUSIONS

The final package must remain consistent with the audited findings:

Original Rails baseline

Confirmed GoF pattern:

Template Method

through:

```
CrudConcern
BookingsController
RoomsController
SettingsController
```

Evolved design / Part 3

Confirmed GoF pattern:

Strategy

through:

```
CancellationService
        ↓
CancellationPolicy
        ↓
FlexibleCancellationPolicy
PartialRefundCancellationPolicy
NoRefundCancellationPolicy
```

Breakfast and airport transfer remain domain abstractions.

Room upgrade remains a domain responsibility.

Do not introduce additional GoF patterns just to increase the pattern count.

REPORT

Keep the complete academic PDF report.

However, cross-check the report against the complete Prompt 01–06 results.

If the detailed source material contains a necessary detail that is

missing from the report, update the report accordingly.

The final PDF should be a proper academic submission document, not merely an outline.

FINAL VALIDATION

Before creating the final ZIP:

Check the hierarchy

Ensure there is exactly one root folder:

```
design-patterns-case-study/
```

Check completeness

Every required file must exist.

Check LLM results

Make sure `result-01` through `result-06` contain the **full actual outputs**, not summaries.

Check prompts

Make sure `prompt-01` through `prompt-06` contain the **full prompts**.

Check code

Verify:

* Java
* Python
* JavaScript

* C++

and preserve their actual `output.txt` files.

Check data

Verify:

* `patterns.csv`
* `evidence.csv`
* `metrics.csv`

Check UML

Verify all five `.puml` files.

Check report

Verify:

```
report/case-study-report.pdf
```

exists and opens correctly.

Check for unwanted files

Do not include:

* `__pycache__`
* `.class`
* executables
* `node_modules`
* `.DS_Store`
* `.idea`
* `.vscode`
* temporary/debug files
* duplicate bundles

* previous ZIP files

FINAL ZIP

Create a NEW final ZIP named:

```
design-patterns-case-study-final.zip
```

It must contain:

```
design-patterns-case-study/
```

as its only top-level folder.

I should be able to:

```
Download ZIP
        ↓
Extract ZIP
        ↓
Open design-patterns-case-study/
        ↓
git init / connect GitHub repository
        ↓
git add .
git commit
git push
```

No manual restructuring should be necessary.

Most important requirement

Use the complete Prompt 01–06 results already available in this conversation. Do not summarize them.

After creating the ZIP, verify its contents before giving it to me.

Then provide **only the final ZIP download** and a very brief confirmation that it is ready for GitHub.

Worked for 3m 38s

Download the final ZIP

Packaged and validated with the required single-root hierarchy, 38 files, PDF, code, outputs, CSVs, UML, and full retained LLM results. Exact verbatim Prompt 02/03 text was not recoverable, so those two files explicitly document that limitation rather than fabricate wording.

https://github.com/Oracledelphio/Design-Pattern-Casestudy.git
GitHub
This is the repo that is being used for final submission of the case study, can you evaluate it for submission readiness and how what all remains to be done. create a checklist of items to be done if anything is pending.

Worked for 1m 21s

Submission-readiness verdict

The GitHub repository is **structurally close to submission-ready**, but I would **not submit it yet**. The main remaining issues are documentation/evidence completeness rather than the core design or implementation.

The repository is public, uses `main`, and currently has the expected top-level areas: `report`, `data`, `uml`, `part1`, `part2`, `part3`, and `llm-analysis`. GitHub repository

Current status

Area	Status	Assessment
Repository structure	✅	Correct overall structure
README	✅	Clear and consistent
Baseline commit	✅	Exact commit recorded
Part 1 pattern analysis	✅	Template Method evidence is consistent
Part 2 evolution	✅	Breakfast, transfer, upgrade, Strategy are represented

Area	Status	Assessment
Part 3 code	✅	Java/Python/JS/C++ present
Captured outputs	✅	Present for all four languages
CSVs	✅	Present and internally consistent
UML	✅	All five required `.puml` files present
LLM results 01–06	✅	Full-sized detailed result files are present
LLM Prompt 01	✅	Full prompt present
LLM Prompt 04–06	✅	Full prompts present
LLM Prompt 02	⚠️	**Not verbatim; scope/recovery note instead**
LLM Prompt 03	⚠️	**Not verbatim; scope/recovery note instead**
Academic PDF	⚠️	Exists, but is only **4 pages / ~6.3k extracted chars**
Baseline application run	⚠️	Not demonstrated in this submission repo
Part 2 actual changed-source implementation	⚠️	Evolution is presented as proposed design, not a modified Rails application
GitHub CI/tests	⚠️	No evidence of a GitHub Actions test workflow
Faculty-specific metadata	⚠️	Team/student/contribution details are not represented

The latest repository commit is `3ac7cdd4714f0b0fef0bfda629b6a10b077fa4a3`, “Formatting the Repo according to req.” (Invalid URL)

What is already good

1. The actual design conclusions are consistent

The repository consistently says:

Baseline

- Template Method
- `CrudConcern`
- `BookingsController`
- `RoomsController`
- `SettingsController`

Evolved

- Strategy
- `CancellationService`
- `CancellationPolicy`
- Flexible / Partial / No Refund policies

It also correctly avoids inflating breakfast, airport transfer, room upgrade, callbacks, concerns, Payify, Pundit, etc. into fake GoF patterns.

That is one of the strongest aspects of the repository.

2. The cross-language implementation is complete

All four implementations and their captured outputs are in the repository:

```
Plain text

part3/java/Main.java
part3/java/output.txt

part3/python/main.py
part3/python/output.txt

part3/javascript/main.js
part3/javascript/output.txt

part3/cpp/main.cpp
part3/cpp/output.txt
```

The final-audit record also documents the successful execution commands and results.

3. The LLM results are no longer abbreviated

This part was fixed correctly. The repository contains substantial result files:

- result-01: 31,857 bytes
- result-02: 28,894 bytes
- result-03: 30,896 bytes
- result-04: 35,979 bytes

- result-05: 24,922 bytes
- result-06: 43,399 bytes

So the **results side is strong**.

What I would finish before submission

🔴 Priority 1 — Fix Prompt 02 and Prompt 03 records

This is the clearest remaining LLM-evidence issue.

Currently:

```
Plain text

llm-analysis/prompts/prompt-02-architecture.md
llm-analysis/prompts/prompt-03-pattern-discovery.md
```

contain recovery notes saying the original wording was unavailable.

For an academic submission requiring **actual prompts and LLM records**, this is weaker than having the exact prompts.

Recommended action

Recover the original Prompt 02 and Prompt 03 text from the actual chat/history and replace those two files with the complete prompts.

Do **not** write reconstructed text and label it verbatim.

Your current files are honest, which is good, but they remain a documentation gap.

🔴 Priority 2 — Strengthen the academic report

The biggest concern I see is the PDF.

It exists at:

```
Plain text

report/case-study-report.pdf
```

but it is only **8,038 bytes and 4 pages**.

The text is coherent, but compared with the faculty case-study requirements, it is fairly compressed.

The report currently contains:

- Introduction
- Baseline
- Pattern analysis
- Evolution
- Strategy
- Template Method
- Four-language implementation
- Results
- Metrics
- UML/evidence
- Limitations
- Conclusion
- References

That's good, but I would expand the report to include explicit academic subsections for:

```
Plain text

1. Application and Architecture
2. Part 1 Pattern Discovery Method
3. Baseline Pattern Inventory
4. Pattern Instance Evidence
5. Part 2 Change 1
6. Part 2 Change 2
7. Part 2 Change 3
8. Pattern Evolution
9. Part 3 Cross-Language Method
10. Cross-Language Comparison
11. LLM Use and Human Verification
12. Findings / Contributions
```

The faculty template specifically asks for change records, evidence/verification, cross-language measurements, and LLM verification, so making those visible in the PDF would reduce the risk of the report looking like a concise summary rather than the full case study.

🔴 Priority 3 — Resolve the Part 2 implementation ambiguity

This is important.

The repository describes Part 2 as a **proposed evolution**, and the Part 3 implementations are conceptual demonstrations.

That is internally consistent.

However, the faculty instructions describe Part 2 in terms of actual evolution/change records, including:

```
Plain text

Change1
Change2
Change3
requirement
diff
test result
```

Your repository does **not** currently contain an actual modified Rails application or three concrete change snapshots.

So you need to decide how you are presenting the work:

Option A — safest academically

Explicitly state in the report:

> Part 2 is a design-level evolution study rather than an executable Rails refactoring of the original repository.

Then explain that the four requirements were analyzed and the resulting abstractions were implemented conceptually in Part 3.

Option B — stronger submission

Actually create three reproducible change snapshots/branches/commits against the Rails application and record:

```
Plain text

Change 1 — Breakfast
Change 2 — Airport transfer / room upgrade
Change 3 — Cancellation policy
```

with diffs and tests.

Given your current repository, **Option A is what the existing evidence supports**. Don't imply that an actual Rails refactor occurred when it didn't.

🔴 Priority 4 — Add explicit LLM verification metadata

Your LLM folder is good, but the faculty template expects things such as:

```
Plain text
model_and_date
prompt_file
input_file
output_file
evaluation
verification_file
```

Your repository currently has the prompts/results themselves, but not a dedicated `llm.csv`.

I would add:

```
Plain text
data/llm.csv
```

with one row for each of the six stages.

Example columns:

```
Plain text
part,model_and_date,prompt_file,input_file,output_file,evaluation,verification_file
```

This would make the LLM audit much easier for a reviewer.

🔴 Priority 5 — Improve evidence granularity in `evidence.csv`

Your `evidence.csv` has 12 rows, which is useful, but it compresses a fairly large amount of evidence into a small table.

For example, Template Method could have separate evidence rows for:

```
Plain text
CrudConcern#index
CrudConcern#new
CrudConcern#create
```

```
CrudConcern#edit
CrudConcern#update
CrudConcern#destroy
CrudConcern#base_class
CrudConcern#object_class
```

That would make the traceability stronger.

This is not a structural failure; it's an opportunity to make the evidence more professor-proof.

🔴 Priority 6 — Add the actual faculty/team information

The faculty template explicitly expects:

```
Plain text

Team
Roll number
Student name
Batch
Contribution
```

The current repository does not appear to contain a corresponding submission metadata section.

Before final submission, add this to the README/report:

```
Plain text

Team: <your team>
Student 1: <roll> <name> <batch>
Student 2: ...
Contributions:
- ...
```

Only include real information.

🔴 Priority 7 — Make testing evidence more explicit

The repository says all four Part 3 programs were executed, which is good.

But there is a distinction between:

```
Plain text
```

```
Part 3 execution verified
```

and:

Plain text

```
Original Rails application tested
```

The latter is **not established**.

Your report should explicitly say:

Plain text

```
Original Rails application:
Execution/test status: not run / not independently verified

Part 3 implementations:
Java: executed successfully
Python: executed successfully
JavaScript: executed successfully
C++: compiled and executed successfully
```

That distinction is already reflected in the analysis, and it should remain visible in the final academic report.

One more thing I would add

Add a `SUBMISSION_CHECKLIST.md`

This isn't required by the faculty template, but it would make the repository extremely easy to review.

Something like:

Plain text

```
SUBMISSION_CHECKLIST.md

[✓] Source repository identified
[✓] Immutable baseline commit recorded
[✓] Baseline architecture documented
[✓] Template Method verified
[✓] Uncertain/rejected patterns documented
[✓] Breakfast requirement analyzed
[✓] Airport transfer analyzed
```

```
[✓] Room upgrade analyzed
[✓] Cancellation Strategy analyzed
[✓] Java implementation
[✓] Python implementation
[✓] JavaScript implementation
[✓] C++ implementation
[✓] Execution outputs preserved
[✓] UML diagrams
[✓] CSV data
[✓] LLM results 01–06
[⚠] Prompt 02 verbatim recovery
```

This makes the submission state transparent.

Final checklist

🔴 Must finish

- [ ] Recover and replace `prompt-02-architecture.md` with the **actual complete prompt**
- [ ] Recover and replace `prompt-03-pattern-discovery.md` with the **actual complete prompt**
- [ ] Expand the academic PDF so it reflects the **full case study**, not only a condensed narrative
- [ ] Explicitly state whether Part 2 is **design-level evolution** or an actual Rails implementation; based on the current repo, it should be presented as design-level

🟠 Strongly recommended

- [ ] Add `data/llm.csv`
- [ ] Add explicit student/team/contribution information
- [ ] Expand evidence granularity in `data/evidence.csv`
- [ ] Make baseline-vs-Part-3 testing status explicit in the PDF
- [ ] Add a repository-level `SUBMISSION_CHECKLIST.md`
- [ ] Add a clear Part 2 before/after/change narrative aligned with the faculty template

🟢 Already done

- [x] Correct GitHub repository
- [x] Correct `main` branch
- [x] Exact baseline commit
- [x] Correct baseline Template Method conclusion
- [x] Correct evolved Strategy conclusion
- [x] Pattern inflation avoided
- [x] Java implementation
- [x] Python implementation
- [x] JavaScript implementation
- [x] C++ implementation

- ☑ Captured outputs
- ☑ `patterns.csv`
- ☑ `evidence.csv`
- ☑ `metrics.csv`
- ☑ Five UML files
- ☑ Full result-01 through result-06 files
- ☑ No compiled binaries or obvious junk files
- ☑ README is internally consistent

Overall

Current state: ~80–90% submission-ready.

The **technical/design portion is in good shape**. The remaining risk is primarily that the submission could be judged as an abbreviated research artifact because the academic report and LLM-record metadata do not yet fully mirror the faculty's requested evidence structure.

I would fix the **four must-finish items first**, especially the report and exact Prompt 02/03 records.

Sunday 6:26 PM

Pasted text(1).txt
Document

Worked for 5m 43s

Completed the final assembly and verification pass.

A. Final ZIP

Download the complete Design Patterns Case Study ZIP

Generated path:

`/mnt/data/Design_Patterns_Case_Study_TEAM_REPLACE_ME.zip`

The package follows the faculty-required team-folder organization, including the report, LaTeX source, data CSVs, Part 1/2/3 evidence, UML source/images, LLM records, tests, and audit manifest.

DP_Csestudy_instruct(1)

Verified

- ZIP integrity: **PASS**
- ZIP entries: **107**
- Compiled binaries/caches/secrets: **none**
- PDF: **5 pages, A4**
- LaTeX compilation: **PASS**
- UML sources: **5** `.puml`

- UML images: **5 `.png`**
- Official CSV headers: **PASS**
- Part 3: **8 snippets executed**
- Part 3 test result: **1/1 for every snippet**
- Timing: **1 warmup + 5 measured runs**, median recorded as required by the faculty methodology.

  DP_Csestudy_instruct(1)
- Ruby Part 2 source syntax checks: **all PASS**

B. What was fixed

Part 1

- Official `patterns.csv`
- Official `pattern_counts.csv`
- Evidence-backed **Template Method only**
- Concrete participants:
  - `CrudConcern`
  - `BookingsController`
  - `RoomsController`
  - `SettingsController`
- Rejected/uncertain candidates documented instead of inflating the pattern count.
- Editable UML + PNG.

Part 2

The major previous weakness was corrected.

Actual Rails-oriented implementation now exists for:

Change 1 — Additional Services

- `AdditionalService`
- `BreakfastPackage`
- `AirportTransfer`
- `BookingService`
- persistence migration
- pricing extraction

Change 2 — Room Upgrade

- `RoomUpgrade`
- availability checking
- price-difference calculation
- room reassignment
- total recalculation

- persistence migration

Change 3 — Cancellation Strategy

- `CancellationService`
- `CancellationPolicy`
- `FlexibleCancellationPolicy`
- `PartialRefundCancellationPolicy`
- `NoRefundCancellationPolicy`

So Strategy is now supported by actual delegation/polymorphism rather than merely a class named `Policy`.

Part 3

Both confirmed patterns are implemented independently in:

- Java
- Python
- JavaScript
- C++

with the same behavioral specifications.

Measured medians are included in `data/cross_language.csv`:

Language	Template Method	Strategy
Java	39.494 ms	50.372 ms
Python	672.008 ms	595.258 ms
JavaScript	21.401 ms	21.762 ms
C++	1.097 ms	1.223 ms

These are actual local measurements, not estimates.

LLM evidence

- Prompt 01 retained.
- Prompts 04–06 retained.
- Prompt 02/03 **were not fabricated**.
- Their files explicitly state that the original verbatim interaction could not be recovered.
- Historical model/date information is marked unknown where it could not be verified.
- `llm.csv` uses the official schema.

Report

Created:

- `TEAM_REPLACE_ME_Report.tex`
- `TEAM_REPLACE_ME_Final_Report.pdf`

The PDF was compiled and visually inspected. It contains architecture, Part 1 evidence, Part 2 evolution, Part 3 comparison, measurements, LLM verification, findings and references.

C. Final folder structure

Plain text

```
TEAM_REPLACE_ME/
├── README.md
├── SUBMISSION_CHECKLIST.md
├── TEAM_REPLACE_ME_Report.tex
├── TEAM_REPLACE_ME_Final_Report.pdf
├── ZIP_CONTENT_MANIFEST.txt
├── FINAL_TREE.txt
│
├── data/
│   ├── patterns.csv
│   ├── pattern_counts.csv
│   ├── patterns_evolution.csv
│   ├── cross_language.csv
│   └── llm.csv
│
├── code/
│   ├── README.md
│   └── app/
│       ├── models/
│       ├── controllers/
```

D. What cannot honestly be marked PASS

There are **three remaining user-input items** in `SUBMISSION_CHECKLIST.md`:

1. **Replace `TEAM_REPLACE_ME`** with the actual team name.
2. Add the actual **roll numbers, names, batches and contributions**.
3. The complete upstream Rails repository source is not physically reproduced in `code/`; the verified baseline is identified by its immutable public commit, while the locally packaged source covers the files directly used by the analysis/evolution.

There is also no actual team slide PDF available, so I did **not** fabricate one.

One important caveat

The environment available to me has Ruby 3.3.8 but **does not have Rails/Bundler installed**. Therefore I could syntax-check the new Rails code, but I cannot truthfully claim that the complete Rails

application or RSpec integration suite was executed. The ZIP explicitly records this limitation rather than manufacturing a test result.

So the package is **assembled, audited, and substantially complete**, but I would **not** falsely label it 100% submission-ready until those personal/team metadata and source-archive issues are resolved.

Pasted text(2).txt
Document

Worked for 3m 50s
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analyzed
Analysis errored
Analyzed
Analyzed
Analyzed
Analyzed

Report generated — Team A8

I used the uploaded master prompt as the authoritative specification, including the exact Team A8/student information and required final deliverables. Pasted text The required output was specifically **`A8_Report.tex`** and **`A8_Final_Report.pdf`**. Pasted text

Files

1. LaTeX source

Download `A8_Report.tex`

2. Final compiled PDF

Download `A8_Final_Report.pdf`

Optional — complete A8 submission package

Download `A8_Design_Pattern_Case_Study_Submission.zip`

Report details

- **Pages:** 25
- **Team:** A8
- **Programme:** B.Tech Artificial Intelligence & Data Science
- **Students:** all four names/roll numbers supplied in the master prompt
- **Title:** *Design Pattern Analysis and Evolution in a Ruby on Rails Hotel Booking Application*
- **Baseline:** `5e15551ee646871b4bed32c982fa17da319d3ee3`
- **Confirmed baseline pattern:** Template Method
- **New evolved pattern:** Strategy
- **Part 3:** Java, Python, JavaScript, C++
- **Figures:** 6
- **Tables:** 14

The report follows the requested structure: application/architecture, Part 1 discovery, Part 2 evolution/refactoring, Part 3 cross-language comparison, LLM verification, findings/contributions, references and appendices. This directly follows the faculty-oriented structure specified in the prompt.

Pasted text

Validation

Check	Result
LaTeX compilation	**PASS**
PDF generated from supplied `.tex`	**PASS**
PDF pages	**25**
PDF visual inspection	**PASS**
Blank pages	**None**
Placeholder text	**None**

Check	Result
Figures	**PASS — 6**
Tables	**PASS — 14**
Team information	**PASS**
Pattern consistency	**PASS**
CSV/timing consistency	**PASS**
Part 3 results	**8/8 snippets passed**
Timing values	**Cross-checked against CSV**
LaTeX warnings/overfull boxes	**0**

The final validation specifically checks the items requested by the master prompt, including compilation, figures, tables, team information, pattern consistency and CSV consistency. Pasted text

Important unresolved items

I have **not** claimed 100% submission completion because two external requirements remain environment-dependent:

1. The package contains the **verified source subset used for the analysis**, but not a complete physical export of the entire upstream Rails repository.
2. No actual faculty/team slide PDF was available, so none was fabricated.

Also, the report transparently states that the complete Rails/RSpec integration suite could not be executed in the available environment because Rails/Bundler was unavailable. The Part 3 implementations, however, were actually executed and measured.

Most importantly, **the report itself contains no fabricated student information, pattern instances, timings, historical LLM metadata, or test results**. This follows the master prompt's explicit “do not invent” requirement. Pasted text ‹/›

A8_Report.tex
File

A8_Final_Report.pdf
PDF